WAINLUX K6 Web abstraction on Pi Zero W

Table of Contents

1. Summary . . . 6
   1.1. Clean-Room Statement . . . 7
2. WARNINGS . . . 7
3. Repository Guides . . . 7
4. Wainlux K6 USB Interface . . . 7
   4.1. Card Placement and Reuse . . . 8
   4.2. Architecture Diagrams . . . 10
   4.3. Getting Started . . . 15
      4.3.1. For Users . . . 15
         4.3.1.1. Deploy to Pi . . . 15
      4.3.2. For Developers . . . 15
         4.3.2.1. Read Code . . . 15
         4.3.2.2. Build and Test the Rust Service . . . 15
         4.3.2.3. Analyze a Run Recording . . . 16
         4.3.2.4. Phone-Driven Audio Sweep and Upload . . . 17
         4.3.2.5. Generate Diagrams . . . 18
      4.3.3. For Documentation Writers . . . 18
         4.3.3.1. Add Diagram . . . 18
      4.3.4. For Contributors . . . 19
         4.3.4.1. Clone Repository . . . 19
         4.3.4.2. Follow Style . . . 19
      4.3.5. Quick Commands . . . 19
   4.4. Reading and Changing the Software . . . 19
      4.4.1. Follow One Attempt . . . 19
      4.4.2. State Carries Its Evidence . . . 20
      4.4.3. One Program, One Runner . . . 20
      4.4.4. Make a Change Locally . . . 21
      4.4.5. Tests Explain the Contract . . . 21
      4.4.6. Interfaces Are Clients . . . 21
      4.4.7. Boundaries Still to Complete . . . 22
   4.5. K6 Raster Protocol . . . 22
      4.5.1. Link and Coordinates . . . 22
      4.5.2. Response Bytes . . . 22
      4.5.3. Commands Used by the Driver . . . 23

4.5.4. JOB_HEADER (0x23) . . . 24
4.5.5. DATA (0x22) . . . 25
4.5.6. Framing, Stop and Status Diagrams . . . 25
4.5.7. Evidence Limits . . . 27
4.6. Hardware Observations . . . 27
4.6.1. Current K6 Profile . . . 27
4.6.1.1. Backing Board Power Sweep . . . 28
4.6.2. Executed Rust Ladder Tests . . . 28
4.6.3. 9b: Height and Gap Comparison . . . 32
4.6.4. Card Result . . . 33
4.6.5. Historical Cork and PLA Close-ups . . . 34
4.6.6. Historical QR Run . . . 36
4.7. NFC Cards and Recovery Backups . . . 37
4.7.1. Reader/Writer Integration . . . 37
4.7.2. Synthetic Demo . . . 37
4.7.3. Capacity . . . 38
4.7.4. Make The Card . . . 38
4.7.5. Recover . . . 39
4.7.6. Protect . . . 39
4.7.6.1. Theft And Password Separation . . . 39
4.7.6.2. Workflow Leaks . . . 39
4.7.6.3. Loss And The Paper Backup . . . 40
4.7.6.4. Optional Access Protection And Encryption . . . 40
4.7.7. Security Review . . . 40
4.7.7.1. Local Files . . . 41
4.7.8. Workflow . . . 41
4.8. Burn Timing . . . 42
4.8.1. Current Report . . . 42
4.8.2. Historical Measurements . . . 43
4.8.3. Audio Sync and Candidate Motion . . . 43
4.9. Next Steps . . . 44
4.9.1. Optional Hardware . . . 44
4.9.1.1. NFC read/write module . . . 44
4.9.1.2. CYD operator display . . . 47
4.9.1.3. Audio/display HAT . . . 48
4.9.2. Supervised power sweep . . . 48
4.9.3. Protocol investigations . . . 49
4.9.4. QR experiments . . . 49
Appendix B: Architecture Decisions . . . 50
Appendix A: ADR-002: K6 Serial Transport . . . 50
4.B.A.1. Status . . . 50

4.B.A.2. Decision . . . 50
4.B.A.3. Consequences . . . 50
4.B.1. ADR-006: Clean-Room Protocol Work . . . 50
4.B.1.1. Status . . . 50
4.B.1.2. Decision . . . 51
4.B.1.3. Consequences . . . 51
4.B.2. ADR-007: AsciiDoc . . . 51
4.B.2.1. Status . . . 51
4.B.2.2. Decision . . . 51
4.B.2.3. Consequences . . . 51
4.B.3. ADR-010: Bytefield Protocol Diagrams . . . 51
4.B.3.1. Status . . . 51
4.B.3.2. Decision . . . 51
4.B.3.3. Consequences . . . 51
4.B.4. ADR-013: Simulator Testing . . . 52
4.B.4.1. Status . . . 52
4.B.4.2. Decision . . . 52
4.B.4.3. Consequences . . . 52
4.B.5. ADR-019: HTTP Review Boundary . . . 52
4.B.5.1. Status . . . 52
4.B.5.2. Decision . . . 52
4.B.5.3. Consequences . . . 52
4.B.6. ADR-021: Rust Service . . . 52
4.B.6.1. Status . . . 53
4.B.6.2. Decision . . . 53
4.B.6.3. State and lifetime . . . 53
4.B.6.4. Consequences . . . 53
4.B.7. ADR-022: ARM Packaging . . . 53
4.B.7.1. Status . . . 53
4.B.7.2. Decision . . . 53
4.B.7.3. Consequences . . . 54
4.B.8. ADR-025: Card Placement . . . 54
4.B.8.1. Status . . . 54
4.B.8.2. Decision . . . 54
4.B.8.3. Consequences . . . 54
4.B.9. ADR-026: Network MCP Hub . . . 54
4.B.9.1. Status . . . 54
4.B.9.2. Decision . . . 55
4.B.9.3. Consequences . . . 55
4.B.10. ADR-027: Audio-First Evidence for Physical End Estimates . . . 55
4.B.10.1. Status . . . 55

4.B.10.2. Context . . . 55
4.B.10.3. Decision . . . 55
4.B.10.4. Evidence and Limits . . . 56
4.B.10.5. Consequences . . . 56
4.B.10.6. Staged Work . . . 57
4.B.11. ADR-028: Pi-Coordinated Capture and External Audio Analysis . . . 57
4.B.11.1. Status . . . 57
4.B.11.2. Context . . . 57
4.B.11.3. Decision . . . 57
4.B.11.4. Data Flow . . . 58
4.B.11.5. Initial Camera Feasibility Check . . . 60
4.B.11.6. Consequences . . . 61
4.B.11.7. Measured Upload Sizing . . . 61
4.B.11.8. MCP and Future Analysis . . . 61
4.B.11.9. HAT Follow-Up . . . 61
4.B.11.10. Alternatives . . . 61
4.B.11.11. Related . . . 62
4.B.12. ADR-029: Local Container Builds for Supply Chain Security . . . 62
4.B.12.1. Status . . . 62
4.B.12.2. Context . . . 62
4.B.12.3. Decision . . . 62
4.B.12.4. Consequences . . . 62
4.B.12.5. Rationale . . . 63
4.B.12.6. Implementation . . . 63
4.B.12.7. Container Inventory . . . 63
4.B.12.8. Examples . . . 64
4.B.12.9. Related Decisions . . . 65
4.B.12.10. Notes . . . 65
Appendix D: Requirements . . . 65
4.D.1. Functional . . . 65
Appendix C: FR-001: Reviewed Raster Execution . . . 65
4.D.C.1. Status . . . 65
4.D.C.2. Requirement . . . 65
4.D.C.3. Acceptance . . . 66
4.D.C.4. Scope . . . 66
4.D.2. FR-002: WiFi QR Composition . . . 66
4.D.2.1. Status . . . 66
4.D.2.2. Requirement . . . 66
4.D.2.3. Acceptance . . . 66
4.D.2.4. Evidence . . . 66
4.D.3. FR-003: Progress and Timing . . . 66

4.D.3.1. Status . . . 66
4.D.3.2. Requirement . . . 67
4.D.3.3. Acceptance . . . 67
4.D.4. FR-007: Stop and Cancellation . . . 67
4.D.4.1. Status . . . 67
4.D.4.2. Requirement . . . 67
4.D.4.3. Acceptance . . . 67
4.D.4.4. Attended Hardware Check (Initial Result) . . . 67
4.D.5. FR-009: Test Ladder . . . 68
4.D.5.1. Status . . . 68
4.D.5.2. Requirement . . . 68
4.D.5.3. Acceptance . . . 68
4.D.6. FR-011: Material Placement . . . 69
4.D.6.1. Status . . . 69
4.D.6.2. Requirement . . . 69
4.D.6.3. Acceptance . . . 69
4.D.7. FR-013: Hardware-Independent Tests . . . 69
4.D.7.1. Status . . . 69
4.D.7.2. Requirement . . . 69
4.D.7.3. Acceptance . . . 69
4.D.8. FR-015: Operator Test Verdicts . . . 69
4.D.8.1. Status . . . 69
4.D.8.2. Requirement . . . 70
4.D.8.3. Acceptance . . . 70
4.D.8.4. Full-Screen Operator UI Acceptance (Pending) . . . 70
4.D.9. FR-016: Reviewed Card Jobs . . . 70
4.D.9.1. Status . . . 70
4.D.9.2. Requirement . . . 70
4.D.9.3. Acceptance . . . 70
4.D.10. FR-017: Physical End Estimates . . . 72
4.D.10.1. Status . . . 72
4.D.10.2. Requirement . . . 72
4.D.10.3. Acceptance . . . 72
4.D.11. Non-Functional . . . 73
4.10. NFR-005: Redacted Run History . . . 73
4.10.1. Status . . . 73
4.10.2. Requirement . . . 73
4.10.3. Acceptance . . . 73
Appendix E: Images . . . 73
5. Docker Containers . . . 74
5.1. Components . . . 75

5.1.1. Laser Service API . . . 75
5.1.1.1. Timing . . . 75
5.1.1.2. Run Analysis With Optional Audio . . . 76
5.1.1.3. Audio Recording Sync . . . 76
5.1.1.4. Submit and Review . . . 78
5.1.1.5. Types . . . 79
5.1.1.6. Execute One Stage . . . 80
5.1.1.7. Trust and Lifetime . . . 80
5.1.1.8. Clear Pi and Account State . . . 80
5.1.1.9. Tests . . . 80
5.1.2. K6 MCP Preview Client . . . 81
5.1.2.1. Scope . . . 81
5.1.2.2. Run . . . 81
5.1.2.3. API . . . 82
5.1.3. Bytefield-SVG . . . 82
6. Scripts . . . 82
6.1. Commands . . . 82
6.2. Steam Deck Flatpak Host Commands . . . 82
7. Card CLI Client . . . 83
7.1. Build and Test . . . 83
7.2. Submit One Example . . . 83
Appendix F: Glossary . . . 84
Appendix G: Copyright and License . . . 93
G.1. Crawler Requests . . . 94
G.2. Clean Room Statement . . . 94
G.3. Copyright Notice . . . 95

Other formats: HTML | GitHub Repository

1. Summary

Source-available host software for personal, non-commercial operation of Wainlux K-series laser engravers using an independently developed protocol implementation. Project-owned material uses one non-commercial, no-AI-training license; see Copyright and License. These restrictions are not an open-source license.



This project is an independently developed, clean-room implementation created through analysis of publicly observable behavior and legally obtained software. It is not affiliated with or endorsed by Wainlux.

Is:

- Host-side control software

- Protocol implementation
- Interoperability tool

Is not:

- ~~Firmware replacement~~
- ~~Vendor software redistribution~~
- ~~Reverse-engineered source release~~

The Rust service serves the operator UI and API on port 8081, reviews card jobs, and controls the K6 over USB serial. See Laser Service API for the API and [build-and-test-the-rust-service] for deployment. The Card CLI Client only submits Wi-Fi, GitHub recovery and vCard content for operator review; the Pi generates all card rasters.

1.1. Clean-Room Statement

Project owner and decision authority: Sean Donnellan. Sean triggered and verified the physical K6 tests. Protocol behavior was established from packet captures, serial monitoring, black-box tests and public comparative material. GitHub Copilot performed isolated Ghidra analysis and separately assisted with implementation; only protocol descriptions were handed between those roles. No vendor source, binary, Ghidra project or analysis artifacts are distributed. This was tool-mediated role separation, not a formal two-human clean-room program. See the included ADR-006: Clean-Room Protocol Work decision for the implementation boundary.

2. WARNINGS



- Laser safety: **always** wear appropriate laser safety glasses.
- Fire hazard: do not run unattended; have a fire-safe surface.
  - Keep a fire extinguisher handy, or a fire blanket.
- Use a smoke detector.

3. Repository Guides

4. Wainlux K6 USB Interface

Current documentation for the Rust K6 service on Pi Zero W. Protocol observations are evidence, not descriptions of vendor code.

The service README documents the HTTP API, served at `GET /api`. OpenAPI 3.1 is at `GET /openapi.yaml`; Swagger UI is at `GET /docs`.

The canonical build and test command is `./scripts/build_laser.sh` from the repository root.

4.1. Card Placement and Reuse

The drawings use the current measured card, K6 working reach and template geometry. The QR symbol is a generic placeholder; exact private pixels are shown only in the live app preview.

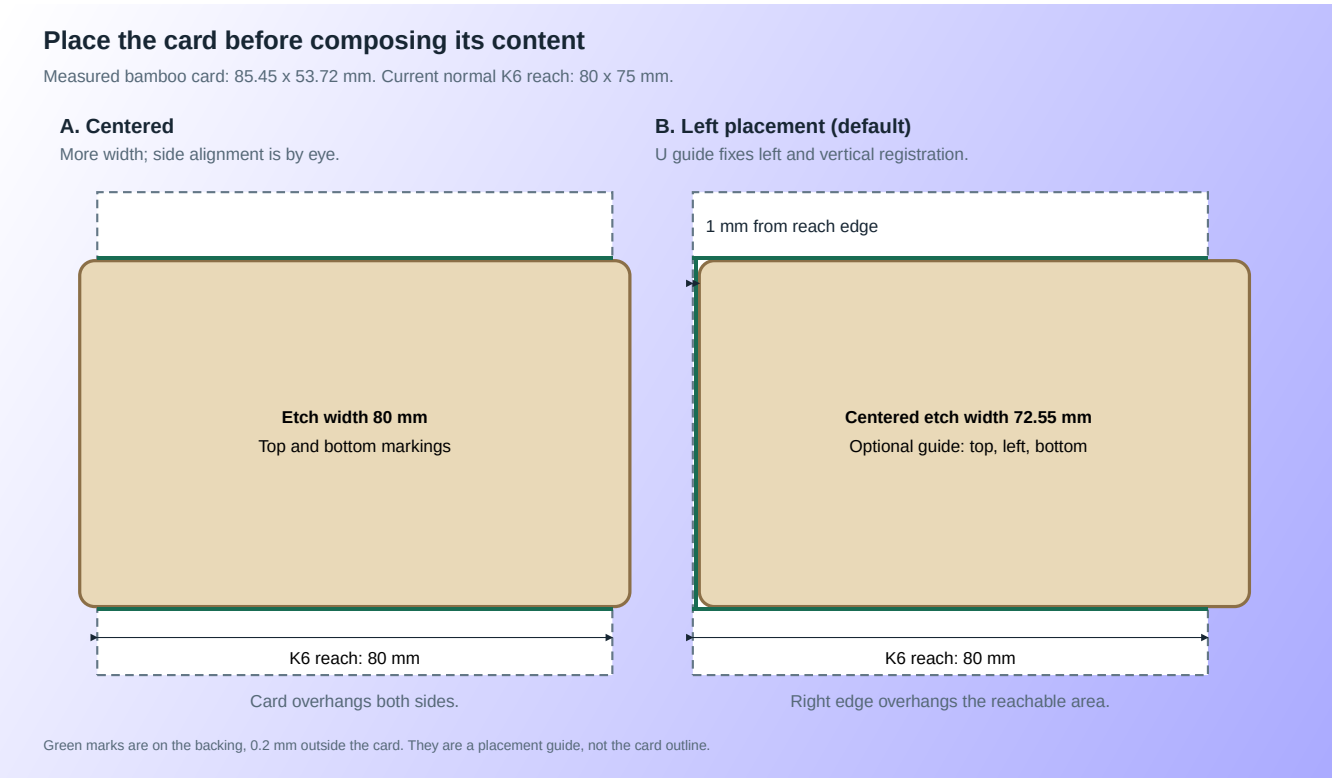


Figure 1. Choose a card placement before placing content

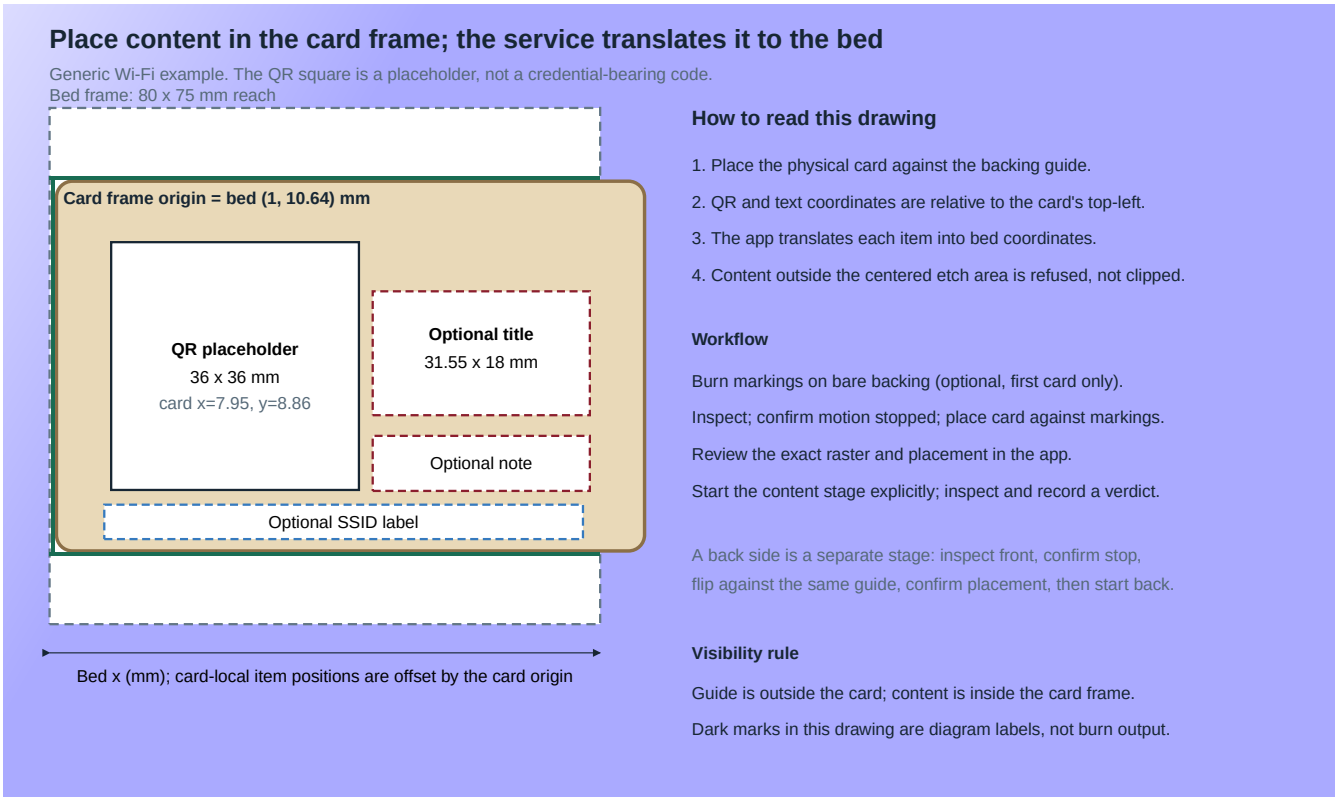


Figure 2. Card-local item coordinates are translated to the bed



Figure 3. First burn the U markings on bare backing; leave the card out

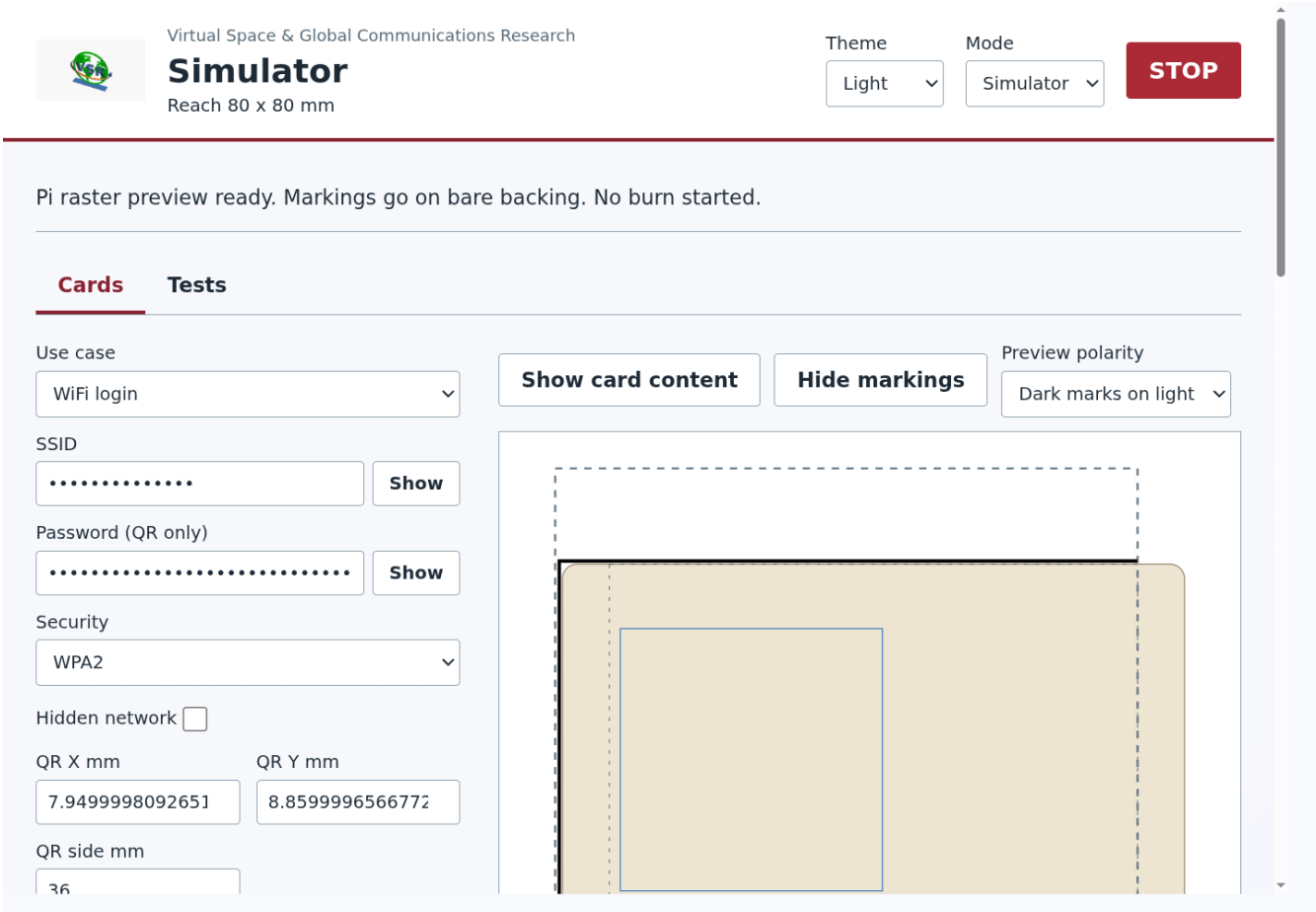


Figure 4. Rust operator UI preview using synthetic input in simulator mode; no burn started

For a new backing, place the bare backing and start the optional markings stage first. Confirm physical rest, inspect the markings and explicitly accept or record a deviation. Only then place the

card against those marks and acknowledge placement. Edit content in the primary form and positions under Layout and sizing; Review output requests the Pi's exact raster. A repeat card can reuse the same markings without burning them again. For a second side, inspect and accept the first side, confirm rest, flip the card into the same guide, confirm placement and start that stage separately. No verdict starts the next stage.

4.2. Architecture Diagrams

These diagrams explain how the running Rust service, its hardware/API boundary, external clients and the manually gated workflow fit together.

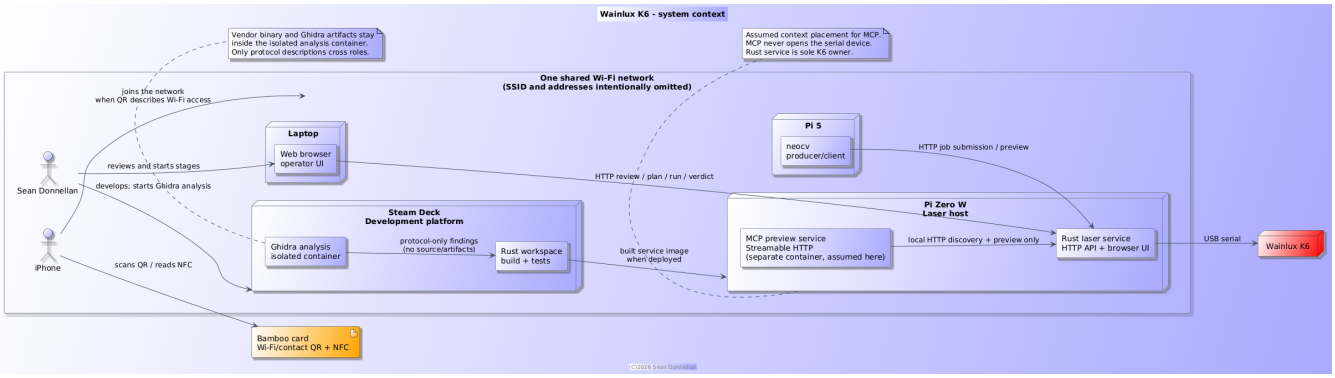


Figure 5. System context across development, production and client devices

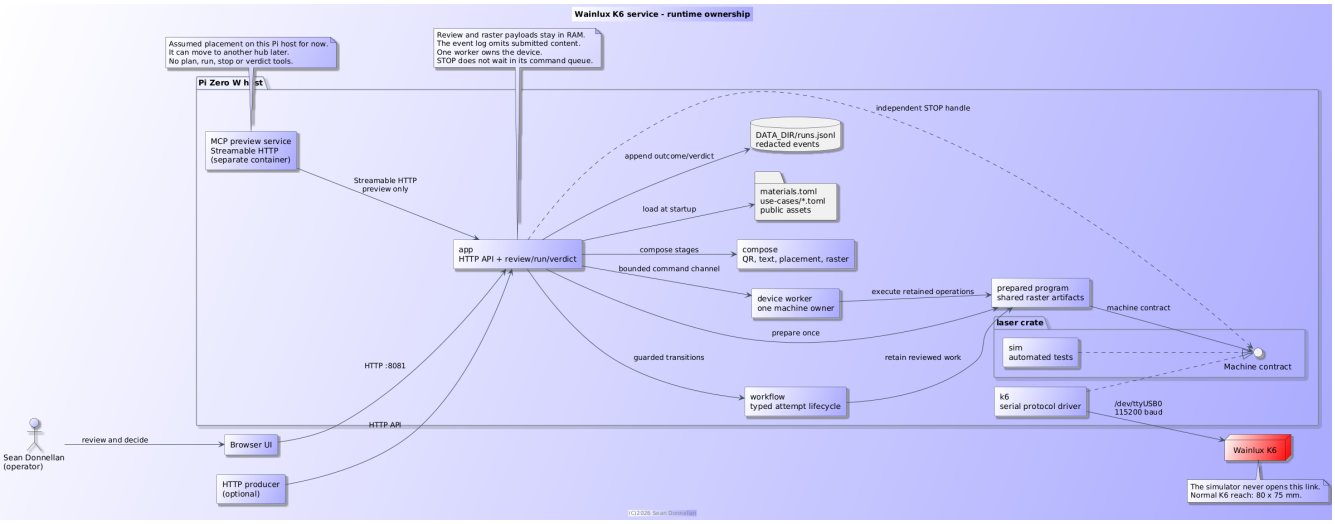


Figure 6. Runtime components and ownership

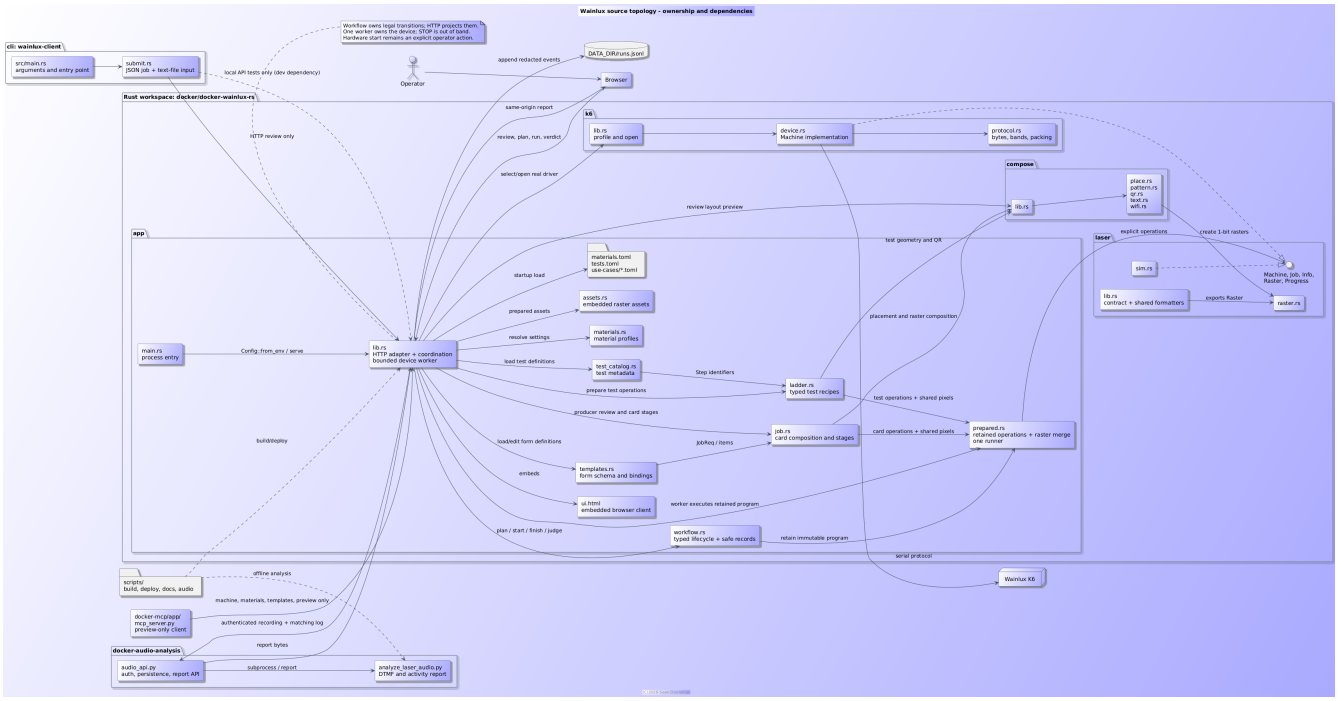


Figure 7. Source files, client paths and cross-feature dependencies

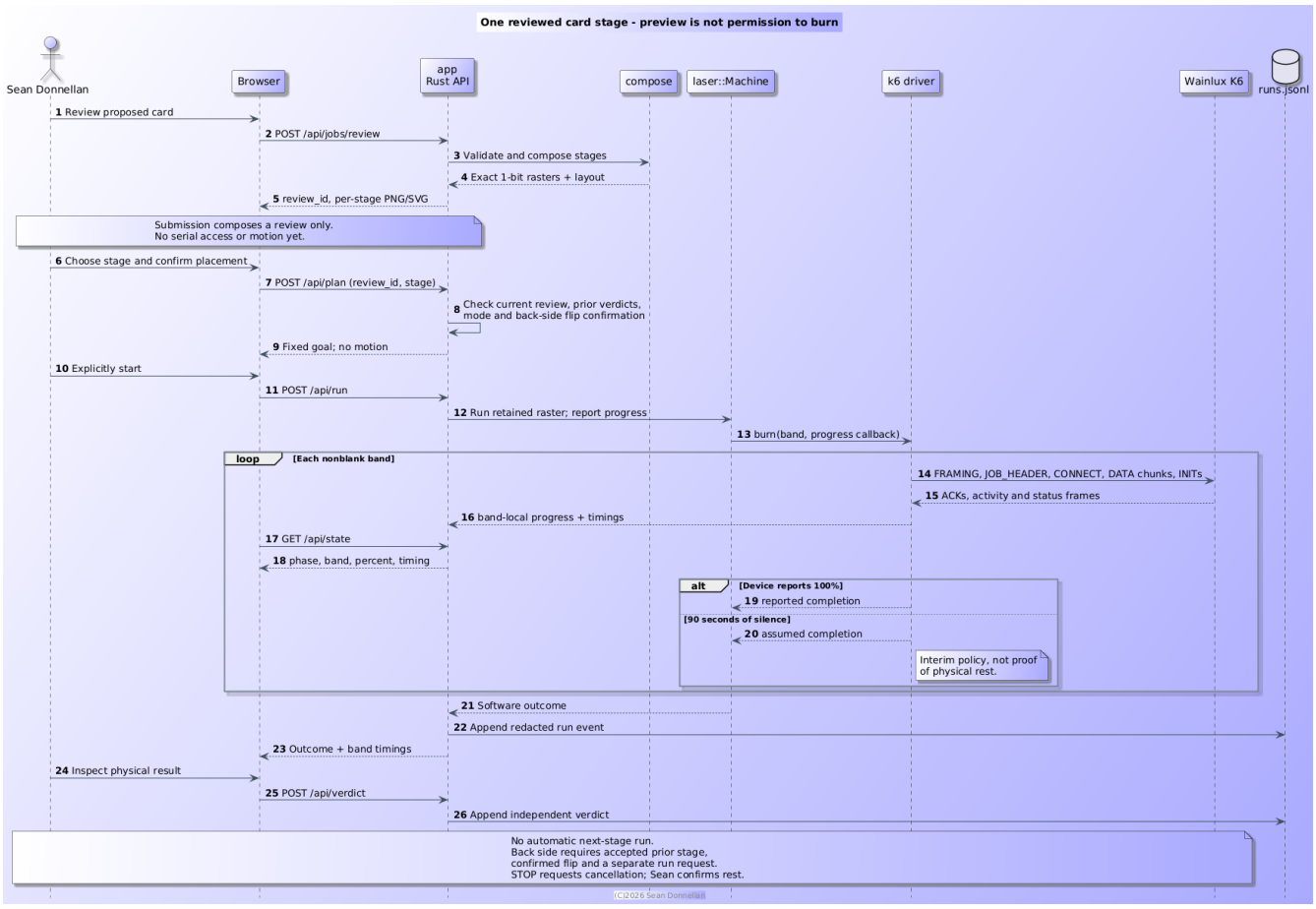


Figure 8. One reviewed stage from API submission through physical verdict

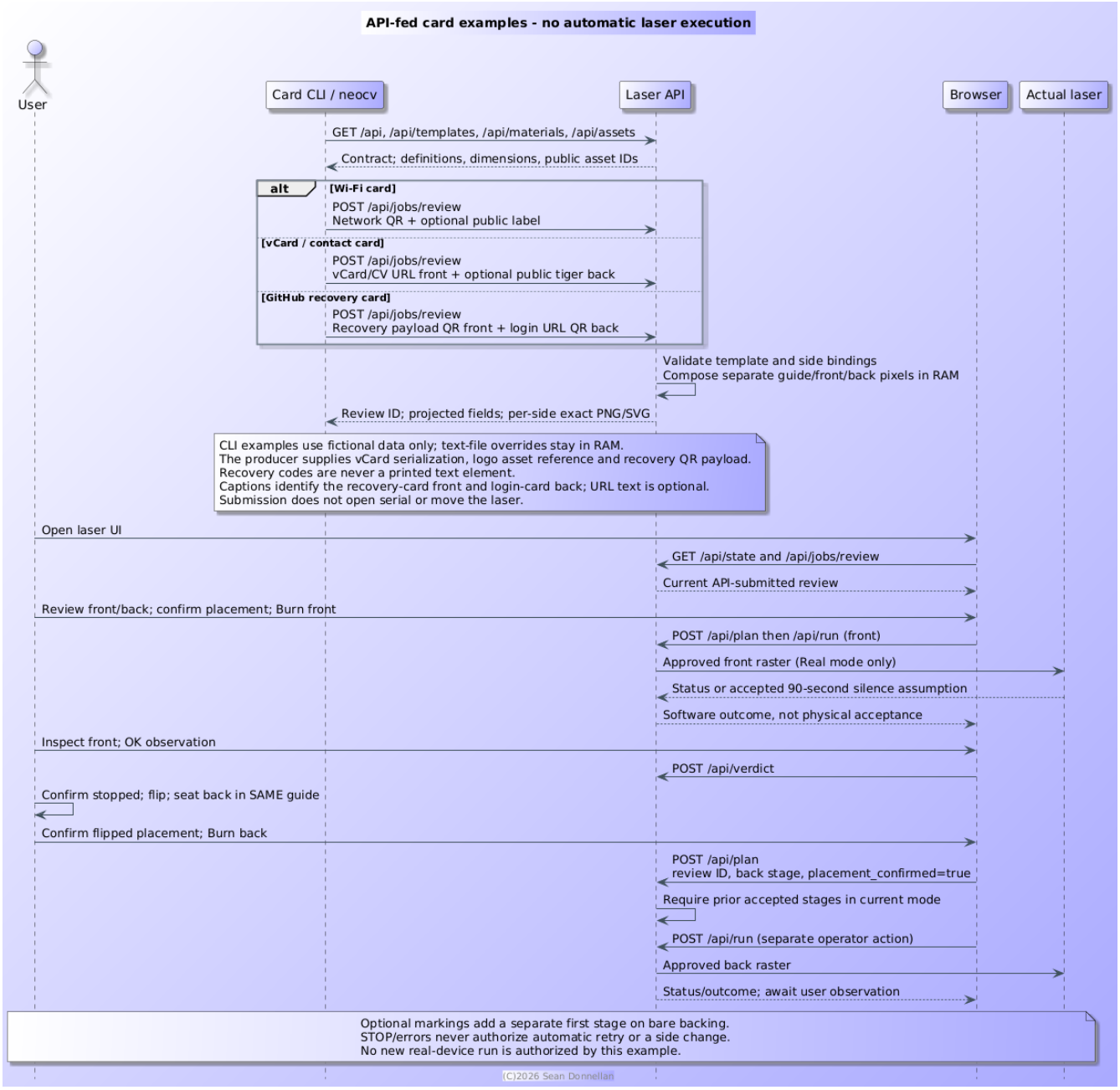


Figure 9. External clients and the operator UI share the same API

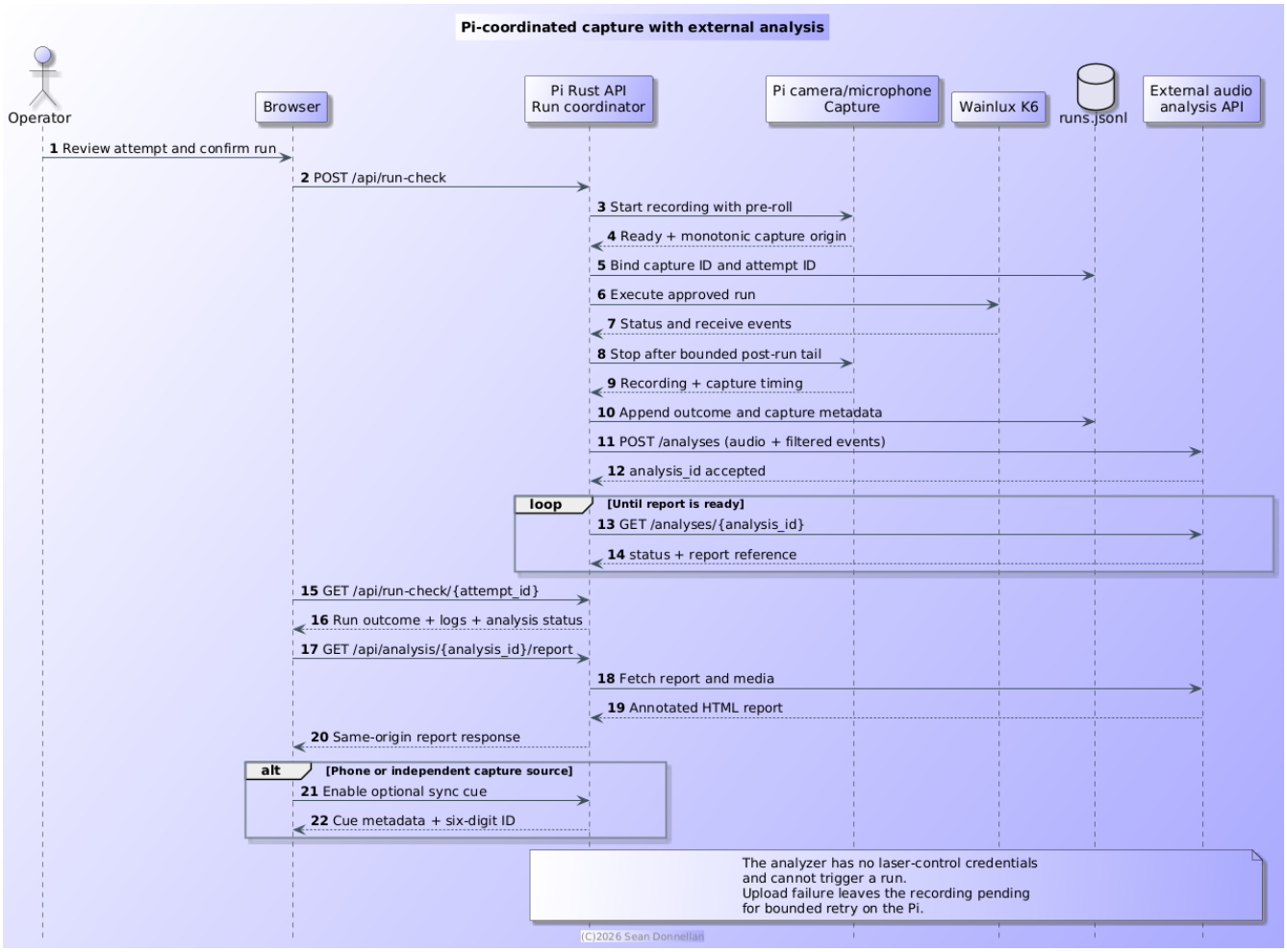


Figure 10. The Pi coordinates local recording and laser events while an external worker analyzes audio

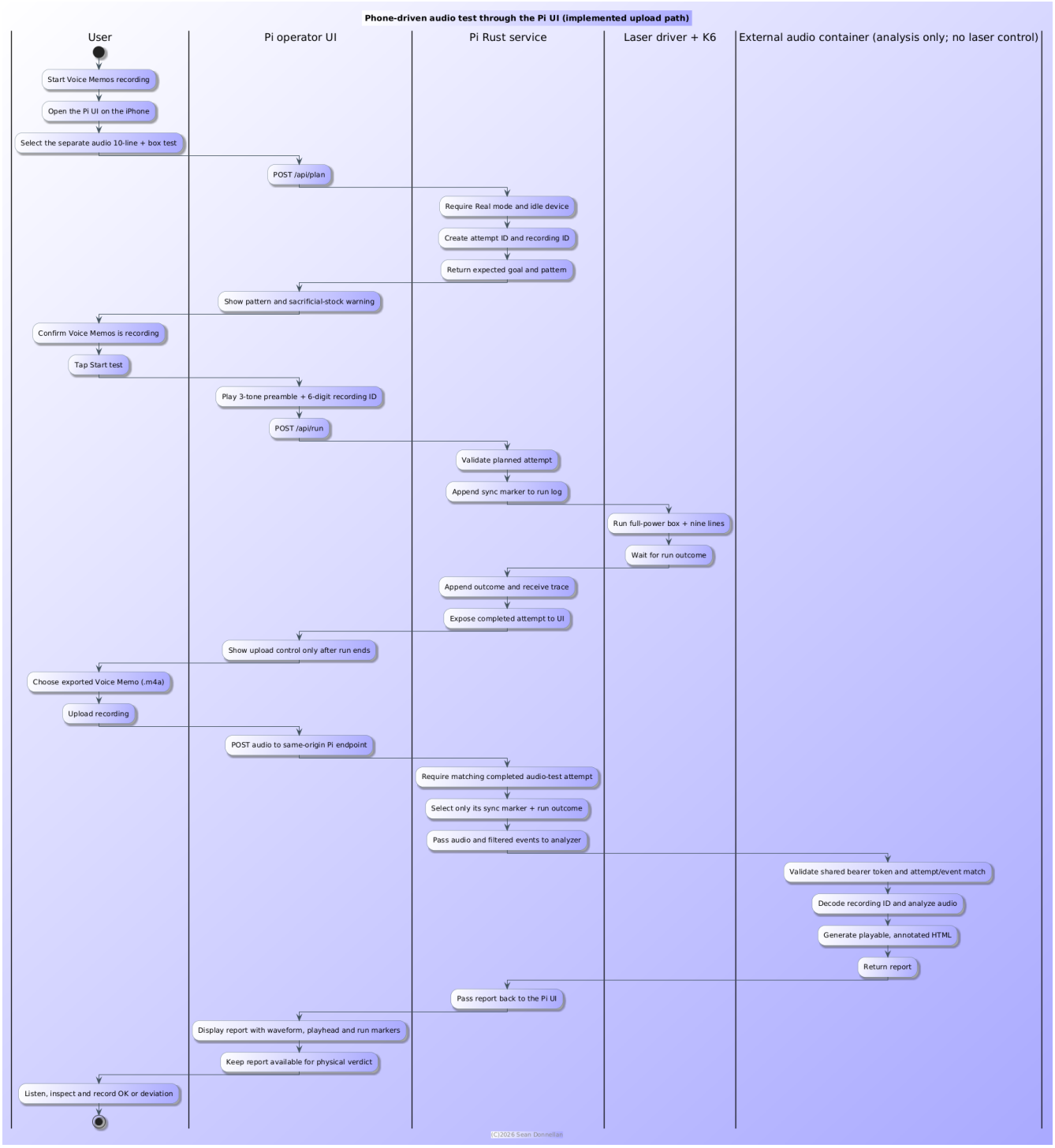


Figure 11. Phone-driven audio test and analyzer pass-through

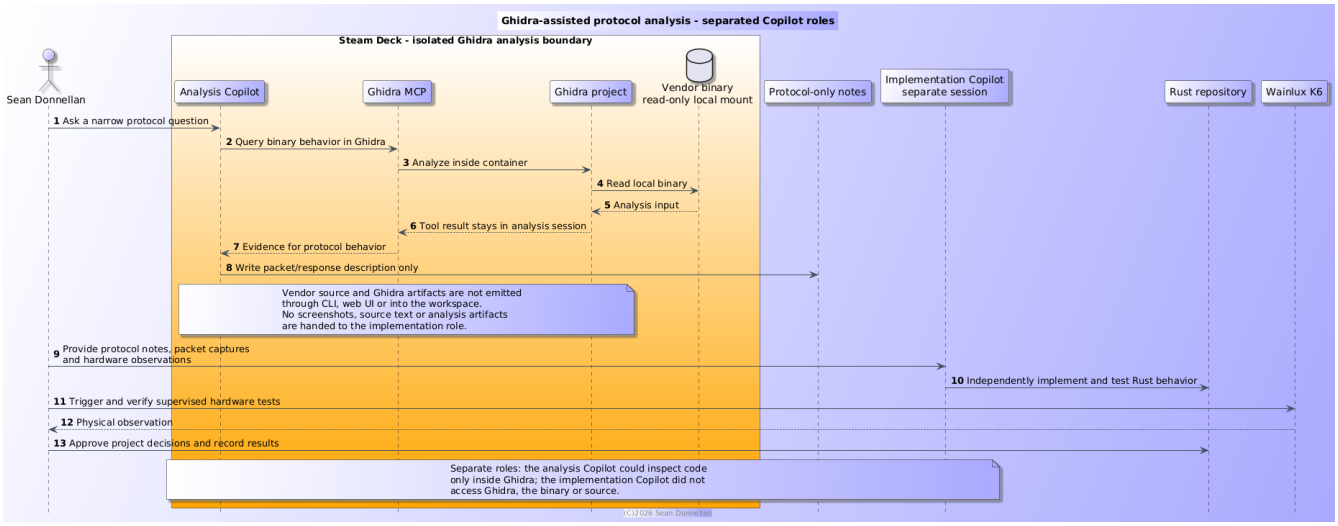


Figure 12. Clean-room Ghidra analysis and implementation handoff

4.3. Getting Started

Quick guide to using this repository.

4.3.1. For Users

4.3.1.1. Deploy to Pi

Start the preloaded ARM image

```
cd docker/docker-wainlux-rs
docker compose up -d
```

Access: `http://<pi-ip>:8081`. The ARM image must already be loaded on the Pi; this compose file does not build it.

4.3.2. For Developers

4.3.2.1. Read Code

- Service API and operator workflow
- K6 Raster Protocol for K6 serial behavior
- Card Placement and Reuse for the embedded operator UI preview

4.3.2.2. Build and Test the Rust Service

Build on the Steam Deck or another development workstation, not on the Pi Zero W. Run the one canonical build command from the repository root:

Test Rust, then build the Pi image

```
./scripts/build_laser.sh
```

The script runs `cargo test --workspace --locked` in a pinned Rust container, then builds the service Dockerfile for `linux/arm/v6` as `localhost/laser-app:arm`. The release binary is compiled for the Pi Zero W's `arm1176jzf-s` CPU; the container platform selects the matching ARMv6 runtime image. The Cargo registry and compiled host test artifacts persist in named volumes `wainlux-pi-cargo-registry` and `wainlux-pi-cargo-target`. Repeated tests reuse the target volume; the ARM release build is a separate target and is rebuilt when source changes invalidate the container build cache. The Cargo lockfile, target platform, test image, cross-compiler image and runtime image are fixed inputs. The result uses the same local image name as the Pi Compose file.

The script prefers a running Docker engine, then Podman; `CONTAINER_ENGINE=docker` or `CONTAINER_ENGINE=podman` selects one explicitly. In a Flatpak, it prefers `flatpak-spawn --host` so piped data such as an image stream reaches SSH correctly, then falls back to `host-spawn`; outside Flatpak it invokes the engine normally. Install and start Docker or Podman on the host. No host Rust toolchain is needed. It does not push, transfer, load, or deploy the image to the Pi.

The image digests are intentional pins. When updating them, verify the replacement manifests still include the build platform and `linux/arm/v6`, then rerun the build command and documentation checks.

4.3.2.3. Analyze a Run Recording

Analyze a phone recording without installing audio software on the workstation:

Match the recording to run logs and processing timings

```
./scripts/analyze_laser_audio.sh /path/to/recording.m4a \
    --attempt-id ATTEMPT-ID --expected-id DDDDDD
```

The script builds/reuses the pinned `docker/docker-audio-analysis` container, mounts the input and optional API log read-only, prints a JSON report, and writes the HTML report beside the recording. The HTML embeds a browser-compatible mono MP3 preview up to 40 MiB; larger reports write an MP3 sidecar. The analyzer decodes the optional DTMF recording ID only to associate it with the attempt, then shows the full recording waveform with time/amplitude zoom and seek, timestamp-aligned run events, and measured processing timings. When the run log includes a receive trace, each captured byte and decoded frame is shown on the waveform and in the packet table; an untraced or older run has no packet-level data to align. It does not classify audible activity or infer motion or laser state. Docker or Podman must be available on the host; Flatpak host routing is handled by the shared script helper.

The UI's **Machine tests** > **Archived exploratory audio report** opens an earlier self-contained example built from an actual 2 mm frame recording. Its sound-activity annotations are historical and are not generated by current uploads. The source recording is embedded in that archive; the old `Verification/` working folder is not needed.

For a manual iPhone Voice Memos recording, use Share in Voice Memos and save the `.m4a` to Files, then transfer it to the analysis host. Analyze it with the matching attempt and a sanitized API log export:

Analyze a manually imported iPhone recording

```
./scripts/analyze_laser_audio.sh /path/to/voice-memo.m4a \
    --run-log /path/to/sanitized-run-log.json \
    --attempt-id ATTEMPT-ID --expected-id DDDDDD
```

4.3.2.4. Phone-Driven Audio Sweep and Upload

For ordinary Real card or machine runs, the planned-run screen also offers an optional Voice Memos recording confirmation. Start the recording and check it before starting; the Pi plays the attempt ID cue and logs the association. After a completed marked run, upload is offered only if the optional analyzer is available. This does not block the burn, and logs/timings remain the primary record. Keep the original in Voice Memos if upload is unavailable.

The analyzer-gated audio test offers two separate choices over the same 20 mm box plus nine 19 mm lines: minimum nonzero power (1/1000, nominally no-mark but not guaranteed) and full power (1000/1000, marking expected). Each is a distinct, human-started run; the full-power comparison is never triggered automatically. Use sacrificial stock and keep the beam path clear for both. In Real mode, the UI requires confirmation that Voice Memos is recording, then plays the attempt ID cue before the human confirms the run. After completion, choose the iPhone-exported `.m4a`. The Pi forwards it with only the matching sync marker/run outcome and retains no audio or HTML. The analyzer stores audio, filtered events, and the report under `docker/docker-audio-analysis/data/ATTEMPT-ID/`. It has no restart policy: stopping it releases resources and hides both audio profiles/report controls, while ordinary burns continue normally.

The measured 50.15-second sample was 836,997 bytes. A successful Wi-Fi card content run lasted 2,054 seconds; at the observed rate, with 36 seconds of recording margin, the estimated file is about 33.3 MiB. The matching Pi/analyzer upload cap is 64 MiB, roughly 1.9x headroom. Actual Voice Memos bitrate and duration may vary.

Run the analyzer on the Deck only when tuning or reviewing test evidence. Keep the shared bearer token out of source control and restrict the analyzer port at the host firewall to the Pi:

Start the external analyzer on the workstation

```
export AUDIO_ANALYZER_TOKEN='set-the-same-private-token-on-both-hosts'
export AUDIO_ANALYZER_BIND_ADDRESS=0.0.0.0
podman compose -f docker/docker-audio-analysis/compose.yaml up -d --build
```

In the Pi's `docker/docker-wainlux-rs/.env`, set `AUDIO_ANALYZER_URL` to `http://WORKSTATION-LAN-IP:8090/api/analyze` and set `AUDIO_ANALYZER_TOKEN` to the same secret. Keep `.env` private. Recreate the Pi service after setting these values. The analyzer's port is loopback-only by default; only bind it to the LAN after firewalling access to the Pi. When analysis is no longer needed, stop the optional container:

Stop the analyzer without affecting the Pi laser service

```
podman compose -f docker/docker-audio-analysis/compose.yaml down
```

Its bind-mounted data remains on the Deck and is available when the container is started again. With it down, the Pi hides the audio-only test and report features. Standard test and burn controls are unaffected.

If the Pi proxy or analyzer is unavailable, the separate CLI workflow above remains usable after transferring the `.m4a` and a sanitized matching run-log export to the analysis host.

To move the finished image from the Deck to the original Pi Zero W, first make sure no job is running and no in-memory review needs to be kept. Then explicitly deploy:

Transfer, load, and restart the Pi container

```
./scripts/deploy_laser.sh --target sean@piz-k6.local --confirm
```

Replace the example host with the SSH name or IP for the intended Pi, or set `LASER_SSH_TARGET` in the environment. The script refuses unless the remote model is Raspberry Pi Zero W Rev 1.1 (`armv6l`) and the service API reports idle, Simulator mode, no attempt, no outcome, and no loaded review. It streams the already-built ARM image from the Deck, verifies the loaded image ID matches, and recreates `laser-app` through the Pi's Compose file. SSH host-key verification remains enabled. The Pi only loads and runs the image; Rust compilation stays on the Deck. The script never starts a laser job.



The Pi's configured hostname is `piz-k3`, but it is also reachable via `piz-k6.local` (mDNS) and `piz-k6.fritz.box` (router DNS). YMMV depending on your network setup. SSH config may make one name work while others fail.

4.3.2.5. Generate Diagrams

Render the documentation diagrams

```
plantuml documentation/images/*.plantuml
```

Output: `documentation/images/*.png`

4.3.3. For Documentation Writers

4.3.3.1. Add Diagram

Create `.plantuml` file in `documentation/images/`:

Minimal source for a PlantUML diagram

```
@startuml

actor User
User --> System
@enduml
```

Render a new PlantUML diagram

```
plantuml images/new-diagram.plantuml
```

Reference the rendered diagram from AsciiDoc

```
\image::documentation/images/new-diagram.png[Description]
```

4.3.4. For Contributors

4.3.4.1. Clone Repository

Clone the repository

```
git clone <repo-url> wainlux-pi
cd wainlux-pi
```

4.3.4.2. Follow Style

- DRY - Don't Repeat Yourself
- KISS - Keep It Simple, Stupid
- Hemingway - Short, direct sentences
- No fluff - Essential info only

4.3.5. Quick Commands

Run documentation and Rust checks

```
# Check documentation
./scripts/test_docs.sh

# Test Rust and build the local ARM image
./scripts/build_laser.sh
```

4.4. Reading and Changing the Software

The design separates decisions from effects. A review does not move hardware, a completed operation is not a human verdict, and a transport request does not own the machine. These distinctions matter more than the number of source files.

4.4.1. Follow One Attempt

1. Startup loads materials, templates and the test catalog. TOML owns configured values; the client does not calculate machine powers or raster content.
2. `job.rs` composes card stages; `ladder.rs` prepares test recipes. `prepared.rs` represents explicit

operations with resolved settings and shared raster pixels. Normal burns and calibration are different operations.

3. `workflow.rs` admits a plan only while idle. The retained attempt contains its operational record and private prepared work; JSON is a transport projection, not the state machine.
4. An explicit start changes Planned to Running. The browser sends the displayed attempt ID, so a stale page cannot start a different plan accidentally.
5. One device worker owns the machine for its entire lifetime. It receives commands through a bounded channel and runs the retained program without recomposing its pixels. STOP uses its separate handle and cancellation latch, not the ordinary command queue.
6. Execution reports an outcome, and the workflow waits for a verdict. Only a software-successful run with an operator OK accepts a card stage. Each side still requires a separate reviewed start.

4.4.2. State Carries Its Evidence

`Workflow` has four alternatives: Idle, Planned(attempt), Running(attempt), and AwaitingVerdict(attempt, outcome). There is no independent running flag that can disagree with an optional outcome. Its transitions know nothing about HTTP, clocks, files or USB. Rejected transitions leave the current attempt intact.

The review remains separate because it can survive several attempts. Changing a review or mode invalidates acceptance; it never silently authorizes another burn. The HTTP state response preserves the existing `attempt`, `running` and `outcome` fields and adds `phase` as a projection of this one authoritative state.

`AttemptRecord` deliberately has no request payload or raster field. Audit serialization uses that record, while `Attempt::view` adds the request projection needed by the operator. Do not solve privacy by adding another list of JSON keys to delete. Operator-entered observations remain audit data and should not contain credentials.

4.4.3. One Program, One Runner

`Prepared` owns a closed list of operations: Talk, Home, Pointer, Frame, Burn and Calibrate. Its constructor validates normal raster geometry and settings before device access. Raster data is shared through `Arc`, so retaining a review and a plan does not require copying all pixels. Execution stops at the first failed operation and checks cancellation between operations.

Card and test previews composite the same positioned raster artifacts used by execution. Frame previews describe a laser-off bounds operation, not a burn. A preview is not a simulation of physical darkness or a guarantee of placement. Calibration remains explicitly bounded by the driver's firmware envelope; it must not enlarge normal working reach.

The device worker catches unwinding execution panics, drops uncertain device handles and records failure rather than leaving Running set forever. This does not recover an abort, power loss or hardware fault, and it does not prove physical rest. No software STOP path is a certified emergency stop.

4.4.4. Make a Change Locally

- A new card type belongs in a use-case TOML file plus synthetic input/API tests. Do not add a renderer to the CLI or browser.
- A new test recipe prepares existing operation types and states its goal. Do not add another device execution loop or hand-drawn raster preview.
- A new physical operation extends the closed operation type and its runner, with an explicit cancellation/failure test. It is not a string command interpreted from TOML.
- A new driver implements the `laser` contract. Keep packet details and measured device quirks out of workflow transitions.
- A new transport calls the same workflow and preparation code. It must not bypass start, side acceptance or placement gates.

Small prepare-and-run helpers remain for isolated tests; production retains the prepared artifact before starting. Avoid introducing a plugin framework, event bus or generic persistence layer merely to rename these existing responsibilities.

4.4.5. Tests Explain the Contract

Pure workflow tests demonstrate rejected transitions, stale starts/completions, verdict rules and failed-stage acceptance. Prepared-program tests record the pixels actually sent to a fake machine, change the original parameters afterward, and inject a failure to prove later operations are not run. Real-HTTP simulator tests verify the same boundaries through the public API; UI tests check which attempt ID the browser sends.

Run `./scripts/build_laser.sh` and `./scripts/test_docs.sh` from the repository root. The build tests the CLI and service and creates the ARM image; it does not deploy. Simulator success is not hardware acceptance. Test STOP, placement, calibration and physical results separately under operator supervision.

4.4.6. Interfaces Are Clients

The operator UI has four visible phases: Prepare, Review and place, Run, Inspect. Its indicator projects server state; it is not permission to move hardware. Content is primary, while numeric layout/font controls and material/placement settings are disclosures. The exact pixels and full goal remain available without repeating the same preview twice on the main surface.

Review submission sits beside the input form; physical starts sit beside the reviewed output and placement acknowledgement. Guide inspection is explicit: accepting markings never starts the card, and clicking the next start never silently records the previous verdict. Front acceptance and back placement remain separate requirements enforced by the API.

Browser, CLI and machine clients use the same review/plan/start/verdict contract. Template fields declare `file_input=true` for editable multiline payloads; the browser renders this capability without knowing whether a field belongs to GitHub or vCard. Local file loading does not submit or start anything, and does not trim the file text. Privacy masks and independently hidden raster content remain presentation controls, not encryption.

STOP bypasses the browser's ordinary busy-action guard. A generation check prevents a pending plan or audio cue from continuing into a later start after STOP. This complements the server cancellation latch; it is not a hardware emergency-stop guarantee. Test both delayed-request paths rather than only testing whether a red button is clickable.

Workspace tabs support arrow keys and Home/End. New active phases are brought into view once, not on every poll. Progress stays scoped to the current operation or band, never relabelled as whole-card completion. Options contains appearance and session clearing; machine mode and STOP stay visible. A clear-session command must not be labelled sign-out while authentication does not exist.

4.4.7. Boundaries Still to Complete

Authentication, authorization, transport protection and session ownership are not implemented. An optional `attempt_id` is a stale-request check, not an access credential; legacy clients may still omit it. The next security change should introduce a verified actor and one command-permission policy, not more browser-only checks.

Template kind/binding strings still need startup resolution into validated types. Machine-specific configuration still uses display-model keys. Audit history still needs bounded queries and rotation, and general HTTP input/concurrency limits need explicit policy. The browser still has presentation/workflow flags to simplify using the server phase. These are remaining changes, not guarantees supplied by this refactor.

4.5. K6 Raster Protocol

This reference describes packet layouts generated or consumed by the current Rust `k6` driver and the device behavior observed while testing it. It is not a description of a vendor application's internals. The driver supports raster work; vectors and firmware updates are not part of this interface.

4.5.1. Link and Coordinates

- Serial device: normally `/dev/ttyUSB0`; baud: 115200.
- Pixel pitch: 0.05 mm (20 pixels/mm).
- Current normal K6 working reach: 80 x 75 mm. The Y value is an operator-selected bound, not an exact measurement of the mechanical stop.
- The firmware-coordinate envelope is 1600 x 1600 pixels. Normal jobs are constrained by the smaller working profile.
- Raster rows are packed MSB-first; each row is padded to a whole byte. Black pixels are set bits.
- For a raster with top-left `(x, y)`, the header center is `center_x = x + width/2 + 67`, `center_y = y + height/2`.

4.5.2. Response Bytes

Bytes	Observed meaning
`09`	ACK for commands/chunks that the driver expects to acknowledge.
`08`	Error response; observed when a DATA payload is entirely zero.
`FF FF FF FE`	Device activity/heartbeat frame, including after a job header.
`FF FF 00 xx`	Progress frame; `xx` is the reported percentage byte.

The Rust driver accepts `FF FF 00 64` (100) as reported completion. If no byte arrives for 90 seconds, it reports assumed completion and may proceed to the next band. Silence is not device confirmation or proof of physical rest.

4.5.3. Commands Used by the Driver

Name	Packet	Driver behavior
STOP	`16 00 04 00`	Writes the stop frame. During an active burn this is also sent through the stop handle while cancellation is latched. It does not by itself prove motion stopped.
VERSION	`FF 00 04 00`	After STOP and input drain, reads three version bytes. Observed firmware: `04 01 06`.
CONNECT	`0A 00 04 00`	Sent twice; each command expects `09`.
HOME	`17 00 04 00`	Expects `09`.
BOUNDS	`20 00 0B` + width, height, center X and center Y as four big-endian `u16` values	Laser-off frame request; expects `09`.
FRAMING	`21 00 04 00`	Expects `09`; ends the frame mode in the observed workflow.
JOB_HEADER	38-byte `0x23` packet described below	The driver waits for an activity/status frame; this packet has no ACK contract in the driver.
DATA	`0x22` packet described below	Up to 1900 payload bytes. Each chunk expects `09`; timeout/error can retry that chunk up to three attempts with backoff.
INIT	`24 00 0B 00 00 00 00 00 00 00 00`	Sent twice after DATA with approximately 200 ms, then 500 ms spacing before status monitoring.

A normal burn band sends FRAMING, JOB_HEADER, a size-dependent delay, two CONNECT commands and the DATA chunks, followed by two INIT commands. The driver then monitors status frames. A multi-band raster repeats the band sequence for each nonblank band.

A normal machine stop also sends STOP, then FRAMING and HOME. An in-burn stop may only reach the immediate stop handle and cancellation path. The operator must confirm physical rest before changing placement or continuing.

4.5.4. JOB_HEADER (0x23)

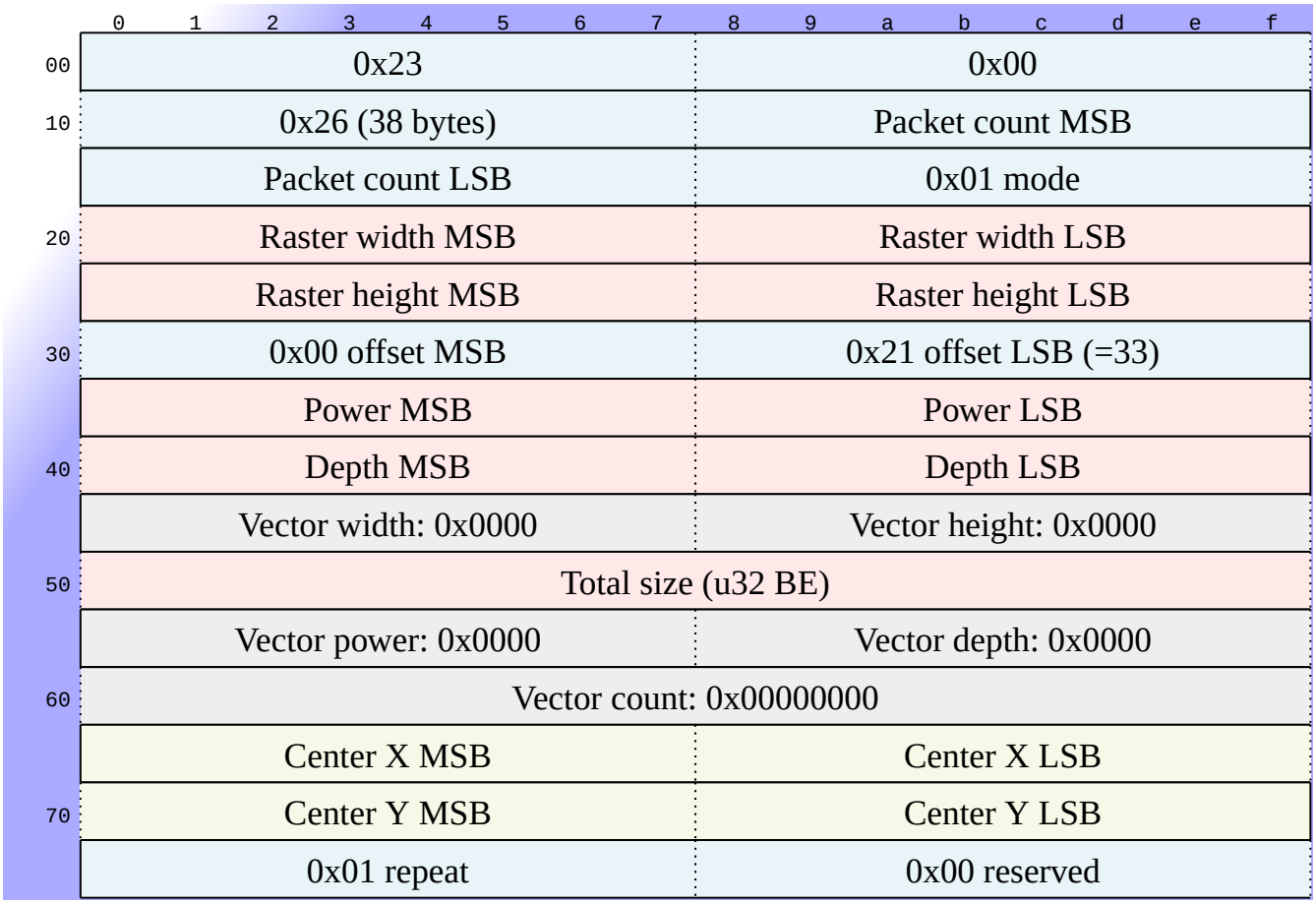


Figure 13. K6 raster JOB_HEADER layout; vector-reserved bytes are zero in the Rust driver

All multibyte fields are big-endian. The current raster-only builder sets:

Bytes	Field	Current value
0-2	Opcode, reserved, length	`23 00 26` (38 bytes)
3-4	Packet count	`(total_size / 4094) + 1`, integer division
5	Mode	`01`
6-7	Raster width	Pixels, `u16`
8-9	Raster height	Pixels, `u16`
10-11	Raster data offset	`00 21` (33)
12-13	Power	`u16`, machine setting
14-15	Depth	`u16`, machine setting
16-19	Vector width/height slots	Zero
20-23	Total size	Raster bytes + 33, `u32`
24-31	Vector power/depth/count slots	Zero
32-33	Center X	`u16`, pixels
34-35	Center Y	`u16`, pixels
36	Repeat	`01`

Bytes	Field	Current value
37	Reserved	`00`

The 33-byte data offset, raster-only values and repeat value describe the current Rust packet builder. Do not infer vector support from the reserved fields.

4.5.5. DATA (0x22)

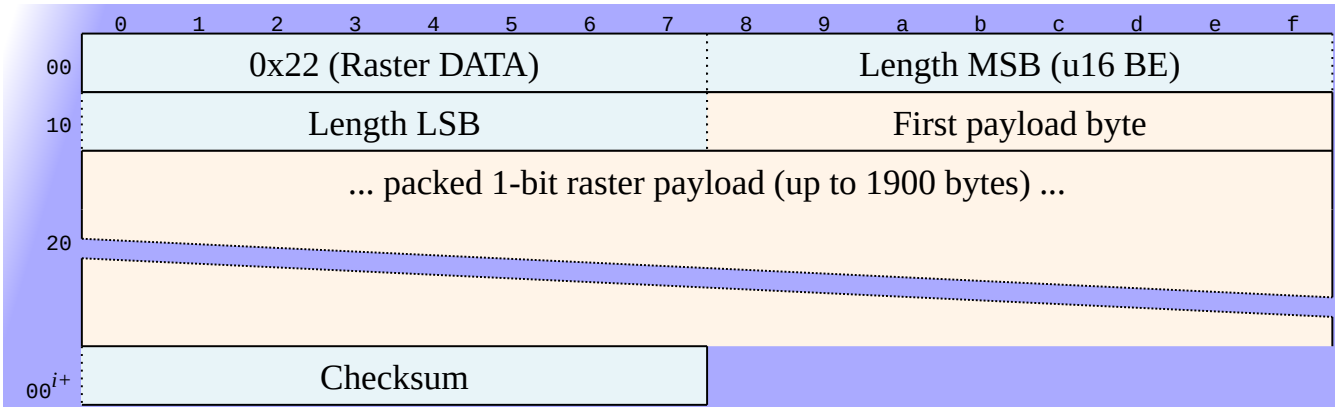


Figure 14. K6 raster DATA layout

- Bytes 0-2: opcode `22` and total packet length as a big-endian `u16`.
- Bytes 3 through the byte before the end: up to 1900 bytes of packed raster payload.
- Final byte: two's-complement checksum. The sum of the complete packet is zero modulo 256.

The driver drops blank edge rows and splits at long blank runs so that no transmitted 1900-byte chunk is all zero. This is a workaround for the observed `08` response to an all-zero DATA payload. It can result in several independent bands for one reviewed raster; progress percentages are band-local.

4.5.6. Framing, Stop and Status Diagrams

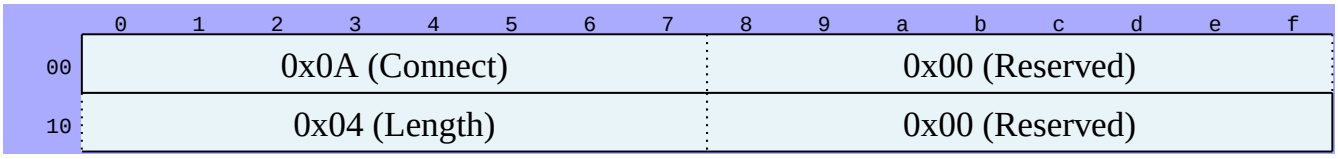


Figure 15. Connect

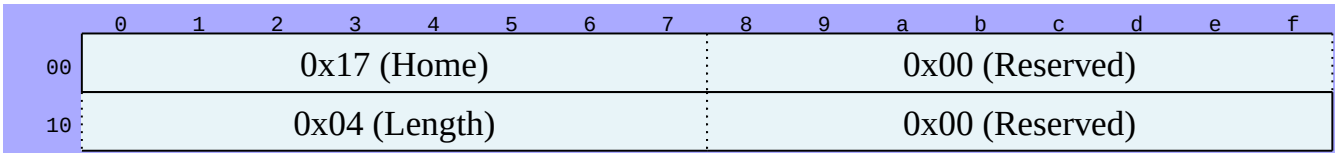


Figure 16. Home

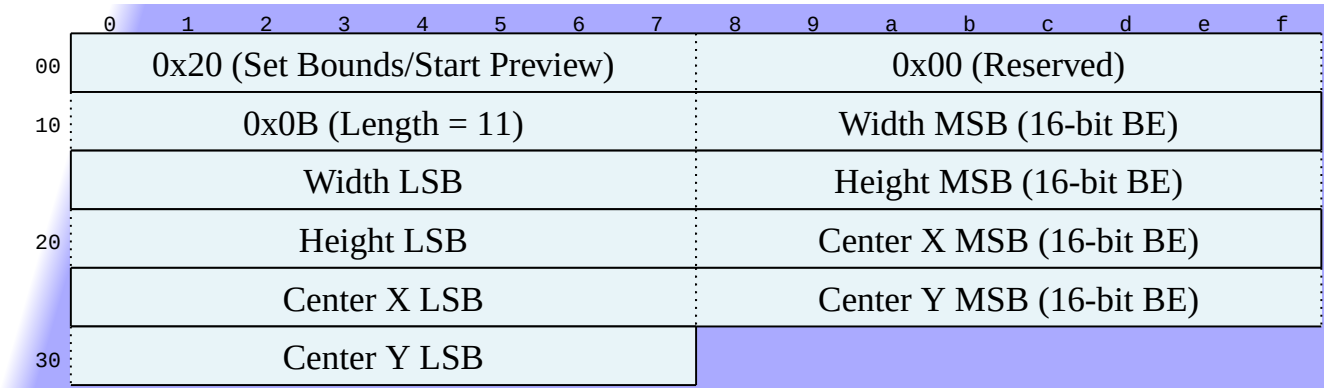


Figure 17. BOUNDS

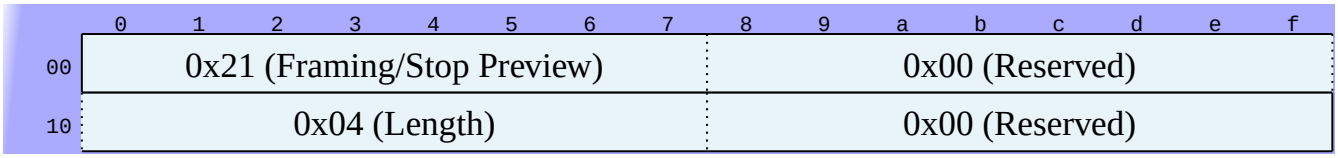


Figure 18. FRAMING

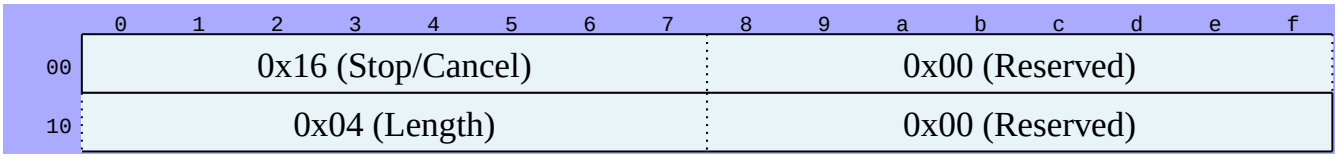


Figure 19. STOP

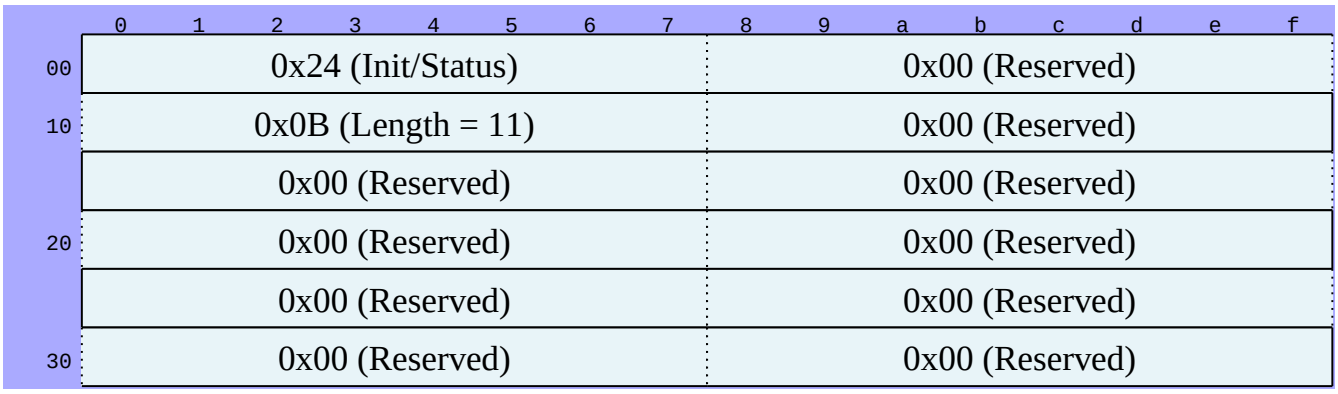


Figure 20. INIT

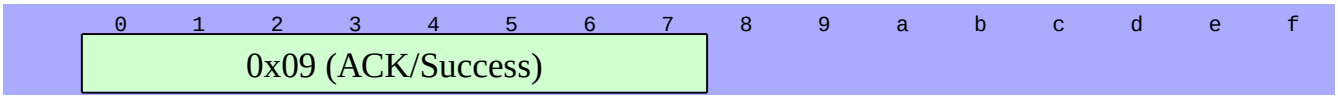


Figure 21. ACK and error

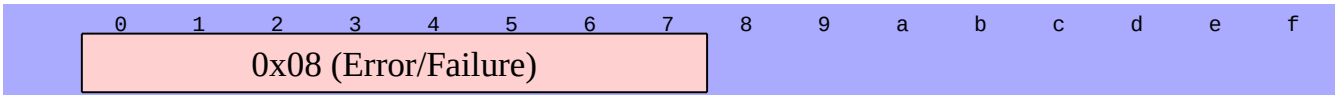


Figure 22. Error response byte

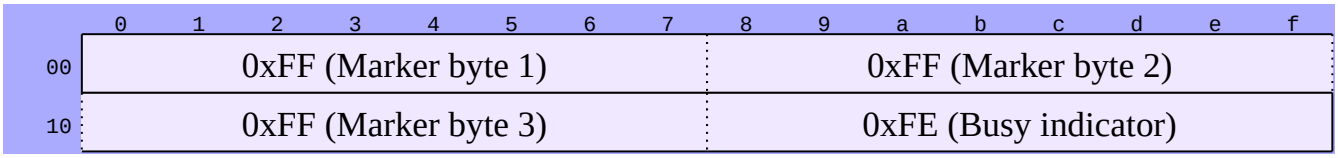


Figure 23. Activity and progress frames

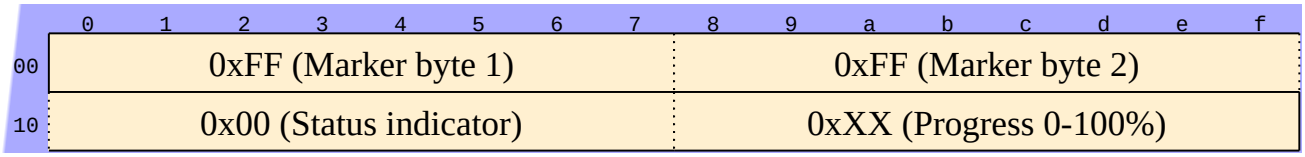


Figure 24. Band progress status frame

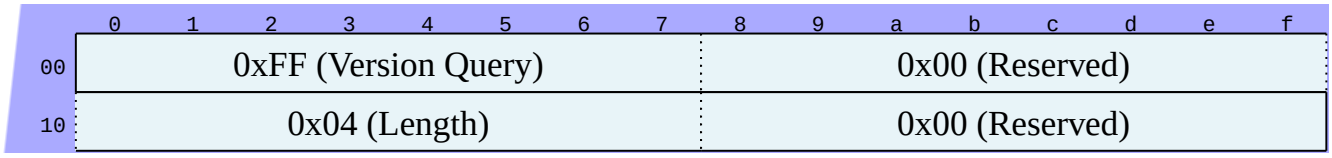


Figure 25. Version request and response

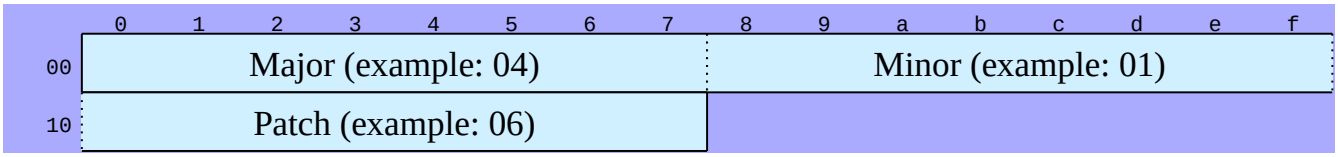


Figure 26. VERSION response

4.5.7. Evidence Limits

Packet fixtures and implementation are in `docker/docker-wainlux-rs/k6/src/protocol.rs`; sequencing and response handling are in `k6/src/device.rs`. The K6 packet SVGs are generated from the adjacent EDN sources with `scripts/generate_diagrams.sh`.

The Rust implementation and tests establish what this driver sends and parses. Hardware Observations distinguishes selected working limits and measured outcomes from assumptions. No command in this reference is a certified safety control.

4.6. Hardware Observations

Sean Donnellan triggered and verified the physical tests summarized here. This page keeps measurements that affect the current Rust service. Values are labeled by evidence type; historical runs are not current performance guarantees.

4.6.1. Current K6 Profile

Value	Evidence
Working reach: 80 x 75 mm; 20 px/mm	Current profile in `k6::info`. Sean selected 75 mm as the normal Y limit from the 9b comparison; it is not a measured hard stop.
Bamboo NFC card: 85.45 x 53.72 mm	Physically measured; configured as `bamboo-ntag215-light`.
Card material settings: power 1000, depth 10	Current configured profile. Safe depth and NFC survival remain material-specific acceptance checks.
Backing board dimensions	Not measured. `backing-first` dimensions are a planning placeholder, not a physical measurement.

4.6.1.1. Backing Board Power Sweep

The `backing-first` material note in `docker/docker-wainlux-rs/materials.toml` keeps the starting QR settings with the dated sweep observations. The starting profile remains power 500 / depth 10; the sweep used depth 10 and a 20 mm full-power frame plus descending-power lines.

On the X=0 block, Sean reported five visible lines, a faint sixth, a partly visible seventh under magnification, and no visible marks for the last three. The frame overlapped/covered the first line. On the adjacent X=20 block, the first line was visible and the frame appeared high relative to lines matching the first block. Alignment is therefore unresolved; do not infer a calibrated power threshold from these blocks.

The X=20 software trace reported 99% for the frame and first row, then 0% for rows 3-10. All ten jobs ended through the 90-second silence assumption, not a distinct completion response. The bounded trace reached its 1,024-byte cap during job 1; decoded progress/quiet events remain available for all jobs. The exact physical stop time per job was not instrumented.

The square-based material/depth test remains pending. Keep the existing QR baseline until that test is reviewed; no new material setting is recommended from the line sweep alone.

4.6.2. Executed Rust Ladder Tests

Sean Donnellan triggered the real-device runs and supplied the observations/verdicts below. Software status and elapsed time are reported separately from the physical observation.

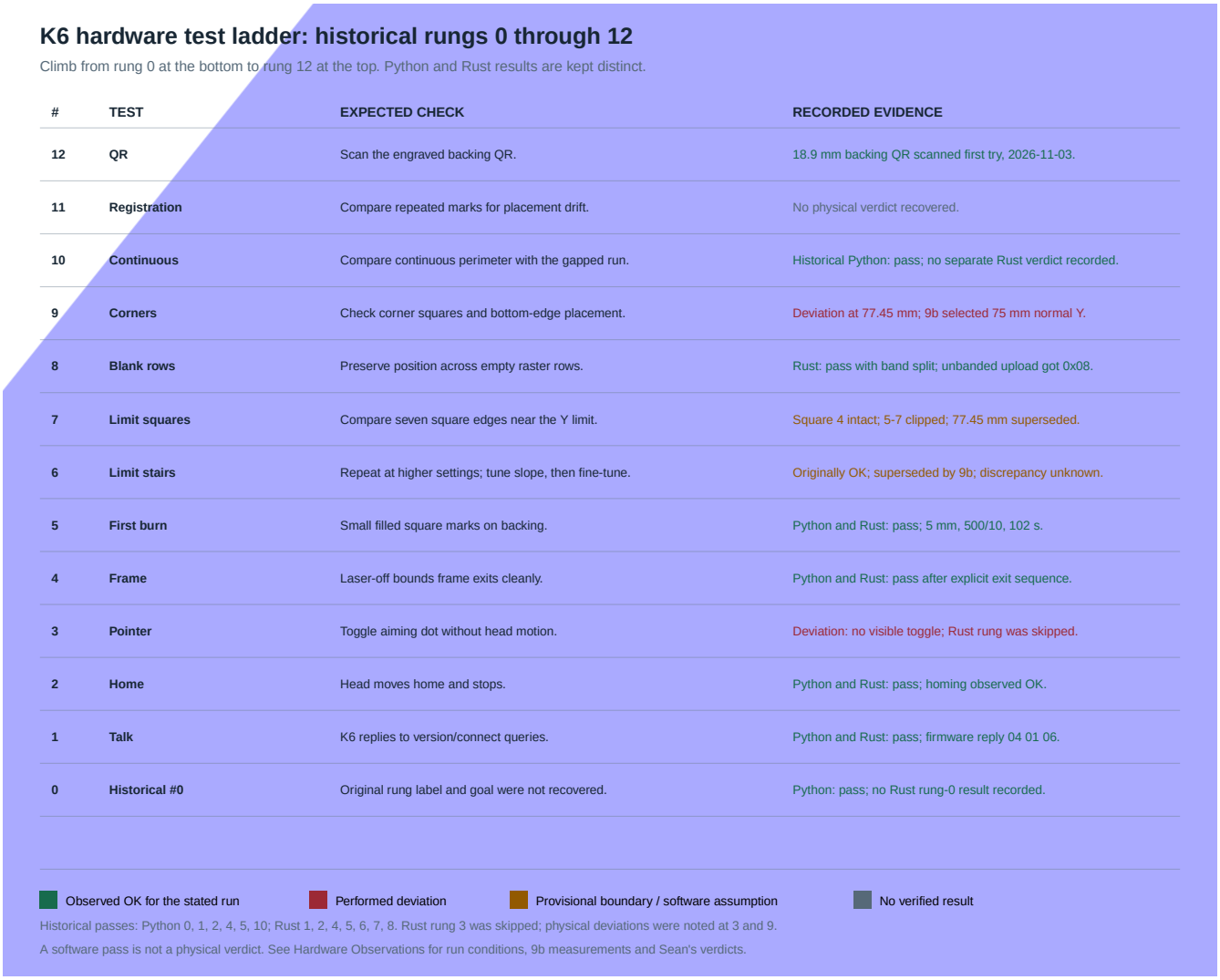


Figure 27. Historical test ladder, climbed from rung 0 at the bottom to rung 12 at the top

Historical rung / test	Conditions / software report	Sean's observation / verdict
Rung 1: Talk	Rust ladder; firmware reply `04 01 06`.	Talk observed OK.
Rung 2: Home	Rust ladder.	Homing observed OK.
Rung 3: Pointer	Python pointer test had no visible toggle; Rust ladder marked this K6 rung skipped.	Deviation for the attempted behavior; not a Rust physical pass.
Rung 4: Frame	Laser-off bounds frame; ended with STOP, FRAMING, HOME.	Frame observed OK after the explicit exit sequence; an earlier frame attempt deviated.
Rung 5: First square	Filled 5 mm square on backing; power 500, depth 10; 102 s; status stalled at 99%.	Square intact; OK.

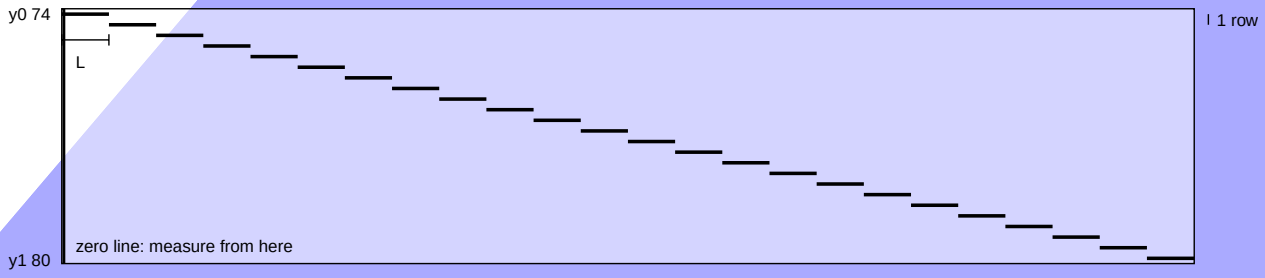
Historical rung / test	Conditions / software report	Sean's observation / verdict
Rung 6: Limit stairs	74-80 mm Y, 80 mm width. 300/5 was unreadable; 500/10 showed a faint kink near 45 mm. Repeated at higher settings; slope was adjusted and fine-tuned until the pattern yielded an interpreted result. Exact later settings and endpoint are not in the current record.	Originally marked OK after tuning. Its result did not match 9b; the cause is unknown.
Rung 7: Limit squares	Attempt `1793779971-limit_squares`; power 1000, depth 10; requested bottoms 77.15-77.75 mm.	Square 4 intact, squares 5-7 clipped. Initial 77.45 mm decision was superseded by the 9b comparison.
Rung 8: Blank rows	First unbanded upload received `08` for an all-zero DATA chunk. Band-split workaround: 205 s at 500/10.	Two squares correctly placed, with no marks between; OK with workaround.
Rung 9: Corners	80 x 77.45 mm, power 1000, depth 10; two bands; 5849 s (97.5 min); status stalled at 99%.	Top squares good. Bottom squares compressed against/below the bottom line; Deviation on bottom placement. Cause was not established.
Rung 10: Continuous perimeter	Historical Python ladder result was pass.	No separate Rust physical verdict recovered; the 9b continuous-strip comparison is recorded below.
Rung 11: Registration	Rust step exists in the current ladder.	No physical verdict recovered.
Rung 12: QR	Backing QR, 377 px (18.9 mm), power 500 / depth 10.	Scanned on the first attempt.

Rung 6 was repeated at higher values, first adjusting the slope and then fine-tuning it, and was marked OK at the time. The later 9b comparison did not agree with its apparent limit: both 77.45 mm strips deviated while the 75 mm gap and continuous strips were OK. No cause for the discrepancy is established. The 9b result is the accepted basis for the 75 mm normal Y profile; it does not measure the hard stop.

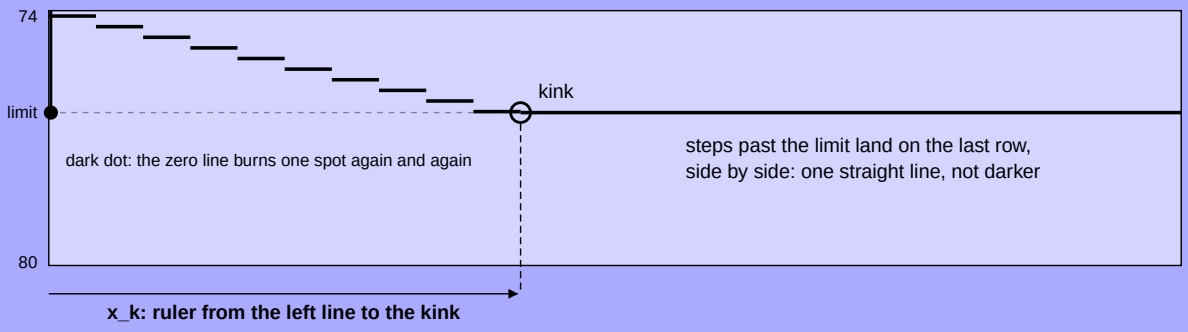
Limit stairs: one line per row, a diagonal until the head stops

Calibration pattern only. Current normal K6 working Y is 75 mm; this is not a stop measurement.

A. What we send: step k on row y0 + k, length L



B. Illustrative response (example only)



Size it

Guess a range y0 to y1 that holds the limit: 74 to 80 mm. Burn a thin line down the left edge.
Steps n = (y1 - y0) / 0.05 mm = 120. One step per row: every row has burn data.
L = width / n = 1600 px / 120 = 13 px (0.65 mm). 1 row tall: our thinnest line.

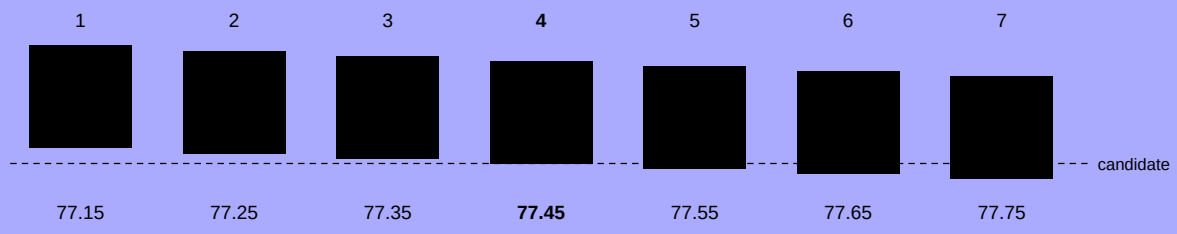
Read it

limit = y0 + (x_k / L) x 0.05 mm. Example: kink at 31.2 mm -> 48 steps -> 74 + 2.4 = 76.4 mm.
Run 2: range +/- 0.5 mm around run 1 -> 20 steps, L = 4 mm. Low power. X: turn 90 degrees.

Figure 28. Limit stairs: requested calibration pattern, not a measured result

7c: seven squares bracket the Y working bound

One raster. HOME first. Seven 2 mm squares, 1 mm gaps. No locator; inspect after the whole burn.



Numbers below squares are requested bottom Y edges, mm. Drawing shows the request, not the captured result.

Calibration pattern only; no normal working limit is implied.

Bottom-edge values are request parameters. Choose a candidate border for the supervised test.

This diagram is explanatory, not printable artwork or a captured result.

Inspect only after the combined burn has stopped; record the actual observation and verdict.

Figure 29. Seven-square calibration pattern: 77.45 mm was provisional and later superseded

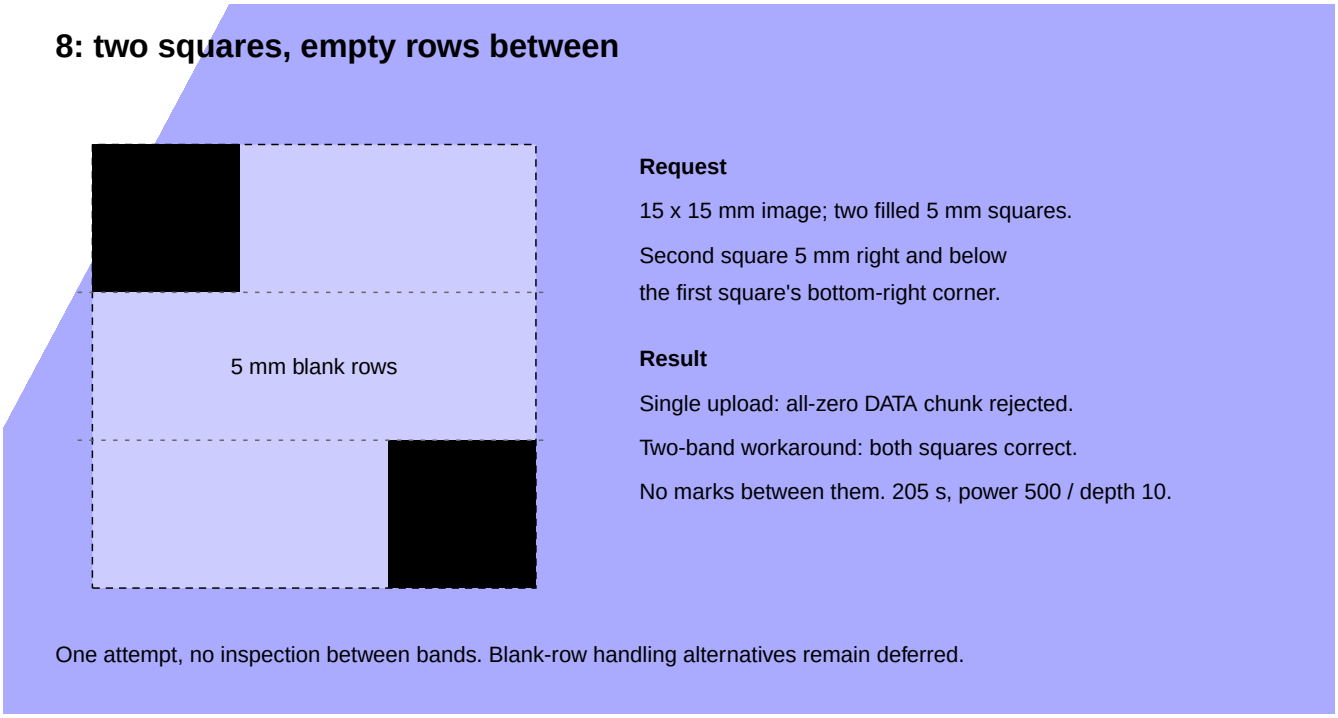


Figure 30. Blank-row test pattern

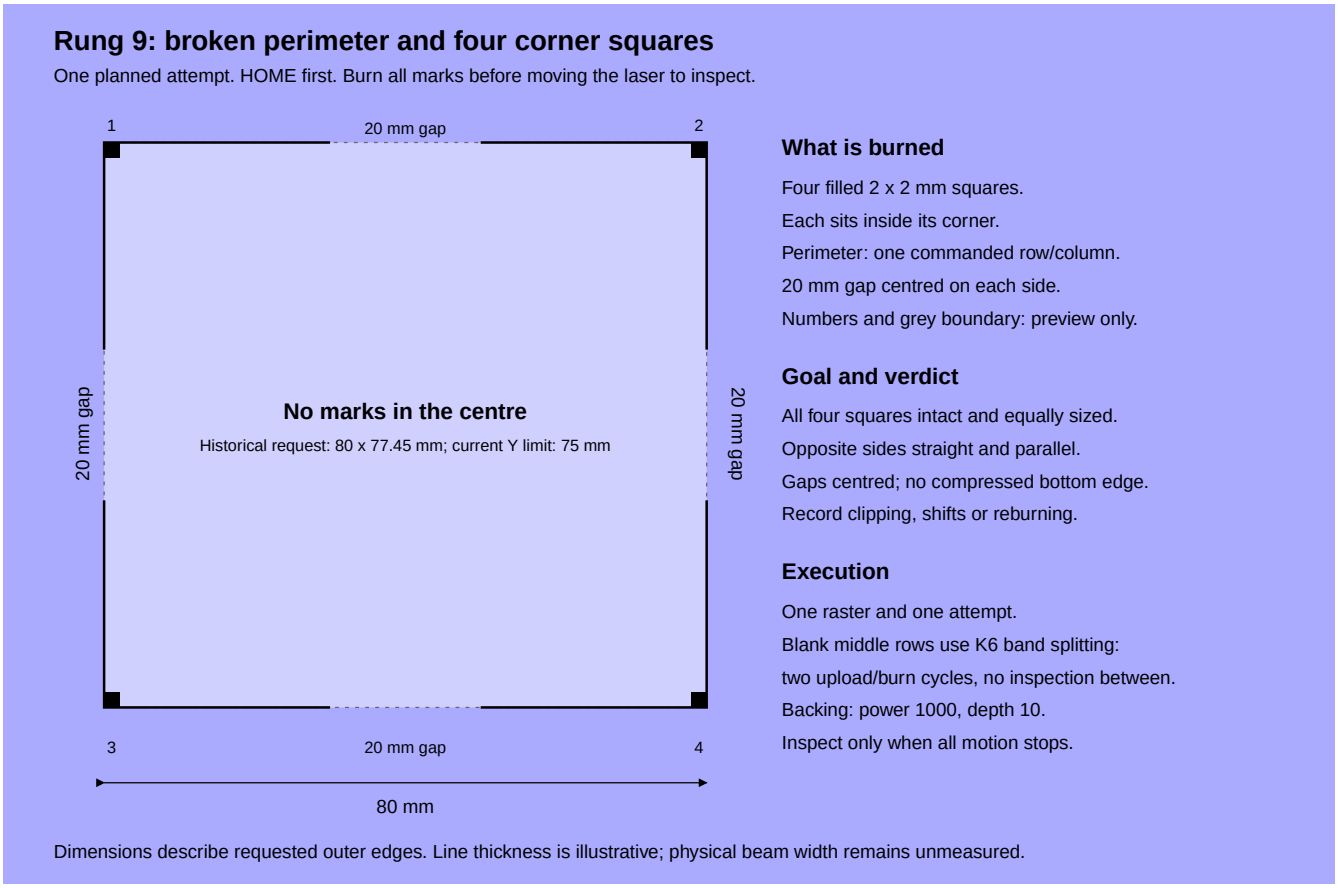


Figure 31. Corner test pattern at the former 77.45 mm height

4.6.3. 9b: Height and Gap Comparison

A/B/C were one joint session on the stock base. D was a later follow-up on the same setup. Each strip was a separate job with HOME before it; the backing was not repositioned. All used power 1000 and depth 10. Software status stalled near 99% in all four runs.

Strip	Pattern	X	Height	Duration	Sean's verdict / observation
A	20 mm gap	10 mm	77.45 mm	223 s	Deviation: bottom marker compressed.
B	Continuous line	20 mm	77.45 mm	242 s	Deviation: same bottom compression as A.
C	20 mm gap	30 mm	75 mm	219 s	OK, approximately 75 mm.
D	Continuous line, later follow-up	40 mm	75 mm	237 s	OK; matched C.

Sean's joint inspection found the gap and continuous pair matched at each height. The 77.45 mm pair deviated at the bottom; the 75 mm pair was OK. Decision: 75 mm normal Y working bound. This did not measure the mechanical hard stop or prove every multi-band placement.

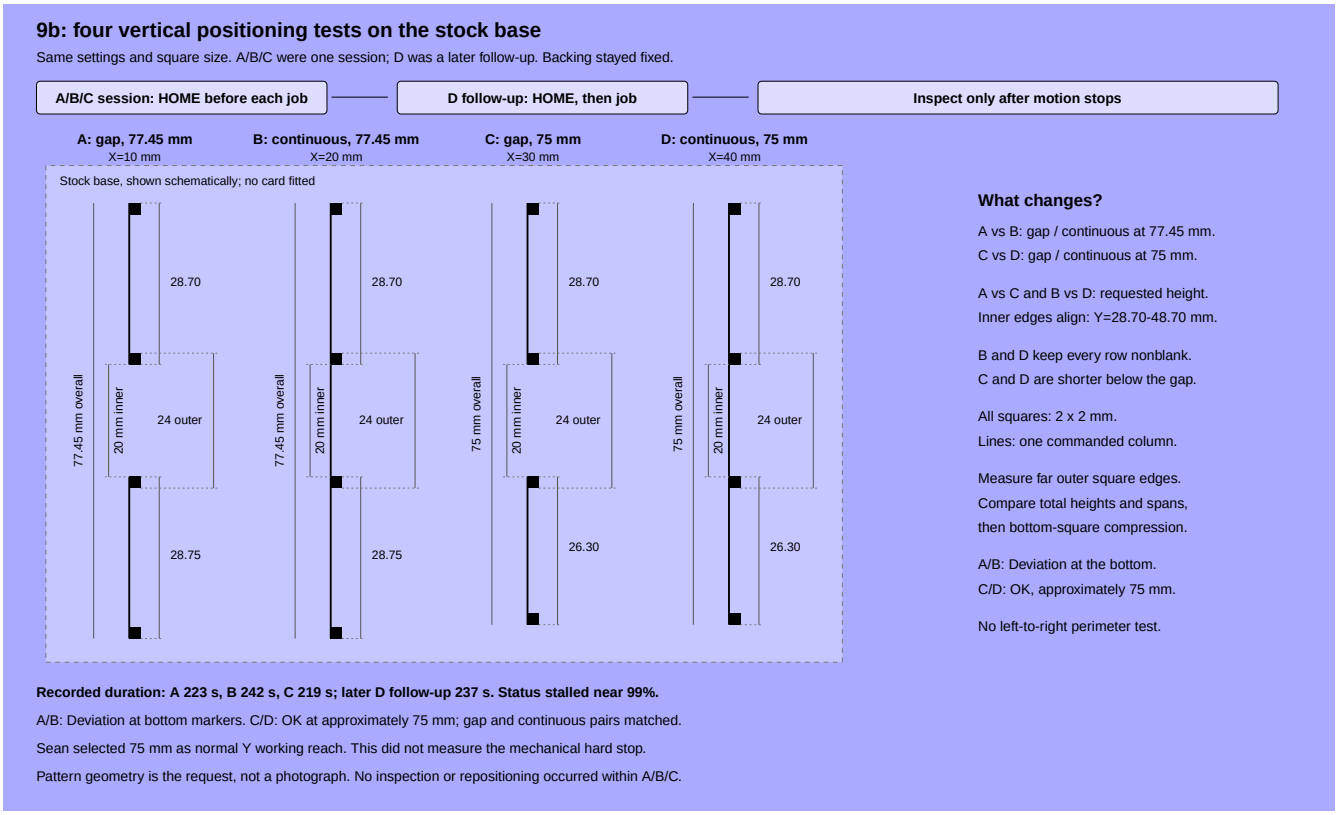


Figure 32. 9b four-strip comparison geometry; D was a later follow-up

4.6.4. Card Result

Sean reported that the first card run had a good guide, readable QR and working NFC. Printed text was too small. The subsequent larger bold text defaults have not been physically accepted in a physical run; verify them on a new supervised card before treating them as measured results.

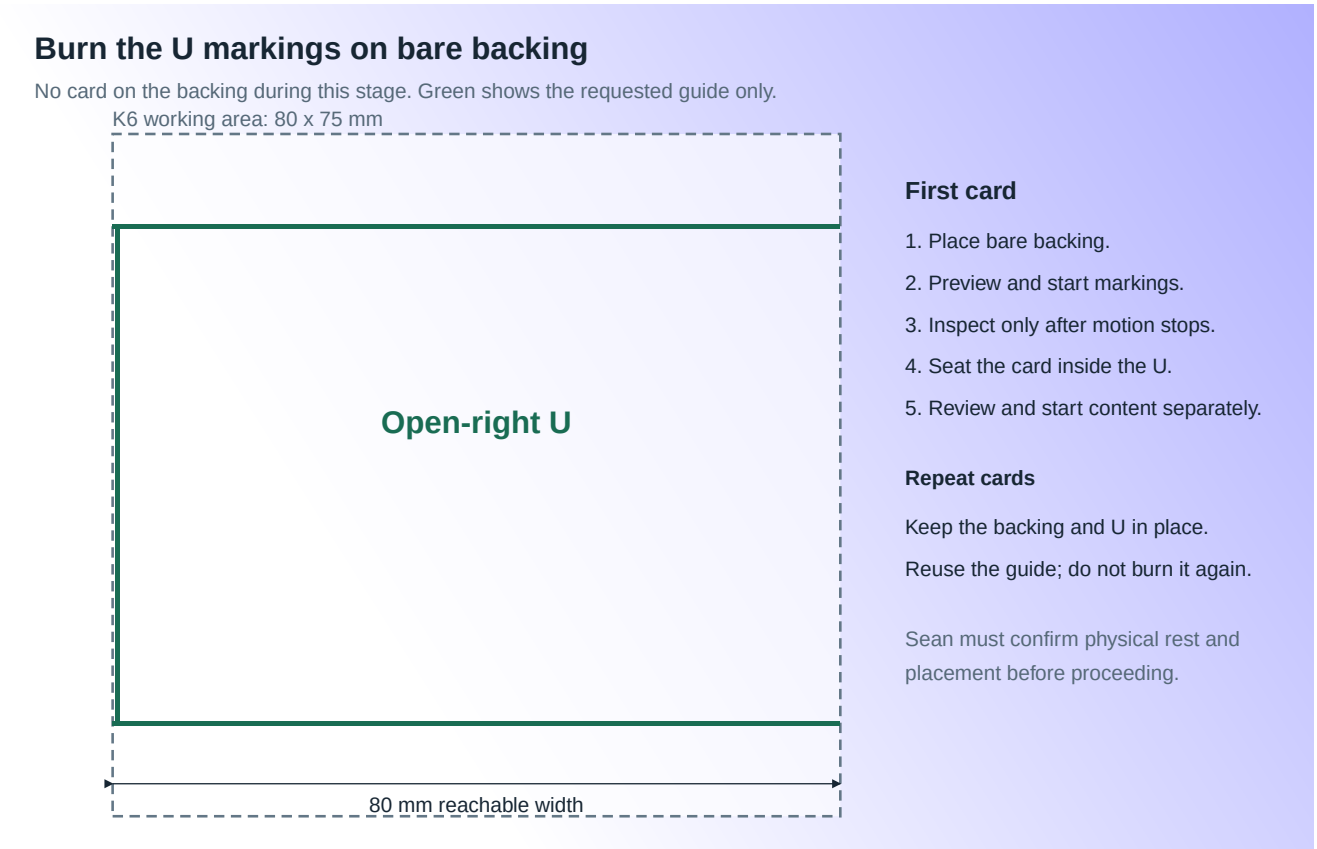


Figure 33. Generic open-right card guide used on bare backing

4.6.5. Historical Cork and PLA Close-ups

These recovered images show material behavior from earlier experiments. They are microscope/camera close-ups of cork and PLA, not the missing close-ups from the 9b corner/limit run.

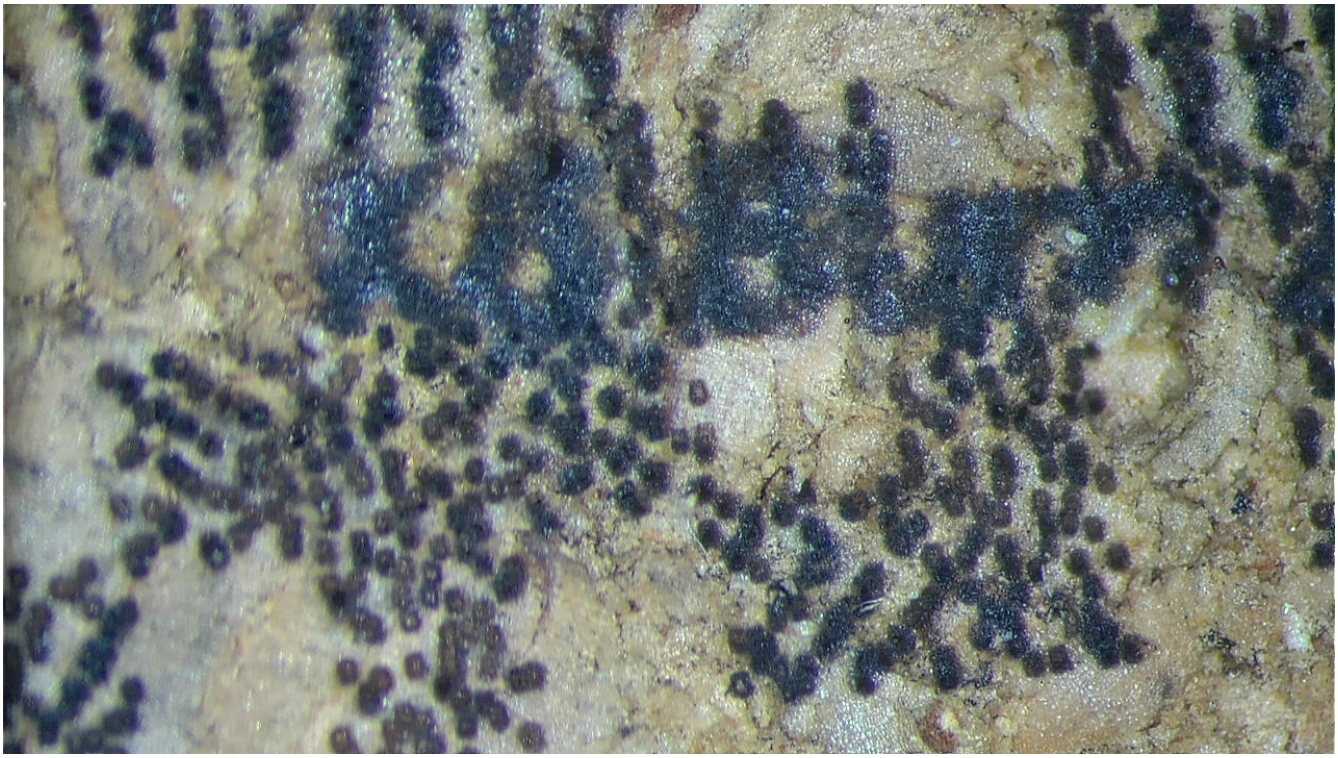


Figure 34. Cork test-pattern surface

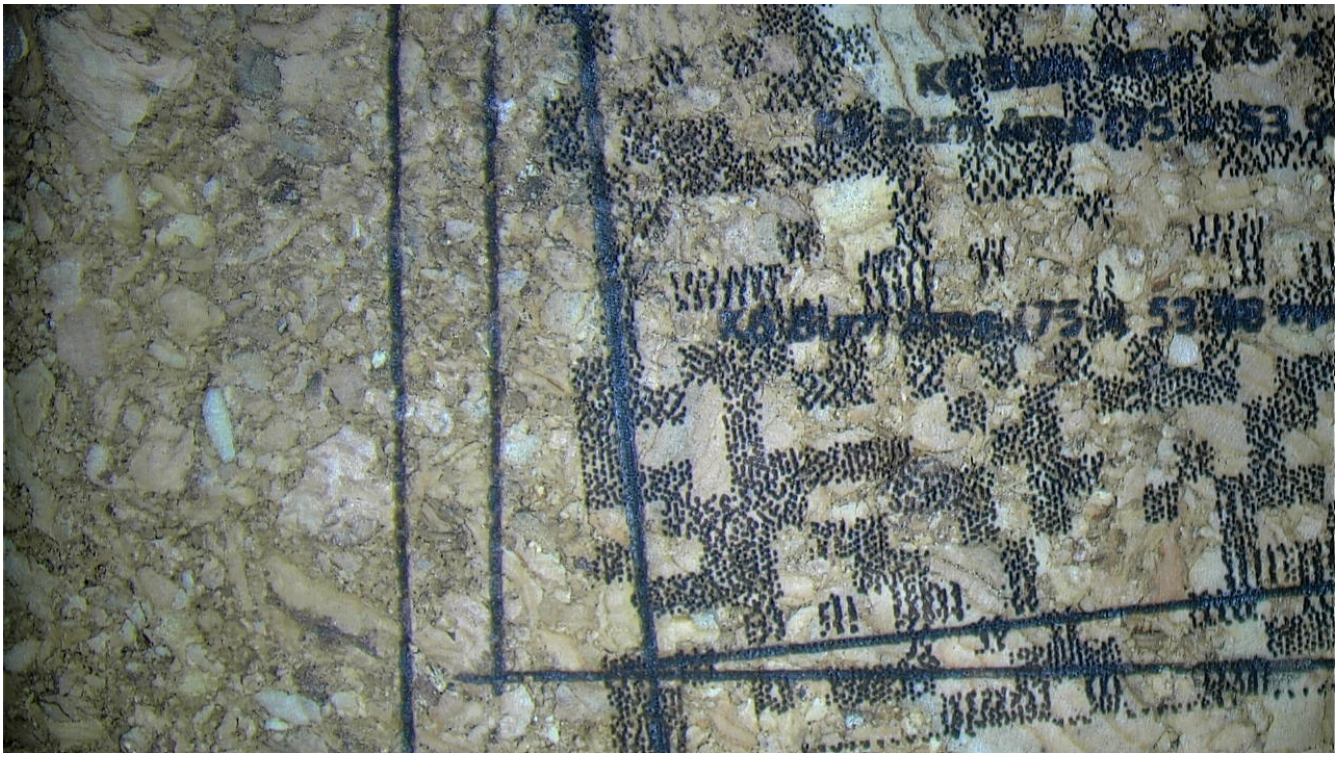


Figure 35. Cork pattern at wider view

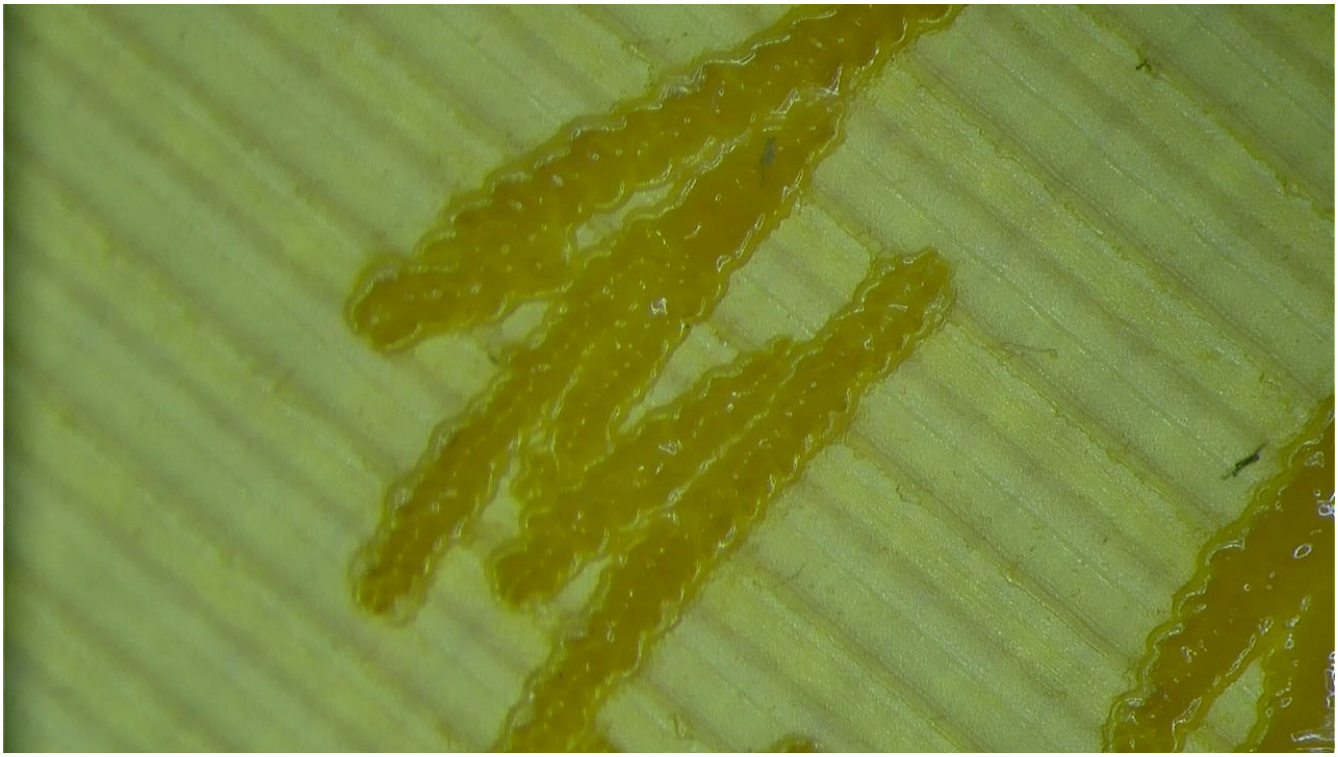


Figure 36. Pulsed line detail on PLA



Figure 37. Tiger pattern on PLA

4.6.6. Historical QR Run

A QR on the backing board was 377 px (18.9 mm), burned at power 500 / depth 10, and scanned on

the first attempt. This was a backing-board run, not the current card template's 36 mm QR.

For the historical transfer-time capture, see the provenance and measurements in `doing-debugging.adoc`.

4.7. NFC Cards and Recovery Backups

Recovery codes and short instructions can share one text payload in a QR and an NFC Text record on a wooden NTAG215 card. A phone can read either. Neither format encrypts the data.

The Card CLI Client owns submission commands and sample inputs. The Pi renders the cards; the CLI has no PDF or QR generator. The same Copyright and License covers this guidance and the code.

4.7.1. Reader/Writer Integration

The proposed HW-147 board has visible SCK/MISO/MOSI/SS SPI pins and a separate TXD/RXD UART header. Its controller, protocol, supply/logic voltage, tag compatibility, GPIO assignment and sharing with the proposed audio/display HAT remain unverified. Do not infer a supported UART or SPI driver from the pin labels. Hardware photos are in the Next Steps hardware section.

There is no NFC reader/writer adapter in the service or CLI yet. Implement it separately from laser control: detect the supported tag and capacity, read existing records, require an explicit operator-confirmed write, then read back and compare the complete payload. A read or write must never start a laser job. Reject incompatible or insufficient-capacity tags without silently truncating data or overwriting unrelated records. Test with synthetic data first.

The capacity reference below covers only one English NDEF Text record on a Type 2 tag. Neither the CLI nor service currently validates NFC storage. Future Wi-Fi or vCard records may use other NDEF record types and overhead; these limits must not be reused unchanged for them. The service's Wi-Fi QR and vCard QR are not themselves an NFC writer implementation.

4.7.2. Synthetic Demo

Use `cli/inputs/synthetic-codes.txt`, not a private file. The fictional values cannot unlock GitHub. Submit the file through the CLI, review the Pi-generated QR, and scan it with QRbot in text view, not a detected link. Test phone reading and NFC read-back before real codes.



Figure 38. Historical synthetic recovery-code QR, not a current Pi rendering

This retained illustration includes fictional instructions and codes. It does not update with the sample file and is not the raster to engrave. Use the live Pi preview for the exact submitted payload and geometry.



Never replace this synthetic example with a real-code QR.

4.7.3. Capacity

Tag	User memory (bytes)	Maximum text (UTF-8 bytes)
NTAG213	144	134
NTAG215	504	489
NTAG216	888	873

These limits count UTF-8 bytes, one English NDEF Text record, and Type 2 TLV overhead. Maximum text includes codes, instructions, and separators under this layout. Use one Text record with language `en`. Add no other records. NFC Tools may use a different layout. Its write check and a read-back are final. Shorten instructions if the payload does not fit. Do not drop codes.

4.7.4. Make The Card

1. Scan the QR on screen with QRbot. Open its text view, not a detected link.
2. Compare all codes and instructions with the source. (Do not spend a code to test it unless you want to test the codes too).
3. Transfer the source payload to the phone through a trusted channel if needed.

4. Keep an independently prepared, checked paper backup in a secure place separate from the NFC card. The CLI does not create it.
5. In NFC Tools, choose Write, Add a record, Text. Paste the whole payload. Use language `en`.
6. Write one record to the NTAG215. Read it back and compare the full text.
7. Submit the card content, inspect the Pi's exact raster and test engraving on scrap. Use strong contrast and keep the QR border.
8. Engrave a separate wooden cover before fitting an NFC inlay; do not laser the antenna or chip. Each side requires its own operator-reviewed start, inspection and verdict.
9. Scan the finished QR and read the assembled NFC card again.
10. Store the card in an opaque, RF-shielded, credit-card-size holder. Check that NFC cannot be read while enclosed.

The service owns QR encoding, fonts, sizing and exact per-side pixels. Submit a codes-only UTF-8 file for a recovery card; do not encode a login password with recovery codes. Review the Pi preview and physical size before starting. Wood grain, surface finish, contrast and module size affect scanning.

4.7.5. Recover

Remove the card from its holder. Read its NFC Text record with NFC Tools, or scan its QR with QRbot. Display or copy the codes and instructions. Follow the service's recovery steps. Stock iPhone Camera and background NFC reading may not display plain text. The reader apps are required for this workflow. Reading works offline.

4.7.6. Protect

Anyone who reads the QR or NFC can read the codes. An opaque holder hides the QR; RF shielding limits NFC access but does not encrypt the tag. Test the holder. Neither the wood nor holder is waterproof or indestructible.

4.7.6.1. Theft And Password Separation

Treat the card and paper backup as secrets. Store them under lock, not in an everyday wallet. A thief can photograph the QR or copy the NFC record without leaving a mark. The holder does not protect the contents after it is opened.

Keep the login password in a secure password manager, separate from these backups. Do not include it in the recovery input, instructions, QR, NFC, or paper backup. For GitHub's password-and-recovery-code flow, a leaked code set alone does not provide the password. Storing both together can give a thief both login factors. Separation reduces that risk; it does not make stolen codes harmless. For some services a recovery code alone is enough. Check the service's rules. The CLI cannot detect a password mistakenly included in an input file.

4.7.6.2. Workflow Leaks

Every plaintext step can leave a copy: downloaded codes, generated files, phone Photos and cloud sync, QRbot history, NFC Tools records, the clipboard, print queues, and laser job files. The apps and

devices must be trusted. Offline use alone does not prove that they will never retain or upload data. Use a trusted local printer and laser. Do not send real jobs to a shop. Check app retention and sync settings. Remove temporary files, scan history, clipboard contents, and job copies when done. Deletion is not secure erasure.

4.7.6.3. Loss And The Paper Backup

Keep an independently prepared paper backup in a separately secured location, not in the card holder. Readable codes and instructions provide another way back in if the card is lost, damaged, or unreadable, without needing an NFC or QR reader. Verify the paper against the source. An extra copy improves availability but creates another secret to secure.

If the card, paper backup, or a digital copy is lost, stolen, or possibly copied, use the remaining recovery method to secure the account. Generate a new code set promptly and replace every backup, including the paper copy and engraved QR. Do not wait for proof of misuse. The remaining backup restores access; it does not revoke the lost one. Keep a spare authentication method too.

Track spent codes separately. When you replace the code set, rewrite NFC and replace the engraved QR or the card. A rewritten tag does not update an old engraving. Marking a spent code on paper does not update NFC or the QR.

4.7.6.4. Optional Access Protection And Encryption

The card has no read password or PIN. The tool relies on physical custody to keep recovery simple.

NTAG215 supports optional 32-bit read protection, but the CLI does not configure it. Write-only protection does not hide codes. A tag password is not encryption and cannot protect the printed or engraved QR. It also adds a secret that must be retained.

An NFC-only card could omit the QR and printed codes, then use tag read protection. Keep the separate paper backup secure. Without read protection, NFC data remains readable.

Encryption would require encrypting both QR and NFC payloads and using a local phone decryptor. It adds a key, a recovery step and tag-space overhead; losing the key can make recovery impossible. The tool does not implement encryption. This design instead separates the login password and secures the card and paper copy. Higher-risk accounts may need stronger protection.

4.7.7. Security Review

This personal backup is not a certified authenticator.

NIST SP 800-63B-4, section 4.2.1.1, allows saved recovery codes in QR form. It says to keep them offline and store them securely. The chosen format fits that advice. Safe custody still depends on the owner. NIST's server-side hashing rules are for the service's verifier. Hashing the backup would make the original codes unrecoverable; it is not a backup protection method.

GitHub's recovery guidance supports printed copies, recommends a secure password manager, and warns against sharing codes. It recommends two or more authentication methods. Keep a spare security key or another supported method separate from the primary phone.

DoD CAC guidance protects the PIN and requires revocation of lost or compromised cards. A CAC gates access with a PIN and locks after failed attempts. The QR and plain NFC record have no such gate. NTAG215 is not a CAC replacement. I do not claim DoD, FIPS, or NIST assurance-level compliance.

4.7.7.1. Local Files



Digital copies are plaintext. Unix permissions do not protect against the file owner, administrators, malware, backups or theft of an unlocked device. Use encrypted storage and trusted, updated devices. Keep unnecessary copies out of Photos, cloud sync, shared folders, email and chat.

Local permissions: use directory mode `0700` and file mode `0600` for `cli/private/`. The CLI does not create outputs or enforce input permissions. Keep these restrictions when replacing files. Permissions do not encrypt data.

Run from the repository root on Unix:

Restrict permissions on private inputs

```
chmod 700 cli/private
chmod 600 cli/private/github-recovery-codes.txt
```



Suitable only as a physically secured personal backup with controlled processing and prompt replacement after loss. It is not theft-resistant storage. Source permissions, device encryption, app retention, holder shielding, and physical custody are not all enforced or verified by this tool. It is your choice to use or not use it.

4.7.8. Workflow

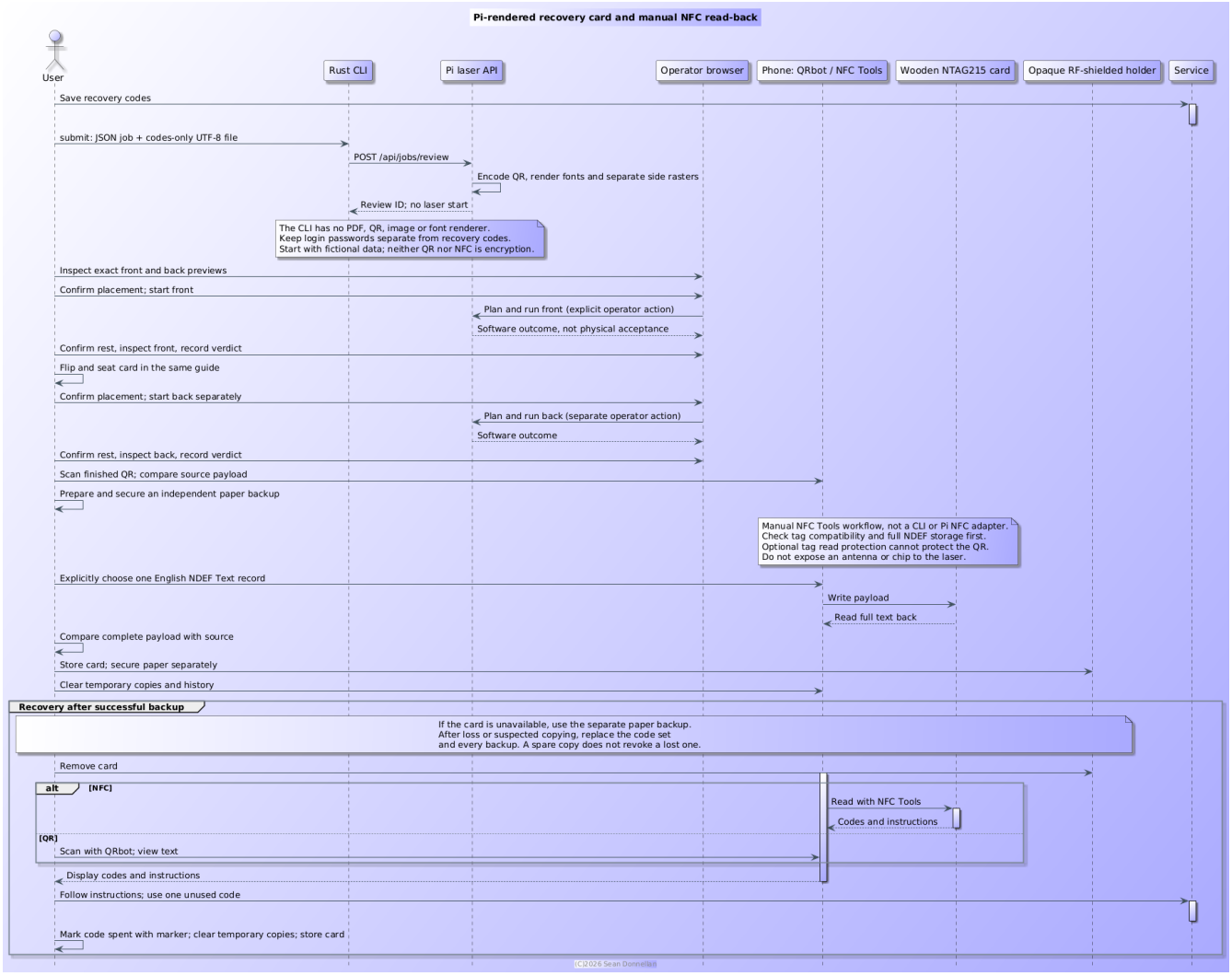


Figure 39. Backup creation, card writing, and recovery workflow

Edit `documentation/images/recovery-workflow.plantuml` and render locally. Never send real codes to a public diagram server.

4.8. Burn Timing

4.8.1. Current Report

The Rust service exposes completed band timings in `GET /api/state` as `band_timings` and includes them in the run outcome. Each record describes one raster band. For a multi-band run, sum the records for a run total.

Field	Meaning
`transfer_ms`	Time spent sending raster data, including any retries.
`setup_ms`	Raster preparation and protocol setup outside transfer and completion wait.
`wait_ms`	Time waiting for K6 completion status after setup.
`elapsed_ms`	Total elapsed time for the band.

Field	Meaning
`phase`	`reported_complete` means the device reported completion; `assumed_complete` means the service observed 90 seconds of silence.
`percent`	Device-reported progress for this band; it is not the percentage for the full side or card.

The timing values are milliseconds. Small differences between summed component times and elapsed time can result from measurement boundaries and millisecond rounding. Assumed completion is not device confirmation.

Card review responses also report raster-generation and preview-encoding milliseconds. The browser and CLI report their own review round trips, which include transport and client-side work and are not additive server phases. The outcome's `elapsed_ms` covers the prepared operation sequence. The UI breaks its measured band totals into transfer, setup and laser feedback wait, with the remaining operation time shown separately. Audio upload/analysis response time is a different workflow and is labelled separately.

For interpretation, the UI can open a self-contained real-run sample at `GET /examples/audio-analysis`. It demonstrates decoded sync ID and candidate activity beside run-log markers. This one sample is not a process capability study: use repeated comparable runs, retain the defined measurement boundaries and analyze distributions before drawing Six Sigma conclusions. Acoustic activity remains a candidate, not confirmation of laser motion.

4.8.2. Historical Measurements

The following measurements are from the retired Python service and are not Rust performance claims.

- A 1600x1600 raster used a 320,000-byte payload in 169 chunks. Transfer took 35.8 seconds, about 8.7 KB/s, with approximately 208-210 ms per chunk.
- In that run, the first observed completion status arrived about 4.9 seconds after data transfer. The first observed percentage was 37%; earlier progress was not captured.
- In a separate completion-detection run, the estimated duration was 336 seconds while the observed burn lasted 889 seconds. The old monitor stopped early; this observation informed later completion handling.

These are dated observations from specific runs. Use `band_timings` to see how a current Rust run divides time among setup, transfer and completion wait.

4.8.3. Audio Sync and Candidate Motion

Use the containerized analyzer from the repository root:

Analyze an audio recording and correlate it with the API log

```
./scripts/analyze_laser_audio.sh /path/to/recording.m4a \
    --run-log /path/to/sanitized-run-log.json \
```

```
    --attempt-id ATTEMPT-ID --expected-id DDDDDD
```

The current upload report includes the recording for playback, its decoded ID when available, a full-length auto-scaled min/max amplitude waveform with time/amplitude zoom, seek and Jump to end, timestamp-aligned receive bytes/decoded frames when captured, run-log markers, and measured processing timings. The packet table lets operators inspect which received values preceded progress/completion events; untraced runs explicitly lack those packet events. It does not classify audible activity or infer laser state. The Analysis tab compares the latest ten run goals/settings and durations, with links to individual uploaded reports and optional side-by-side waveform views. The in-app `/examples/audio-analysis` page is an older exploratory report.

The M4A creation-time tag differed from the API marker wall clock by about 2 h 40 min even though the audio DTMF ID matched. Use the ID for association; do not treat file metadata as a physical timing reference. The analyzer uses FFmpeg 5.1.9 in its pinned container and Python standard-library Goertzel analysis for the sync ID; no audio packages are installed on the Deck.

The current detector still waits 90 seconds after the final received byte. The run log, measured timings and recording playback do not prove exact physical stop. See ADR-027 and FR-017 for the evidence and decision gate.

4.9. Next Steps

These are investigation and experiment ideas, not commitments or claims of device support. Check the current implementation and tests first; use MCP Ghidra analysis when observable behavior or packet captures leave a device-side question unresolved.

4.9.1. Optional Hardware

4.9.1.1. NFC read/write module

The photos show both sides of an HW-147 reader board. It has SCK/MISO/MOSI/SS SPI pins and a separate TXD/RXD UART header. Confirm the controller, UART protocol, voltage, tag support, GPIO allocation and HAT pin sharing before wiring it to the original Pi Zero W.

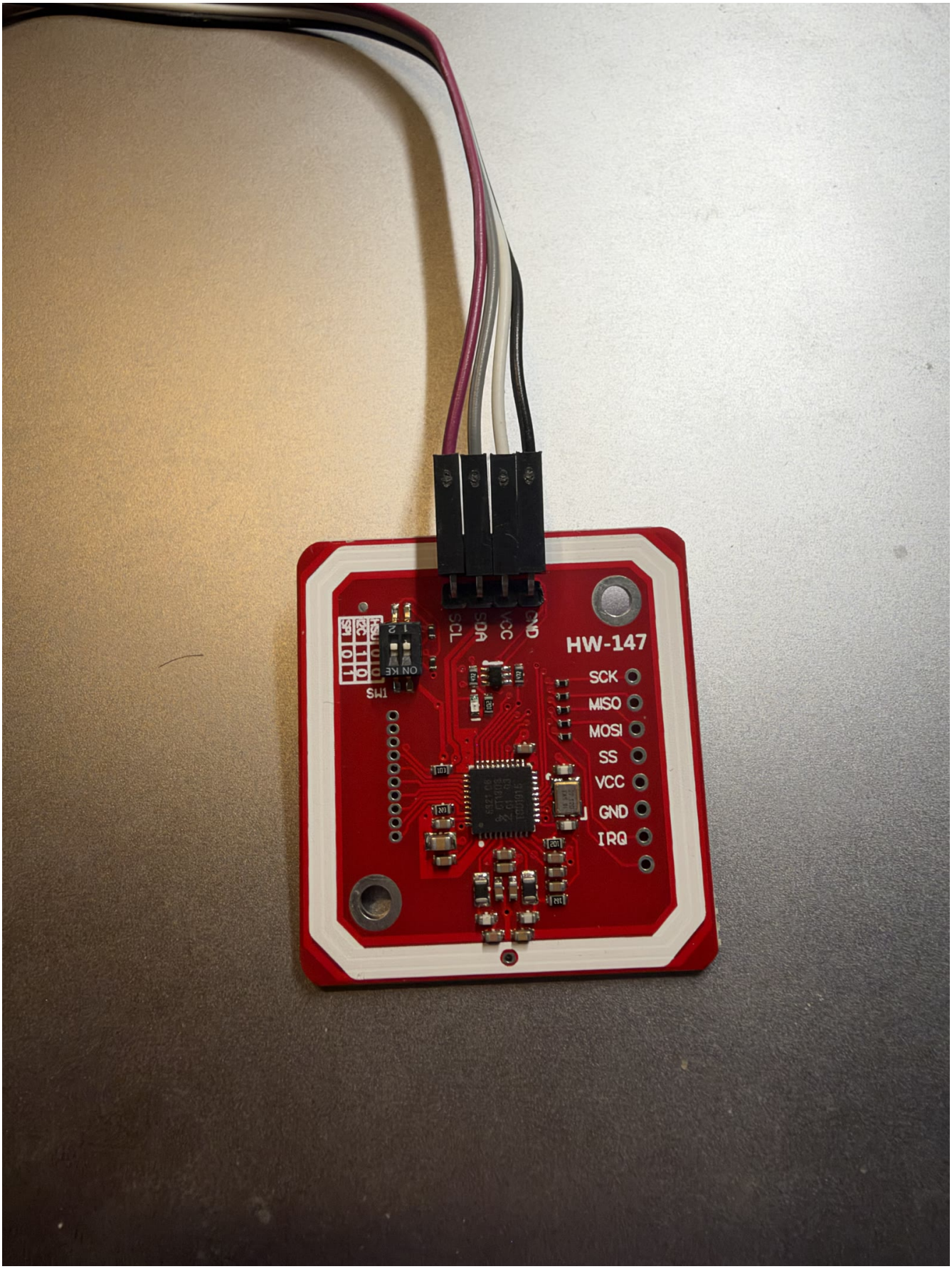


Figure 40. Proposed HW-147 NFC reader, component side

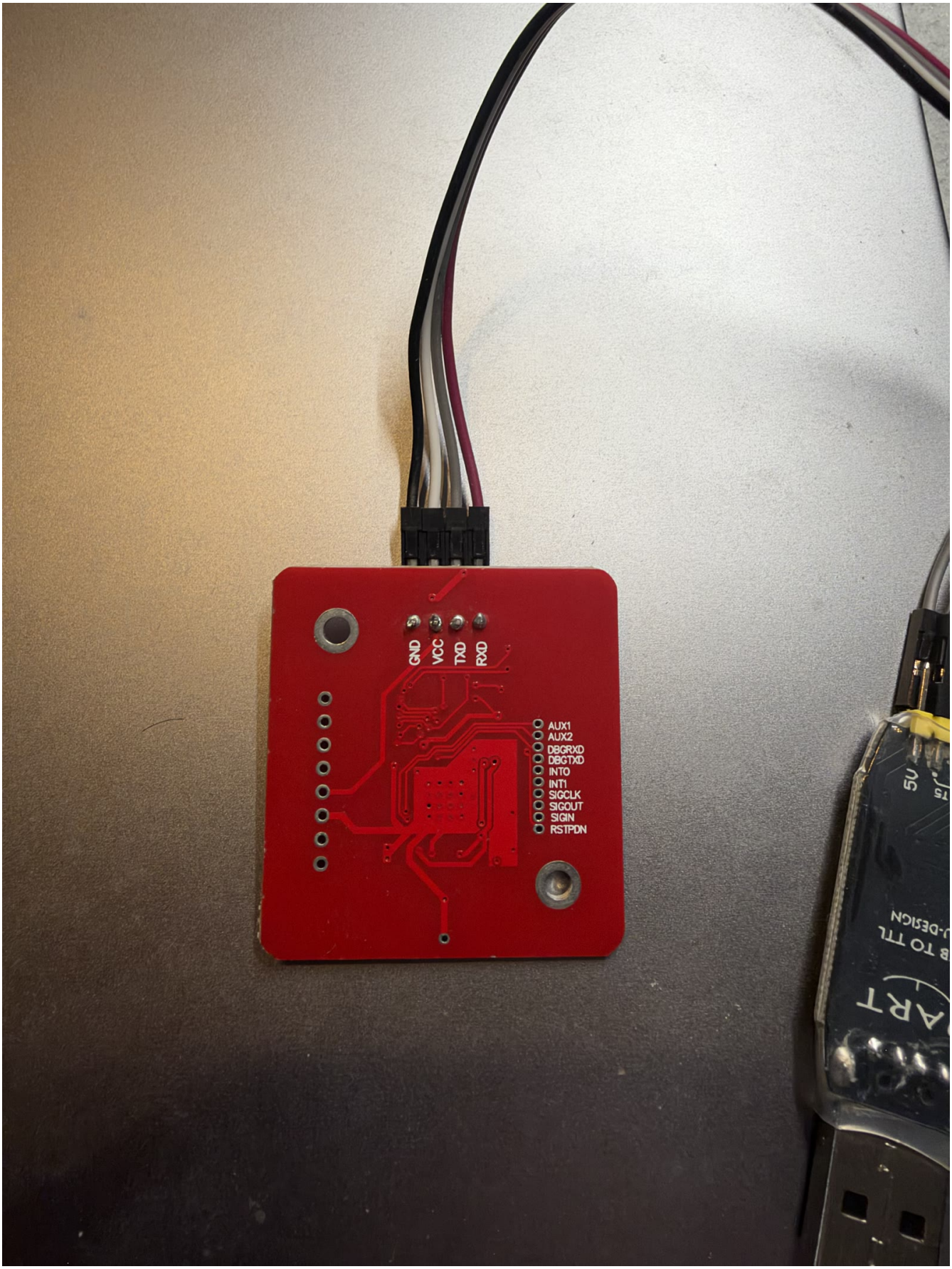


Figure 41. Proposed HW-147 NFC reader, reverse side

If compatible, expose NFC read, write and read-back as a separate adapter and operator-confirmed API action. Verify the payload by reading the tag after writing it. Keep tag operations separate from laser plan/run and use them to test cards after engraving.

4.9.1.2. CYD operator display

The photo shows a 2.8-inch, 240 x 320 CYD. It needs separate firmware and should use Wi-Fi to call the API, not GPIO to control the laser. Preserve the existing review and explicit plan/run flow.



Figure 42. Proposed 2.8-inch CYD board

4.9.1.3. Audio/display HAT

The LAFVIN audio/display HAT is ordered, not installed or tested. Before use, verify original Zero W ARMv6/32-bit support, GPIO/SPI allocation, power draw, capture quality and coexistence with an NFC reader.

4.9.2. Supervised power sweep

- **Purpose:** Compare the adjacent 20 mm block of commanded power levels with the previous block, capturing a bounded RX trace for each job.
- **Layout:** Keep the laser fixed in place. Block 1 starts at X=0; the next block starts immediately to its right at X=20 mm. On sacrificial stock, burn a 20 x 20 mm frame at (20, 0), then nine 19 mm horizontal lines at X=20.5, Y=2 through 18 mm in 2 mm steps. The frame's top edge is the full-power 1000 reference and replaces a duplicate line. Remaining powers descend from 889 to 0. Zero is not assumed laser-off; every mark may be active.
- **Procedure:** Review the entire layout and start one attempt. In Real mode, the UI plays a three-tone preamble and six DTMF digits (last six digits of the attempt ID) before sending the run request; record the room audio on an iPhone with computer speakers audible and use that ID to label the recording. The service homes and submits the frame, then each remaining row as a separate sequential job. A bounded RX trace records raw received bytes, decoded frames, progress and quiet intervals, keyed by job index; it captures no outgoing raster data. A software/device error stops the remaining submissions. The existing 100% and 90-second silence rules are unchanged; silence remains an assumption, not proof of rest. After the run physically stops, judge both blocks together and record one verdict. Keep the unit and stock in place.
- **Observation:** On the first block, Sean reports the first five lines visible, line 6 very faint, line 7 only partly visible under magnification, and later lines not visible on the backing. The reference frame appeared offset above the first line. This is a material/power visibility result, not an end-detection result. The adjacent traced block's physical result remains to be judged.
- **Fixture:** If a locating jig is used, it must register the laser without shifting its established position. Measure and verify the actual unit before relying on a downloaded or printed fixture; published K6 trays note unit/printer dimensional variation.

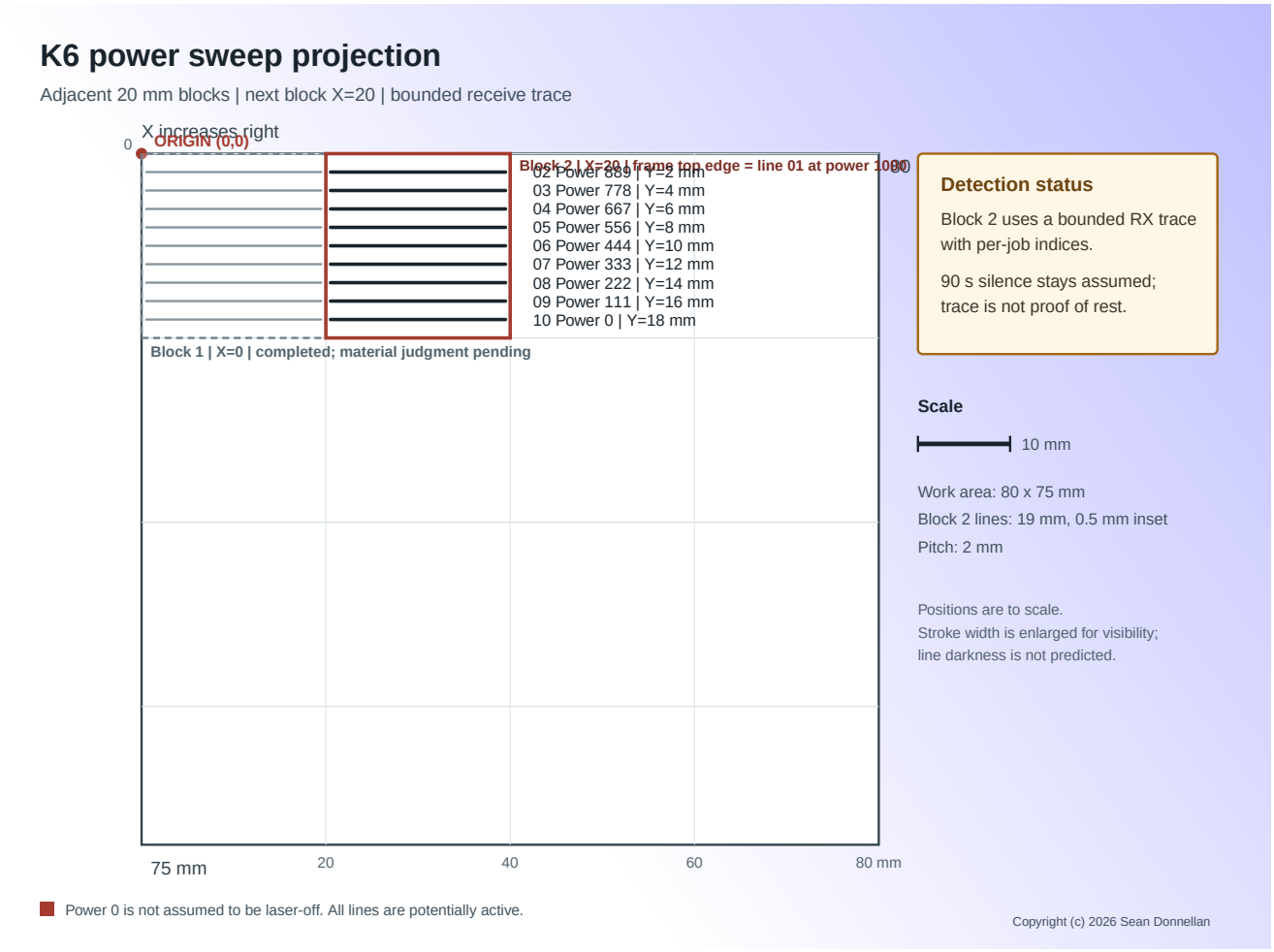


Figure 43. To-scale projection of both adjacent 20 mm blocks, with X=20 selected for the next traced run

4.9.3. Protocol investigations

- **Job completion:** No signal reliably proves physical rest. Preserve the reported-100% and 90-second silence-assumption rules. Compare bounded traces and audio with attended stops before changing them; never auto-advance on silence.
- **Pi-local capture:** Check microphone and OS support without installing packages or changing the Pi. Keep iPhone recording available.
- **Vector jobs:** The K6 driver remains raster-only. Do not infer support from reserved fields. Packet hypotheses and safety-gated experiments remain in `documentation/unverified-protocol-experiments.adoc`.

4.9.4. QR experiments

- **Modified QR patterns:** Explore candidates from HQR and its DrawOnQR tool. Identify which can be rendered as the project's raster input while staying scannable after engraving. The goal is laser-friendly codes that remain visually distinctive:
  - A human-readable text rectangle; the linked example uses one rectangle with two lines.
  - Custom pencil or rectangle marks, starting with metadata and alignment/timing patterns protected while the tool optimizes QR parameters around the drawing.
  - Marks that alter QR metadata, using the tool's separate metadata option.

    - Marks that alter alignment or timing patterns. The tool warns that these may not scan with all readers, so treat them as higher-risk trials.

- **Halftone QR:** Try a small, controlled test based on Halftone QR. This is lower-confidence for a laser because engraving reduces the pattern to material marks, but a supervised scan test can establish whether any useful effect survives.

Appendix B: Architecture Decisions

Only decisions that remain in force for the Rust service are listed here. Sean Donnellan approved these project decisions; superseded implementation history was removed from the working docs, with provenance left to Git history.

Clean-room methodology is summarized in Clean-Room Statement. * ADR-002: K6 serial transport * ADR-006: Clean-room protocol work * ADR-007: AsciiDoc * ADR-010: Bytefield diagrams * ADR-013: Simulator testing * ADR-019: HTTP review boundary * ADR-021: Rust service * ADR-022: ARM packaging * ADR-025: Card placement * ADR-026: Network MCP hub * ADR-027: Audio-first end-detection evidence * ADR-028: Pi-coordinated capture and external analysis * ADR-029: Local Container Builds for Supply Chain Security

Appendix A: ADR-002: K6 Serial Transport

4.B.A.1. Status

Accepted; current.

4.B.A.2. Decision

The Rust `k6` crate owns all K6 protocol framing and communicates through the Linux serial device, normally `/dev/ttyUSB0` at 115200 baud. The CP210x bridge is accessed through the kernel serial driver; the application does not speak USB directly.

The `laser::Machine` contract keeps serial details out of composition and HTTP handling. Host tests use the simulator and do not require a serial device.

4.B.A.3. Consequences

- Only the K6 driver opens the serial port.
- A second process must not own `/dev/ttyUSB0` at the same time.
- Packet evidence and observed byte layouts are in the K6 protocol reference.

4.B.1. ADR-006: Clean-Room Protocol Work

4.B.1.1. Status

Accepted; ongoing constraint.

4.B.1.2. Decision

Describe observed bytes and device behavior, not vendor source structure or implementation. The current Rust service is independently implemented from observable protocol behavior. Decompiled material was used only as a verification aid; it is not included or translated into the service.

4.B.1.3. Consequences

- Keep packet captures, hardware observations and uncertainty distinct from implementation claims.
- Do not copy vendor code, names, comments or design structure.
- The methodology and contributor boundary are summarized in Clean-Room Statement. This record makes no legal conclusion.

4.B.2. ADR-007: AsciiDoc

4.B.2.1. Status

Accepted; current.

4.B.2.2. Decision

Use AsciiDoc for repository documentation, with PlantUML for process diagrams and Bytefield-SVG for packet layouts. Keep sources beside the documents that use them.

4.B.2.3. Consequences

- Validate documentation with `./scripts/test_docs.sh` from the repository root.
- Keep generated diagrams reproducible from checked-in source where a generator exists.

4.B.3. ADR-010: Bytefield Protocol Diagrams

4.B.3.1. Status

Accepted; used by the K6 protocol reference.

4.B.3.2. Decision

Keep packet layouts as EDN specifications and generated SVGs. `scripts/generate_diagrams.sh` renders the specifications in `documentation/images/` with the `docker/docker-bytefield` container.

4.B.3.3. Consequences

- EDN is the editable source; SVG is the rendered documentation asset.
- These diagrams document observed packet layouts. They are not runtime configuration or a claim that every documented opcode is exposed by the Rust service.
- The Pi does not need Node.js or Bytefield-SVG.

4.B.4. ADR-013: Simulator Testing

4.B.4.1. Status

Accepted; current.

4.B.4.2. Decision

Use the Rust simulator for API and workflow tests that must not access the K6. `MACHINE=sim` removes the serial-device dependency; tests exercise the HTTP contract against that driver.

This is simulation, not a real-device dry-run. Real mode can move and burn. A preview or simulator verdict is not evidence of physical completion.

4.B.4.3. Consequences

- Keep hardware-independent tests deterministic and explicit about their simulator driver.
- Do not describe the retired Python dry-run toggle or its operation modes as Rust features.
- Physical acceptance remains an operator observation recorded separately from software outcome.

4.B.5. ADR-019: HTTP Review Boundary

4.B.5.1. Status

Accepted; current, with an explicit trust limit.

4.B.5.2. Decision

The Pi service is the single owner of the K6 serial port. A producer may submit a job for validation and preview through HTTP; submission does not execute it. The operator reviews the raster, then separately plans and runs each stage and records a verdict.

The API is for a trusted LAN. Producer and operator roles are not authenticated, and there is no per-client ownership lease. This boundary is workflow separation, not access control.

4.B.5.3. Consequences

- Browser and external clients use the same HTTP contract; neither accesses serial directly.
- Do not expose the service to an untrusted network.
- Reviews are held in process memory; run, stop and verdict events are appended to `DATA_DIR/runs.jsonl` without submitted job payloads.
- Restart loses the active review; the JSONL log is not a job database or restart-recovery mechanism.

4.B.6. ADR-021: Rust Service

4.B.6.1. Status

Accepted; current.

4.B.6.2. Decision

The Pi service is a Rust workspace with four responsibilities:

- `app`: HTTP API, review/run/verdict workflow and embedded UI.
- `compose`: pure content, placement and raster generation.
- `k6`: K6 serial protocol driver.
- `laser`: machine, raster, settings, progress and stop contracts, plus simulator.

The service handles raster jobs. Each user stage requires a separate plan and run request; a later card side requires the prior stage verdict and explicit placement confirmation.

4.B.6.3. State and lifetime

Active review and attempt state are in process memory and disappear on restart. The service appends redacted run, stop, mode and verdict events to `DATA_DIR/runs.jsonl`. It does not persist submitted QR/text payloads or raster jobs. Public assets and TOML definitions are files, not user job state.

4.B.6.4. Consequences

- Protocol and hardware behavior stay in the K6 driver; composition is testable without a laser.
- There is no job queue, per-client session owner or restart recovery. These are current limits, not promised features.
- The canonical API contract is included at Laser Service API and served at `GET /api`.

4.B.7. ADR-022: ARM Packaging

4.B.7.1. Status

Accepted; current.

4.B.7.2. Decision

Build the Pi ARMv6 container away from the Pi. The Rust cross-build targets `arm-unknown-linux-musleabihf`; the runtime image copies in the binary and configuration. GitHub Actions also builds the ARMv6 image from `docker/docker-wainlux-rs/Dockerfile`.

The root-level `./scripts/build_laser.sh` is the canonical workstation workflow: it runs `cargo test --workspace --locked` in a pinned Rust container, then builds the Dockerfile as `linux/arm/v6` and tags it `localhost/laser-app:arm`. The release binary targets the Pi Zero W's `arm1176jzf-s` CPU. It auto-selects a running Docker or Podman engine; `CONTAINER_ENGINE` can select one explicitly. Inside Flatpak, host commands use `flatpak-spawn --host` or `host-spawn`. The Dockerfile pins the cross-compiler and Alpine multi-platform image indexes; Cargo dependencies are locked.

The target is the original Raspberry Pi Zero W (`armv6l`), not the Zero 2 W. After a successful workstation build, `./scripts/deploy_laser.sh --target USER@PI-HOST --confirm` streams the finished image to the explicitly selected host, loads it with the Pi's Docker runtime, and recreates the Compose service only after checking the exact board and that the API is idle, in Simulator mode, and has no in-memory review. It verifies the loaded image ID and service state. Neither script starts a laser job; deployment is explicit and separate from building.

The Pi Compose file in `docker/docker-wainlux-rs/compose.yaml` runs the loaded `localhost/laser-app:arm` image, mounts `/dev/ttyUSB0` and persistent `data/`, and serves port 8081. Compose does not compile the service on the Pi, and the build script does not transfer or deploy the image.

4.B.7.3. Consequences

- Pi Zero W is a runtime target, not a Rust build host; no compilation occurs on the Pi.
- Keep image architecture, target triple and serial-device mapping aligned with the actual Pi deployment.
- Build and deployment commands belong in the repository's root-level scripts and central Getting Started guide.

4.B.8. ADR-025: Card Placement

4.B.8.1. Status

Accepted; current.

4.B.8.2. Decision

Placement uses millimetres and the machine's declared reach. The current K6 profile is 80 x 75 mm. For the measured 85.45 x 53.72 mm bamboo card, the default left placement uses a 1 mm margin; its centered reachable etch area is 72.55 x 53.72 mm.

The optional guide is generated only on sides that fit wholly inside reach. For the default card placement it is an open-right U, with a 0.5 mm line 0.2 mm outside the piece. Items are positioned in the card coordinate frame and translated to bed coordinates. Any content outside the etch area is refused, not clipped.

4.B.8.3. Consequences

- Materials and per-machine settings are declared in `docker/docker-wainlux-rs/materials.toml`.
- Use the 80 x 75 mm profile for normal jobs. The 75 mm Y value is an operator-selected working bound; it is not an exact stop measurement.
- The configured backing dimensions are a placeholder until the physical board is measured.

4.B.9. ADR-026: Network MCP Hub

4.B.9.1. Status

Accepted by Sean Donnellan. The read-only Streamable HTTP service may run on the Pi or a trusted

operator host; the current Deck-local instance calls the Pi API over Wi-Fi.

4.B.9.2. Decision

Run the MCP server as a separate Streamable HTTP process/container, not as part of the Rust app or a stdio child of the IDE. It is an HTTP client of the Rust service and never opens `/dev/ttyUSB0`. Hosting it beside the IDE allows loopback-only MCP binding while leaving the Pi API on the trusted LAN.

The read-only tool slice provides machine/material/template discovery, non-mutating job preview, cursor-based run-log/state polling, and attempt-scoped audio timing/report metadata. It does not create or replace a retained review and has no plan, run, stop or verdict tools. Streamable HTTP push notifications are not implemented or relied upon; clients pass the returned cursor to fetch deltas. The operator UI remains the only supported path for inspecting exact raster pixels and starting work.

4.B.9.3. Consequences

- The MCP container's Rust API URL is configurable. The Deck-local instance reaches the Pi over Wi-Fi and exposes MCP only on Deck loopback port 8082.
- Keep it on the private network and use firewall/TLS/access controls where available; the service API itself has no authentication.
- The current MCP and laser APIs have no authentication. Do not expose either publicly.
- Future mutating MCP tools require an explicit ADR and authorization design; do not add burn controls by implication.

4.B.10. ADR-027: Audio-First Evidence for Physical End Estimates

4.B.10.1. Status

Accepted for evidence collection. Automatic physical end estimation remains unimplemented.

4.B.10.2. Context

The K6's visible status has not been shown to reliably mark physical motion stop. The Rust service accepts a reported 100-percent frame as device completion and currently assumes completion after 90 seconds without received bytes. Silence is not proof of physical rest. The operator can hear head movement, while a camera may be useful for visible illumination/marking; the Pi's camera microphone is unknown. A phone can record room audio independently.

The operator needs to associate an uploaded recording with per-job protocol events without installing audio software on the Steam Deck or changing the Pi build/runtime. Sound itself is not used to infer laser state or physical rest.

4.B.10.3. Decision

- Keep the existing reported-completion and 90-second assumed-completion rules unchanged while evidence is gathered. Never label silence as device confirmation.

- Prefer audio as the first low-friction observation channel because mechanical motion is audible. The target architecture uses Pi-coordinated local capture and an external analysis worker; it does not require decoding on the Pi. A phone remains a valid alternate capture source.
- When recording externally, the UI can play a three-tone preamble and six standard DTMF digits before `/api/run`. Pi-local capture can instead correlate capture-relative monotonic times with the run log and does not need an audible cue. The six-digit external recording ID is the last six decimal digits of the planned attempt's timestamp ID; it is a short label, not a globally unique identifier. The run log remains authoritative for the full ID.
- Capture bounded receive traces for each power-sweep job: monotonic relative time, job index, received bytes, decoded status/heartbeat frames, progress and quiet intervals. Cap raw RX bytes and event count; report truncation. Never capture outgoing raster/job data or user content.
- Process recordings with `./scripts/analyze_laser_audio.sh` in a pinned Docker/Podman container. The audio file is mounted read-only. FFmpeg decodes the recording; Python's standard-library Goertzel implementation reads the DTMF ID only for association with the run log. The report presents playback, a full-length zoomable amplitude envelope, timestamp-aligned received bytes and decoded frames when a receive trace exists, run events and measured processing timings. Older/untraced runs cannot show packet arrivals. It does not classify audible activity. No audio tools or Python packages are installed on the Deck.
- Keep OK/deviation/comment as a separate human judgment. Logs, timings, sync matching and playback do not prove physical completion.
- Camera/video is optional evidence, not a prerequisite for audio analysis. Add it only when a specific unresolved signal needs it and it can share the same attempt and time reference.

4.B.10.4. Evidence and Limits

The first audio-sync test used a short 2 mm laser-off frame at X=20. The M4A decoded to recording ID `321995`, matching attempt `1794321995-frame`. The preamble began at 15.8 seconds in the file; the digits followed at 16.48-17.70 seconds. The nominal run request was 18.5 seconds. An earlier exploratory report also listed acoustic intervals; that output is archived and is not part of the current analysis or a validated motion label.

The M4A embedded creation time differed from the API marker wall clock by about 2 hours 40 minutes, while the DTMF ID matched. Prefer the audible ID and within-recording relative timing over file metadata until clock provenance is understood.

The adjacent X=20 power-block trace captured 1,024 raw bytes and 1,454 events, then marked truncation. Decoded progress/quiet events were present for all ten jobs; raw bytes were exhausted during job 1. Frame and first-row statuses reached 99%; remaining rows reported 0%. All jobs ended under the existing 90-second silence assumption. No distinct terminal event was established, so no production completion-policy change is justified.

4.B.10.5. Consequences

- The run is reproducible with a compact audio code and the full attempt ID remains in the audit log.
- The container gives ordinary Docker/Podman hosts a repeatable analyzer and avoids modifying

the Steam Deck host.

- Container/image/decoder metadata, DTMF confidence and activity thresholds must accompany future reports; false decodes and room-noise intervals remain possible.
- Physical correlation remains an attended operator task. No automatic next-job or next-band decision may rely only on this trace or audio analysis.

4.B.10.6. Staged Work

1. Phone-audio proof of concept: decode the six-digit marker and inspect a recording. Complete for one real 2 mm frame recording; creation-time metadata was offset from the Pi log. Its self-contained report is archived at `/examples/audio-analysis`.
2. Visualization and log join: implemented in the analyzer. Current reports include playable audio, a full-length min/max amplitude waveform with amplitude/time zoom, seeking and Jump to end, receive-byte/frame markers with an aligned packet table, sync/log events and timings. This is visual evidence only; no acoustic activity classifier or physical-state inference is performed.
3. Pi capture integration: coordinate local microphone capture and the laser run from the Pi, use monotonic capture-relative event times, then submit audio and filtered run metadata to an external analysis service. Keep the optional cue for phone/independent capture. Measure capture CPU, memory and storage before enabling it in routine runs; see ADR-028.
4. Synchronization accuracy: compare audio/video markers, receive trace, and attended physical stop across short non-marking frames and representative material runs. Quantify offsets, missed detections and false acoustic activity.
5. End-estimate evaluation: only after accuracy is measured, evaluate a conservative end estimate and whether the 90-second safety margin can be reduced. Keep silence-assumed completion and operator confirmation as fallback; do not make a candidate estimate an automatic progression trigger until evidence supports it.

4.B.11. ADR-028: Pi-Coordinated Capture and External Audio Analysis

4.B.11.1. Status

Manual iPhone upload, analyzer pass-through and report proxy are implemented. The LAFVIN audio/display HAT is ordered but not installed or tested; local capture remains pending.

4.B.11.2. Context

The Raspberry Pi Zero W owns the laser connection. Its OV5647 camera is available, but no microphone is currently connected. USB audio capture is out of scope because the OTG port is occupied by the laser interface and the extra dongle/cable adds unwanted bulk. Audio decoding and report rendering are separate workloads and should not be added to the constrained laser service process.

4.B.11.3. Decision

- The dedicated audio test uses iPhone Voice Memos. The operator confirms recording readiness in the Pi UI before the Pi plays the attempt ID cue and runs the approved test. Future Pi-local HAT capture will instead start before the run and associate audio using a monotonic capture-

relative time origin.

- An external analysis service accepts the recording and only the filtered run records needed for correlation. It performs decoding and report generation; it has no laser-control credentials or serial-device access.
- The Pi UI remains the operator's entry point. Any completed Real attempt with an operator-confirmed recording and matching sync marker may be uploaded; the Pi passes only that marker and run event to the analyzer, then proxies the report through the Pi origin. The Pi does not persist the audio or returned report. Upload and analyzer availability do not gate ordinary runs.
- Keep two capture modes permanently: manual iPhone Voice Memos import and Pi-local capture when a supported microphone HAT is installed. Manual upload must not be removed when local capture becomes available.
- The analyzer owns durable audio, filtered run-log events, and HTML reports in its mounted data directory, keyed by attempt ID. It has no automatic restart policy. When it is stopped or not configured, hide the audio-only test and report controls; ordinary Pi laser operation and the existing power sweep remain available. If an upload never reaches the analyzer, the original remains in Voice Memos for retry or local-CLI use.
- A human explicitly confirms the stock, plans, and starts every physical run in the Pi UI. MCP and analysis tools are read-only and cannot fire the laser or apply parameters.
- Keep the audible sync cue as an optional mode for phone or other independently started recordings. Do not make the audio analyzer a run trigger.
- The selected HAT candidate is the [LAFVIN Audio Display HAT](https://lafvintech.com/products/lafvin-audio-display-hat-for-raspberry-pi-with-1-69-inch-color-display-screen-audio-display-expansion-board). Its published specifications list dual MEMS microphones, a WM8960 codec, an 8-ohm/1-watt speaker, RGB LED, one physical button, and a 1.69-inch 240x280 ST7789 SPI display. The display is non-touch and is suitable for concise status and warning states, not as a replacement for the browser operator UI.
- Do not treat the HAT as compatible with the current original Pi Zero W. LAFVIN's [user guide](https://lafvin-audio-display-hat.readthedocs.io/en/latest/about_this_kit.html#compatibility-and-requirements) lists Zero 2 W, Pi 3 B+, Pi 4 and Pi 5, recommends 64-bit Raspberry Pi OS, and does not list Zero W. Verify the exact board revision, ARMv6/32-bit support, GPIO/SPI use, power draw, drivers and service changes before installation. Do not run the vendor installer on the laser Pi as a probe.

4.B.11.4. Data Flow

For any Real run, the operator may start Voice Memos and check the optional recording confirmation before starting. The Pi then plays the attempt ID cue. Without that checkbox, ordinary run behavior is unchanged. The two dedicated audio profiles use the same 20 mm box plus nine 19 mm lines, each at one fixed power: minimum nonzero power (currently 1/1000, nominally no-mark but not guaranteed) and full power (1000/1000, marking expected). These remain separate analyzer-gated manual runs with no automatic transition. After a marked run completes, its iPhone `.m4a` can be uploaded when the analyzer is available. The analyzer stores the source media, filtered events, and report on its own volume. The Pi proxies the report without retaining audio or report copies.

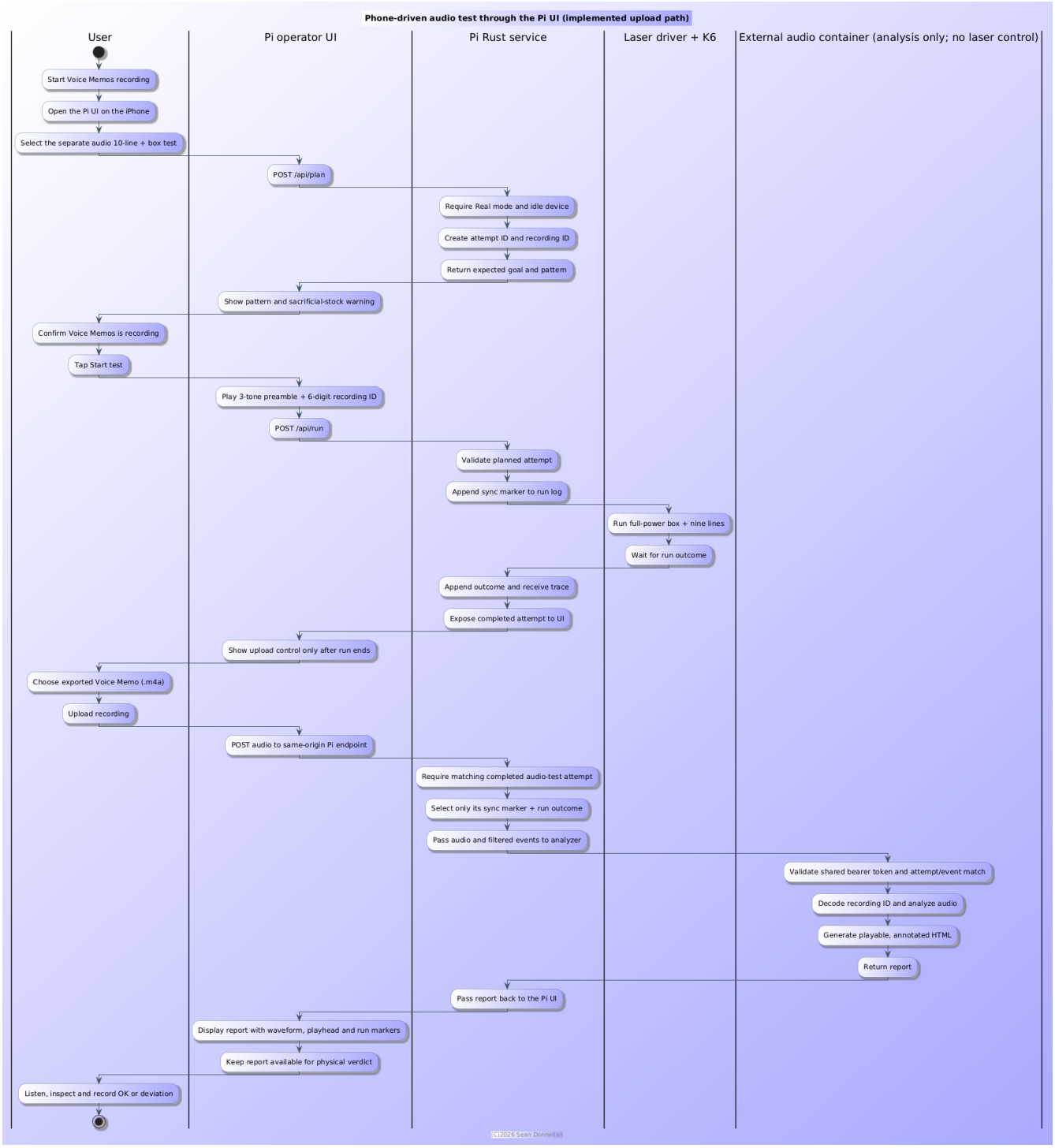


Figure 44. Implemented phone-driven test and upload pass-through

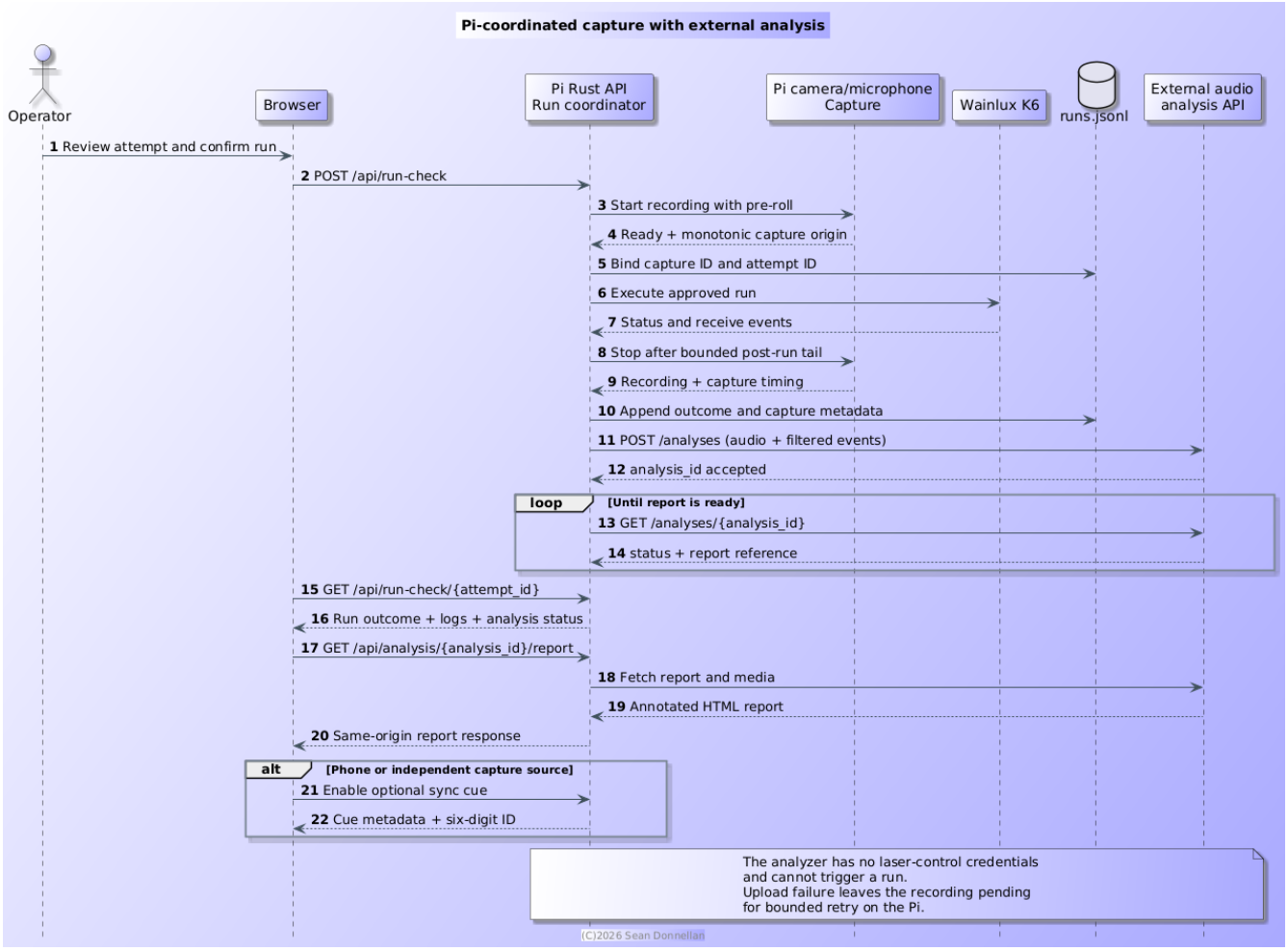


Figure 45. Future Pi-local HAT capture

The Pi upload route accepts a completed Real attempt with its matching sync marker. It accepts a raw recording up to 64 MiB and forwards only the matching marker and run event to the configured analyzer. The analyzer URL and bearer token are server-side configuration; neither is exposed to the browser. The report provides recording playback, a full-length seekable min/max amplitude waveform, sync/run-log markers and measured processing timings. The Analysis tab lists the latest ten runs for goal/settings and duration comparison, with links to each uploaded waveform. Sound is not classified as laser state and does not establish physical rest; the human physical verdict remains authoritative.

4.B.11.5. Initial Camera Feasibility Check

On 2026-11-11, an initial read-only check of the Pi Zero W Rev 1.1 on Raspberry Pi OS 13 (kernel `6.18.60+rpt-rpi-v6`) found `/dev/video*` and `/dev/media*` nodes and the `rpicam-hello` and `v4l2-ctl` utilities. The SSH user belongs to the `video` group, but the first `rpicam-hello --list-cameras` call reported `No cameras available!` and `vcgencmd get_camera` reported `supported=0 detected=0`. No image was captured during that first check.

A later check on the same day enumerated an OV5647 (`/base/soc/i2c0mux/i2c@1/ov5647@36`) with modes up to 2592x1944. `rpicam-still --nopreview --timeout 1000` successfully captured a 2592x1944 JPEG (about 1.2 MB). The image was visually inspected. This confirms Pi-local still capture is available; video and microphone capture, sustained capture load, and synchronization with a run remain untested. No laser job was started and no software was installed.

4.B.11.6. Consequences

- The Pi is a bounded transient pass-through; the analyzer volume is the only durable home of the source audio, filtered run events and report.
- The analyzer has no automatic restart policy. Stopping it releases CPU/RAM but preserves its volume; while it is unavailable, the audio test and report UI are hidden and ordinary laser operation continues.
- The original iPhone recording remains available in Voice Memos until the analyzer accepts and persists it. The local-file CLI is an independent fallback.
- The Pi UI's confirmation and explicit run action remain the sole laser-start authority. The MCP and analysis services provide evidence/suggestions only, not run or settings mutations.
- Keep both the unauthenticated Pi API and authenticated analyzer on a trusted network; firewall the analyzer port to the Pi.
- No end-detection policy or automatic job progression changes as part of this decision.

4.B.11.7. Measured Upload Sizing

The real frame sample embedded in the UI report originated as an 836,997-byte M4A for 50.154667 seconds, about 16,688 bytes/second (133.5 kbit/s). A successful Wi-Fi card content run lasted 2,054 seconds. Adding the sample's approximately 36.2 seconds of pre/post-run audio estimates 34.9 MB (33.3 MiB). A 16 MiB limit covers only about 1,005 seconds; the matching 64 MiB cap gives about 1.9x headroom. Actual Voice Memos bitrate and duration may vary. The verification working folder has been removed; the self-contained HTML demo preserves playback and annotations.

4.B.11.8. MCP and Future Analysis

The Pi MCP hub is intended as a read-only discovery/query path for attempts, run logs, operator comments/verdicts, analyzer metadata/reports, and eventually Pi-camera stills. Audio is one independent tool among future camera and pattern-analysis tools. An LLM may compare evidence or suggest end-detection/material experiments, but must not fire the laser or silently apply settings. Human review and the explicit Pi UI plan/run flow remain authoritative. Camera scanability analysis is future work.

4.B.11.9. HAT Follow-Up

Before enabling HAT capture, resolve the original Zero W compatibility gap or move the laser service to a supported Pi. Test microphone capture, audio/display load, RAM/CPU, 5 V power, speaker, and warning/status UI. The HAT button must not add another laser-start or authorization path.

4.B.11.10. Alternatives

- Keep iPhone Voice Memos as a permanent manual capture mode even after HAT capture is available.
- Keep decoding/report work off the Pi so the analyzer can be stopped when not needed.
- Do not expose a laser-start operation through MCP or the analyzer.

4.B.11.11. Related

See ADR-027 for evidence policy and FR-017 for acceptance criteria.

4.B.12. ADR-029: Local Container Builds for Supply Chain Security

4.B.12.1. Status

Accepted; ongoing constraint.

4.B.12.2. Context

The project uses Docker containers for: * Cross-compiling Rust code for ARMv6 (Pi Zero W) * Generating protocol diagrams (bytefield-svg) * Generating architecture diagrams (PlantUML) * Running the laser service and supporting tools

Historically, some containers used remote images (e.g., `plantuml/plantuml:latest`, `rust:1.x`), while others used local Dockerfiles (e.g., `docker-bytefield`). This inconsistency creates supply chain risks and maintainability issues.

4.B.12.3. Decision

All tool containers must use local Dockerfiles in the `docker/` directory with pinned base images.

- Build containers once from pinned base images
- Scan containers for vulnerabilities before use
- Version containers in git history
- Never use `:latest` tags for tool containers

4.B.12.4. Consequences

4.B.12.4.1. Positive

- **Supply chain security:** We control all dependencies; no implicit trust in external registries
- **Reproducibility:** Exact container versions are committed to git
- **Rollback capability:** If a build breaks, git history shows exactly what changed
- **Cost control:** Avoids Docker Hub rate limits and bandwidth costs
- **Vulnerability management:** We scan containers before deployment

4.B.12.4.2. Negative

- **Maintenance burden:** More Dockerfiles to maintain
- **Storage overhead:** Larger repository size
- **Build time:** Initial build takes longer

4.B.12.5. Rationale

4.B.12.5.1. The `:latest` Problem

Using `:latest` tags creates several risks:

- **No version history:** When `:latest` breaks, we cannot determine what version was previously working
- **No rollback path:** There is no tag to revert to
- **Implicit updates:** New versions may be pulled automatically, introducing breaking changes or vulnerabilities
- **Audit difficulty:** Cannot trace exactly what was used in historical builds

4.B.12.5.2. The Local Dockerfile Solution

With local Dockerfiles:

- **Git is the source of truth:** All container definitions are in the repository
- **Versioned by commit:** Every change to a Dockerfile creates a new, auditable version
- **Rollback via git:** `git checkout <commit>` restores the exact container that was used
- **Scan before merge:** Vulnerability scanning can be part of the CI/CD pipeline

4.B.12.6. Implementation

4.B.12.7. Container Inventory

Container	Directory	Status	Action
bytefield-svg	`docker/docker-bytefield/`	□ Local Dockerfile	Keep
plantuml	Not in docker/	□ Remote `:latest`	Create local Dockerfile
rust-cross	Not in docker/	□□ Remote pinned	Create local Dockerfile
wainlux-rs	`docker/docker-wainlux-rs/`	□ Local Dockerfile	Keep
mcp	`docker/docker-mcp/`	□ Local Dockerfile	Keep
audio-analysis	`docker/docker-audio-analysis/`	□ Local Dockerfile	Keep

4.B.12.7.1. Build Process

Create `docker-compose.build.yml` at repository root:

```
version: '3.8'
services:
  bytefield-svg:
    build: ./docker/docker-bytefield
    image: localhost/bytefield-svg:latest
```

```
  plantuml:
    build: ./docker/docker-plantuml
    image: localhost/plantuml:latest
  rust-cross:
    build: ./docker/docker-rust
    image: localhost/rust-cross:arm
```

Build all tool containers:

```
docker compose -f docker-compose.build.yml build
```

4.B.12.7.2. Scanning

Scan all local containers before use:

```
trivy image localhost/bytefield-svg:latest
trivy image localhost/plantuml:latest
trivy image localhost/rust-cross:arm
```

4.B.12.7.3. Versioning

After scanning, tag containers with semantic versions:

```
docker tag localhost/bytefield-svg:latest localhost/bytefield-svg:v1.0.0
docker tag localhost/plantuml:latest localhost/plantuml:v1.0.0
docker tag localhost/rust-cross:arm localhost/rust-cross:v1.0.0
```

Update scripts to reference versioned tags.

4.B.12.8. Examples

4.B.12.8.1. PlantUML Container

docker/docker-plantuml/Dockerfile

```
FROM eclipse-temurin:17-jre-
alpine@sha256:589a938476b126362d575a248e24c9b795947319c3d6283f54482e79744f8444

RUN apk add --no-cache openjdk17-jre
RUN mkdir -p /opt/plantuml && cd /opt/plantuml && \
    curl -L
https://github.com/plantuml/plantuml/releases/download/v1.2026.10/plantuml-
1.2026.10.jar -o plantuml.jar

WORKDIR /diagrams
ENTRYPOINT ["java", "-jar", "/opt/plantuml/plantuml.jar"]
CMD ["-help"]
```

4.B.12.8.2. Rust Cross-Compile Container

docker/docker-rust/Dockerfile

```
FROM rust:1.85.0-
slim@sha256:9daf2b244ccc30523e55cafe32edf8465671ba567d0101620e5612ccb76ea1a7

RUN rustup target add arm-unknown-linux-musleabihf
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc-arm-linux-gnueabihf \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /workspace
```

4.B.12.9. Related Decisions

- ADR-022: ARM Packaging - Service build and deployment workflow
- ADR-006: Clean-Room Protocol Work - Clean-room methodology that benefits from supply chain security

4.B.12.10. Notes

The `:latest` tag should never be used for tool containers. For runtime containers (like the laser service), `:latest` may be acceptable if the service itself handles versioning, but tool containers that affect build reproducibility must be pinned and local.

Appendix D: Requirements

Active requirements are listed below. Numbering gaps are intentional; superseded and proposed work was removed.

4.D.1. Functional

Historical physical evidence is in Hardware Observations. * FR-001: Reviewed raster execution * FR-002: WiFi QR composition * FR-003: Progress and timing * FR-007: Stop and cancellation * FR-009: Test ladder * FR-011: Material placement * FR-013: Hardware-independent tests * FR-015: Operator test verdicts * FR-016: Card review workflow * FR-017: Physical end estimates

Appendix C: FR-001: Reviewed Raster Execution

4.D.C.1. Status

Implemented; see the service API contract and HTTP tests.

4.D.C.2. Requirement

Compose supported card content into bounded 1-bit raster stages. A user must be able to inspect the exact pixels and placement before execution. Planning and running are separate operations.

4.D.C.3. Acceptance

- Review responses contain exact per-stage PNG/SVG pixels, dimensions and physical placement.
- Settings and geometry are validated; content outside the declared etch area is rejected, not clipped.
- A plan names a reviewed stage; only a separate run request accesses the selected machine.
- Software outcome and operator verdict are recorded separately.

4.D.C.4. Scope

Content kinds are QR, WiFi QR, text, prepared public assets and checkboxes. Arbitrary image uploads and vector burns are not part of this contract. See the Rust service README and `app/tests/api.rs`.

4.D.2. FR-002: WiFi QR Composition

4.D.2.1. Status

Implemented in the `wifi-login` use case; physical acceptance is material and phone dependent.

4.D.2.2. Requirement

Compose a WiFi payload as a QR and preview the resulting raster before a stage can run.

4.D.2.3. Acceptance

- Accept SSID, password, security type and hidden-network flag through declared fields or the typed API.
- Encode credentials in the QR; never bind the password to printed text.
- Keep QR size and position within the selected material's etch area; reject overflow.
- Return preview pixels and dimensions before planning or running.

4.D.2.4. Evidence

The included API contract defines the template behavior. Physical results and their limits are in Hardware Observations.

For background on QR structure, see QR decoding by hand. This is reference material, not a planned project task.

4.D.3. FR-003: Progress and Timing

4.D.3.1. Status

Implemented; Rust API and UI tests cover the structured report.

4.D.3.2. Requirement

Report progress and elapsed work without implying that one band percentage is whole-job progress or that software completion proves physical rest.

4.D.3.3. Acceptance

- `/api/state.progress` identifies phase, band, source rows and band-local percentage.
- Completed bands appear in `band_timings` and the run outcome with `transfer_ms`, `setup_ms`, `wait_ms` and `elapsed_ms`.
- Reported device completion and assumed completion after 90 seconds of silence remain distinct.
- The UI labels waiting and assumed completion explicitly.

All durations are milliseconds. Historical Python timings are not performance targets for Rust. See Burn Timing.

4.D.4. FR-007: Stop and Cancellation

4.D.4.1. Status

Software cancellation implemented; physical stop behavior is not certified.

4.D.4.2. Requirement

A stop request must cancel queued work and request a device stop when a device handle exists. It must not automatically retry, start another band or authorize a later stage.

4.D.4.3. Acceptance

- Cancellation is checked during connection, upload, wait and between bands.
- The API reports whether a stop request was sent and whether work was cancelled.
- Errors, STOP and uncertain completion do not trigger automatic job, stage or band retries. The driver may retry an individual DATA chunk according to the protocol retry policy.
- An operator confirms motion has stopped before changing placement or proceeding.

A successful HTTP response is not proof of physical rest. Use the machine's physical emergency controls and supervision; this API is not a certified emergency-stop system.

4.D.4.4. Attended Hardware Check (Initial Result)

The software STOP uses a separate serial write handle and bypasses the normal device-worker queue. It still serializes with an in-flight packet write, and neither its priority nor physical effect is certified until observed on the machine.

Test whether the UI STOP can stop a job started directly from the machine's middle button, including when the service reports no active software attempt:

1. Use an attended, enclosed setup and sacrificial stock. Leave the physical emergency control within immediate reach. Select a previously reviewed, minimal, non-sensitive job; do not use a valuable card or a job with unknown settings.
2. Confirm the machine is physically at rest and the UI has no active attempt. Press the machine's middle button once. If it does not produce the expected controlled replay, stop and record the result; do not improvise with other controls.
3. While the replay is visibly active, press the persistent UI STOP once. Observe the machine independently and confirm whether motion and marking stop. Record the UI response, observed stop latency, and any remaining motion; an HTTP response alone is not evidence of success.
4. After physical rest, inspect the stock and software state. Confirm no automatic retry, next band, or later job was started. Record any mismatch between the device replay and the service's run log.
5. If STOP does not stop the replay promptly, use the machine's physical emergency control and do not repeat the test until the cause is understood.

On 2026-11-12, the operator reported one test in which a job repeated with the machine's middle button was stopped using the UI STOP control. The reported repeat stopped. Exact stop latency, independent motion/beam observation, and repetition count beyond this trial were not recorded. This is limited evidence for that scenario, not an emergency-stop guarantee; repeat under the checklist above and keep the physical emergency control within reach.

4.D.5. FR-009: Test Ladder

4.D.5.1. Status

Implemented for the K6 and simulator; individual steps have machine capability gates.

4.D.5.2. Requirement

Expose named machine tests with a stated goal and preview where available. Tests must use the same Rust machine contract as card jobs.

4.D.5.3. Acceptance

- `/api/steps` lists the steps supported by the selected machine.
- Supported steps include communication, home, frame, square, limits, blank rows, corner/registration comparisons and QR tests.
- A plan records the expected goal before execution; the API does not label a physical test OK on the operator's behalf.
- Rasters used by tests obey the machine's declared bounds unless the step explicitly tests the calibration envelope.

Run results, selected measurements, the 0–12 ladder and pattern illustrations are in Hardware Observations.

4.D.6. FR-011: Material Placement

4.D.6.1. Status

Implemented in the Rust composition layer.

4.D.6.2. Requirement

Represent the piece, backing, guide and content geometry in millimetres. Refuse any generated burn rectangle outside the K6's declared reach or the piece's etch area.

4.D.6.3. Acceptance

- Materials define dimensions, role and machine-specific settings.
- Placement supports the configured centered and left layouts; guide segments are emitted only where they fit.
- Content positions are relative to the piece and translated to bed coordinates.
- Out-of-area content is rejected without clipping or silent resizing.

The current K6 normal profile is 80 x 75 mm. Card dimensions and placement assumptions are in ADR-025 and the included API contract.

4.D.7. FR-013: Hardware-Independent Tests

4.D.7.1. Status

Implemented for simulator-backed tests. No real-device dry run exists.

4.D.7.2. Requirement

Run API, composition and workflow tests without connecting to or moving a K6.

4.D.7.3. Acceptance

- Tests use the simulator, a local HTTP port and synthetic data.
- Simulator mode never opens `/dev/ttyUSB0` or sends machine commands.
- Tests cover review, plan, run, verdict, stage gating and redacted logs.
- Simulator results do not prove burn quality, device stop or physical completion.

Preview is not a real-device dry run. Real mode can move and burn.

4.D.8. FR-015: Operator Test Verdicts

4.D.8.1. Status

Implemented for planned test steps and card stages.

4.D.8.2. Requirement

Record the goal before a run, the software outcome after it, and an independent operator verdict. Preserve the original goal when an observation differs.

4.D.8.3. Acceptance

- A plan fixes the test goal before the machine runs.
- `POST /api/run` executes only the planned attempt.
- `POST /api/verdict` records `ok`, `deviation` or `skipped`; deviation and skipped require a comment.
- `ok` requires an outcome. A software outcome alone is not a physical verdict.
- Later reviewed card stages require acceptance of earlier stages; a back stage also requires explicit placement confirmation.

Use the simulator for automated contract tests. Real-device checks require an attended operator and a recorded physical observation.

4.D.8.4. Full-Screen Operator UI Acceptance (Pending)

- On desktop, enter full screen from the UI; confirm the workflow, current machine mode and STOP remain visible and usable. Exit with the UI control and the browser's Escape shortcut.
- On iPhone Safari, add the service to the Home Screen and launch the installed app. Confirm it opens as a standalone view, respects the camera/sensor safe areas, and keeps STOP fully visible and tappable in portrait and landscape.
- If Safari exposes the Fullscreen API for the page, test the in-browser toggle as well. If it does not, the standalone Home Screen launch is the supported full-screen path.
- Verify the page remains usable when returning from full screen; do not start a laser job as part of this display check.

These device/browser checks have not yet been performed. Automated UI tests cover toggle state and persistent STOP visibility only.

4.D.9. FR-016: Reviewed Card Jobs

4.D.9.1. Status

Implemented for WiFi, vCard and GitHub recovery templates; integration and physical acceptance remain use-case dependent.

4.D.9.2. Requirement

Accept a typed card job, compose and return separate reviewed stages, and require explicit operator action for each physical stage.

4.D.9.3. Acceptance

- Supported templates are supplied by TOML definitions; API clients may submit typed content or

edit declared fields.

- Review returns side-specific raster pixels, dimensions, layout and display policy without opening the serial device.
- Front and back are separate stages. A back stage requires an accepted preceding stage and explicit flip/placement confirmation.
- Only separate plan and run requests execute a stage. Software outcome and operator verdict remain distinct.
- Private job/review data stays in process memory and is omitted from operational logs; restart loses the review.

See ADR-019 for the trusted-LAN and authentication limits.

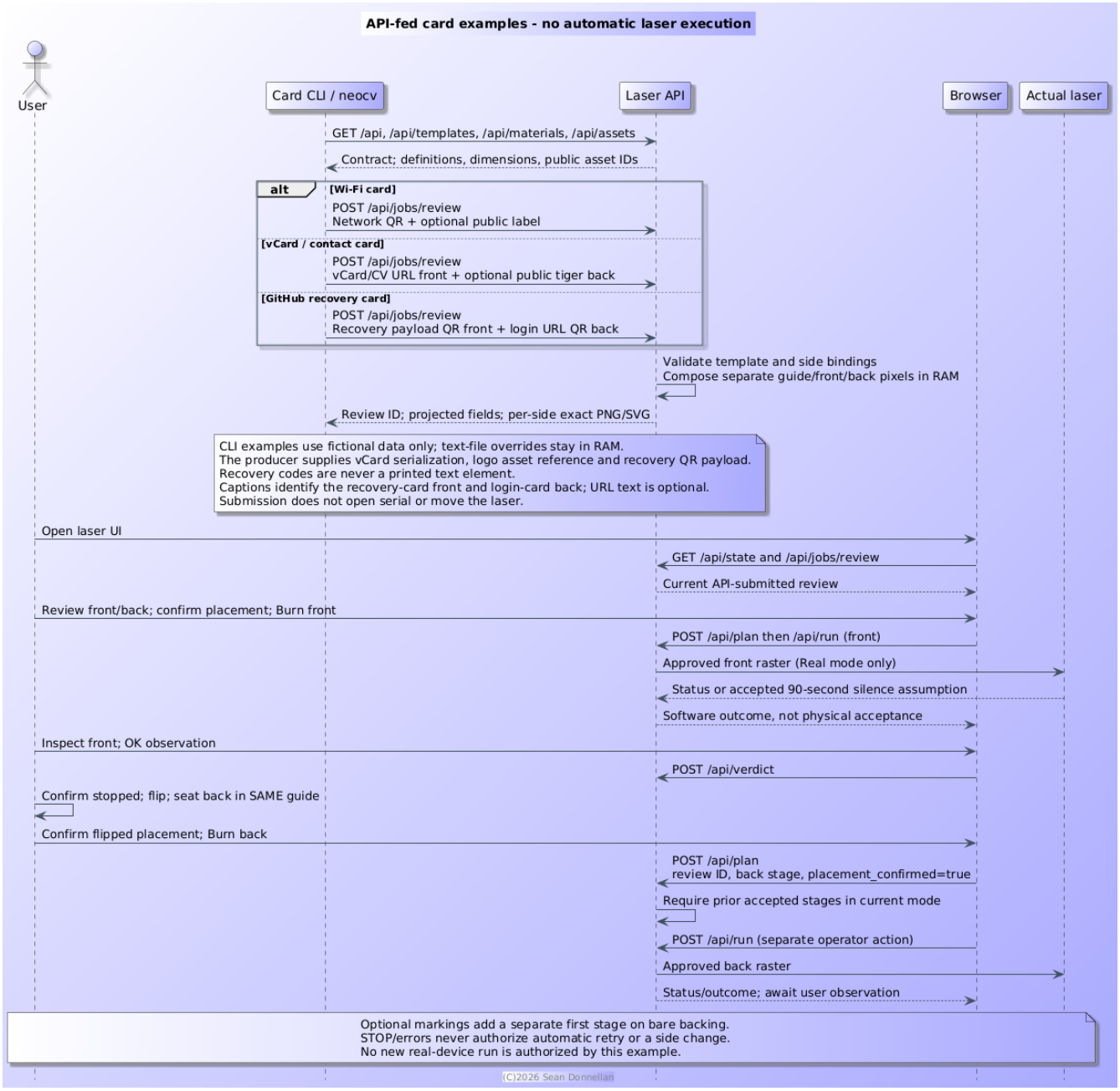


Figure 46. Current API submission, review, and operator-controlled run sequence

4.D.10. FR-017: Physical End Estimates

4.D.10.1. Status

Partially implemented. Bounded per-job receive traces and recording synchronization are available for power-sweep attempts. No validated physical end estimator exists.

4.D.10.2. Requirement

Report evidence about when laser motion or marking may have ended without claiming that a protocol response, timeout, or acoustic quiet proves physical rest. Preserve the distinction between device-reported completion and assumed completion. Use synchronized external observations to evaluate candidate end estimates before changing automatic job sequencing.

4.D.10.3. Acceptance

- The K6 driver can capture a bounded, receive-only trace per job, including monotonic event time, job index, raw received bytes, decoded status/heartbeat frames, progress and quiet intervals. The trace reports its byte/event limits and whether it was truncated.
- Trace capture records no outgoing DATA/raster bytes or submitted user content and does not change the completion policy.
- A Real-mode UI run can play a preamble plus a six-digit DTMF recording ID derived from the planned attempt ID. The audit log records the attempt ID, recording ID, nominal cue duration/lead and marker time; this metadata does not prove a microphone captured the cue.
- A containerized analysis tool accepts an audio recording read-only, decodes the DTMF ID only to associate the recording with its attempt, and presents a full-length time/amplitude-zoomable waveform, timestamp-aligned receive bytes/decoded frames when a receive trace exists, run-log markers, and measured processing timings. It does not classify audible activity or infer head motion, laser state or physical completion from sound; untraced runs must be identified as such.
- Pi-local capture associates audio with the attempt through a capture-relative monotonic time origin, without requiring an audible cue. Filtered run events and recordings are analyzed off-Pi, and the Pi UI presents processing state and the resulting report. An audible cue remains optional for phone or independent capture.
- Manual iPhone Voice Memos import remains available independently of any Pi microphone HAT. The upload is associated with a selected attempt and filtered run events; the audible recording ID is checked when present.
- A microphone HAT is not assumed compatible based only on its 40-pin connector. Validate the original Zero W's CPU/OS support, GPIO allocation, power use and capture performance before installation; do not add a USB microphone to the current OTG laser setup.
- A physical end estimate is not promoted to automatic progression until it has been correlated against attended observations over repeated runs and material/settings combinations. Ambiguous evidence requires operator judgment.
- Operator OK/deviation/comment remains the authoritative physical assessment for the current experiment.

See Next Steps and ADR-027.

4.D.11. Non-Functional

- NFR-005 defines the JSONL run-history and redaction contract. :leveloffset: +1

4.10. NFR-005: Redacted Run History

4.10.1. Status

Implemented for run, stop, mode and verdict events.

4.10.2. Requirement

Persist operational outcomes in an append-only JSONL file without persisting submitted card payloads or raster jobs.

4.10.3. Acceptance

- Events are appended to `DATA_DIR/runs.jsonl` and are available through `GET /api/log`.
- Run records retain a redacted goal, software outcome and band timings; verdict records retain the operator's status and comment.
- Submitted job and parameter objects are removed before logging; tests verify synthetic SSIDs and passwords do not appear in the log.
- The log is not a job database: active review state is in RAM and is lost on restart.

The log currently has no rotation or retention policy.

Appendix E: Images

Current diagrams used by the documentation.

- `k6-prot-*.edn` → `.svg` with Bytefield-SVG (`scripts/generate_diagrams.sh`).
- `plantuml-style.puml` is shared by all PlantUML sources. `system-context.plantuml`, `architecture-rust.plantuml`, `code-topology.plantuml`, `sequence-stage-run.plantuml`, `card-api-clients.plantuml`, `sequence-audio-capture.plantuml`, `activity-audio-test.plantuml` and `sequence-ghidra-analysis.plantuml` show system ownership and workflows. Their PNGs are generated with PlantUML.
- `card-guide.svg` is a generic open-right guide with no submitted content or credentials.
- `card-placement.svg` and `card-items.svg` are generic illustrations of the current K6/card geometry and template coordinates.
- `ui-simulator-preview.png` is a Rust operator UI screenshot with synthetic Wi-Fi values, simulator mode and no burn started. The simulator's 80 x 80 mm profile is not the measured K6 working reach.
- `hardware-nfc-reader-hw147-front.jpg`, `hardware-nfc-reader-hw147-back.jpg` and `hardware-cyd-`

2.8-tft.jpg are proposed hardware references, not installed or tested devices.

- recovery-demo-qr.png is a historical illustration containing synthetic instructions and codes, not a current Pi raster. recovery-workflow.plantuml and its PNG describe Pi-reviewed card submission, manual NFC write/read-back and recovery, not an implemented NFC adapter or CLI exporter.
- test-ladder.svg, test-case-07a-limit-stairs.svg, test-case-07c-seven-squares.svg, test-case-08-blank-rows.svg and test-case-09-corners.svg show Rust test coverage/evidence. The 9b comparison is shared from docker/docker-wainlux-rs/app/assets/test-patterns/vertical-comparison.svg.
- capture_20260127_124112.jpg, capture_20260127_124223.jpg, capture_20260127_124337.jpg and capture.2426611769506763096.jpg are recovered historical cork/PLA close-ups; they are not 9b ladder microscope captures.

Measured results are in Hardware Observations. Unverified design proposals and retired Python/UI diagrams have been removed.

5. Docker Containers

Docker Compose projects for the laser service and supporting tools live here.

The Pi runs a prebuilt ARMv6 image. Compose does not build it.

Start the preloaded Rust service on the Pi

```
cd docker/docker-wainlux-rs
docker compose up -d
```

The service uses /dev/ttyUSB0 and port 8081. Build and deployment: [build-and-test-the-rust-service].



All tool containers must use **local Dockerfiles** with **pinned base images**. Never use :latest tags.

The :latest tag creates unacceptable supply chain risk: * No version history — if :latest breaks, we cannot determine what version was previously working * No rollback path — there is no tag to revert to * Implicit updates — new versions may be pulled automatically, introducing breaking changes or vulnerabilities * Audit difficulty — cannot trace exactly what was used in historical builds

With local Dockerfiles in git: * Git is the source of truth — all container definitions are in the repository * Versioned by commit — every change to a Dockerfile creates a new, auditable version * Rollback via git — git checkout <commit> restores the exact container that was used * Scan before merge — vulnerability scanning can be part of the CI/CD pipeline

See ADR-029: Local Container Builds for Supply Chain Security for the full rationale.

5.1. Components

5.1.1. Laser Service API

Standalone Pi Zero service. Clients submit content; the operator reviews exact 1-bit rasters and starts each burn separately. `GET /api` serves this contract. The browser and external clients use the same endpoints. OpenAPI 3.1 is at `GET /openapi.yaml`; Swagger UI is at `GET /docs` and enables Try It Out for GET only. License notices are in `app/assets/swagger-ui/`.

5.1.1.1. Timing

Progress percentages are band-local, not full-side/card progress. `GET /api/state` exposes terminal bands in `band_timings` and the run outcome. Each band records `transfer_ms`, `setup_ms`, `wait_ms` and `elapsed_ms`. Sum them for run totals. Values are milliseconds.

After 90 seconds without a received byte, completion is assumed, not device-confirmed. The UI labels this separately; the next band event records whether the previous band was assumed complete.

Each review response includes `timing.raster_generation_ms` for composing stage rasters and `timing.preview_encoding_ms` for PNG/SVG preview encoding. The browser reports its own review-request round trip; the CLI prints its client round trip separately. These boundaries overlap and must not be added as if they were disjoint server stages. Each run outcome includes `elapsed_ms`; its band measurements split K6 transfer, protocol setup and completion feedback wait. The Inspection panel shows those components and the residual outside bands. For audio upload, the response includes Pi receive/prepare/total, analyzer persistence/process/report-read/total and nested analyzer sub-phases. The browser separately reports its upload/analysis response and report-download time. Analyzer sub-phases are inside the analyzer process duration; do not sum overlapping measurements.

`GET /examples/audio-analysis` serves one self-contained report generated from a real recorded 2 mm frame test. It is an interpretation example, not a benchmark, statistical baseline or proof that audible activity is machine motion. Its embedded sample is loaded only when the user expands the example in the UI.

Power-sweep and sync-marked attempts also return a bounded `receive_trace` in `GET /api/state`, the run outcome and the audit log. It includes per-job indices, monotonic event timestamps, raw received bytes, decoded frame names/bytes, progress, quiet intervals, duration, captured-byte count and truncation status. Command writes made through the request/reply driver path have write-return timestamps and protocol bytes, except DATA payloads are always omitted. Bulk DATA transfer start/end/error events form spans; band metadata records row range, geometry, power and depth. This is not a complete wire capture: out-of-band STOP and direct-write paths are not included. Capture is capped at 1,024 RX bytes and 2,048 events per attempt. It contains no outgoing DATA/raster bytes or user content. The UI displays the trace after the run. Instrumentation does not change the 100% completion rule or the 90-second silence assumption.

Audio reports use a shared clock for the signed waveform and separate Laser RX, Host TX, Jobs / bands and Run / sync lanes. The full recording fits at 1x; up to 128x redraws a continuous min/max envelope from 2 ms signed 16-bit bins of the decoded mono 8 kHz signal. At high zoom, minor ruler

ticks are 200 ms apart; fractional labels appear where pixel spacing permits. These display intervals are not a claim of sync accuracy. A draggable overview window, its resize edges, horizontal scrolling and keyboard panning replace the timeline-position slider. Amplitude accepts 0.25x through 32x with peak-relative or full-scale reference. Selecting a packet or table row retains its decoded/raw details without seeking; a separate action seeks playback. The red cursor spans every lane and reports its time and difference from the selected event. Template/review/stage and band metadata identify jobs without retaining private card content. Truncation and unavailable historical traces are explicit; missing packets or job boundaries are not reconstructed from sound. A software finish marker remains duration-aligned to the nominal request, not an observation of stopped motion.

When a trace budget fills, older packet events are evicted while job/transfer boundaries and recent progress remain available within the same limits. The trace is marked truncated; it is not a complete packet history. This keeps late progress and software-completion events available for audio comparison on long recordings.

5.1.1.2. Run Analysis With Optional Audio

When an analysis URL and token are configured, ordinary runs also enable bounded event capture without requiring a recording or sync cue. After a Real attempt finishes, a background request checks the existing analysis container and submits only the completed, redacted `ran` event. Device completion, subsequent operator actions and ordinary burns never wait for analysis; an unavailable container produces no report and does not change the outcome.

The existing audio analyzer accepts `application/json` at its unchanged `/api/analyze` endpoint with `attempt_id` and `run_log`. It persists a timing-only report without FFmpeg or an audio file. The report labels time since trace start (or run start when no trace exists), shows recorded RX/TX/job events and band timings, and explicitly marks audio absent. It never invents historical packets, sync sounds or physical completion. A later recording upload can replace the timing-only report with an audio-aligned report; a log-only request does not replace an existing audio-backed report.

The Analysis tab links either kind of report. `POST /api/audio-analyses/{attempt_id}/report` requests an existing completed Real run's report when the container is available; its Analyze action permits backfilling older runs. Old runs without a saved trace can show only recorded summary/band timings. Success appends the compatible `audio_analysis` audit event with `has_audio=false`; recording uploads use `has_audio=true`. The newest event determines the current report. The Pi retains neither report HTML nor media. Container, route and configuration names remain unchanged.

5.1.1.3. Audio Recording Sync

For Real-mode UI runs, the checked-by-default audio cue plays a short three-tone preamble followed by six standard DTMF digits. The six digits are the last six decimal digits of the planned attempt's timestamp ID and are shown in the planned-run panel. Leave the computer's speakers audible to the phone recorder; the cue takes about 2 seconds, followed by a logged 700 ms lead before `/api/run` is requested. The `sync_marker` audit event stores the attempt ID, six-digit recording ID, cue duration, lead and request timestamp. Use the digits in the audio filename or notes to match the recording to the run log. This is a timing marker, not confirmation of laser motion or completion.

A template can set `audio_recording_enabled = true` to show the optional Voice Memos confirmation for Real card stages. The card action now plans first and waits on the planned-run screen; checking recording causes the cue/marker, then the operator explicitly starts the run. After an optional upload succeeds, `/api/log` receives an `audio_analysis` event with the attempt ID, phase timings and same-origin report URL only. The Pi does not retain audio or report HTML; analyzer availability does not gate ordinary burns.

The operator UI's Analysis tab shows the latest ten completed runs as a timing breakdown graph and links an uploaded report when available. Audio analysis allows up to 900 seconds of processing; the Pi waits 930 seconds for the analyzer. If analysis times out, the upload returns an error, but the analyzer retains the source recording and matching run log under its attempt directory. Re-uploading from the same attempt QR retries analysis. == Routes

This inventory is extracted from the current `app/src/lib.rs` dispatcher. `GET /openapi.yaml` is the machine-readable contract.

Method	Path	Purpose
GET	`/`	Embedded operator UI
GET	`/api`	This API guide
GET	`/examples/audio-analysis`	Self-contained real timing/audio interpretation example; one run is not a benchmark
GET	`/openapi.yaml`	OpenAPI contract
GET	`/docs`, `/docs/`	Bundled Swagger UI
GET	`/docs/swagger-ui.css`, `/docs/swagger-ui-bundle.js`	Swagger UI assets
GET	`/docs/favicon-16x16.png`, `/docs/favicon-32x32.png`	Swagger UI icons
GET	`/assets/vsr-logo.png`	Operator UI logo
GET	`/api/machine`, `/api/mode`, `/api/materials`, `/api/health`	Machine profile, mode, material settings, and service health
GET	`/api/templates`, `/api/assets`, `/api/assets/tiger.png`	Templates and prepared assets
GET	`/api/steps`	Supported test-ladder steps
GET, DELETE	`/api/state`	Read state; DELETE clears idle RAM state but retains the run log
GET	`/api/attempt/preview`	Exact active-stage preview
GET	`/api/log`, `/api/pending-verdicts`	Redacted audit events (including timing-only `audio_analysis` events) and outstanding verdicts
POST	`/api/jobs/preview`, `/api/tests/preview`	Preview a card job or test pattern; no run
GET, POST, DELETE	`/api/jobs/review`	Read, submit, or discard a review

Method	Path	Purpose
POST	`/api/jobs/form`	Edit a template review
POST	`/api/mode`	Change real/simulator mode while idle
POST	`/api/plan`, `/api/run`, `/api/verdict`, `/api/stop`	Plan, run, judge, or request stop
POST	`/api/audio-analyses`	Upload a recording for a completed Real attempt with its matching sync marker; optional analyzer
GET, POST	`/api/audio-analyses/{attempt_id}/report`	Read a report, or request timing-only analysis for a completed Real run
GET	`/audio-upload/{attempt_id}`	iPhone-friendly Voice Memo file picker for a completed marked Real attempt
GET	`/api/audio-analyses/{attempt_id}/upload-qr.svg`	QR code linking to that attempt's phone upload page

5.1.1.4. Submit and Review

Two submission forms are available. Neither opens the serial connection or starts the laser.

- Producer: `POST /api/jobs/review` with a `job` object: `template`, `piece`, `backing`, `placement`, `guide`, `items`, optional `values`.
- Template form: `POST /api/jobs/form` with `template`, `review_id` (`null` for a new idle review), `piece`, `backing`, `placement`, `guide`, and changed `fields` keyed by template field ID. Field edit policy is enforced.

A template may set `audio_recording_enabled = true` to show the optional Voice Memos confirmation for Real card stages. This checkbox is not a run prerequisite when left unchecked and never requires the analyzer to be available. Checking it makes the UI play and log the attempt-ID cue; after the completed run, upload is offered only if the optional analyzer is reachable. Dedicated audio profiles retain their stricter recording requirements.

Each item names a declared element `id`, `side` (`front` or `back`) and `content`. Content kinds are `qr` (`text` payload), `text` (`text`, `font_size_mm`, optional `font`), `wifi` (network fields) or `image` (`asset_id`). Positions and dimensions omitted by a producer come from the template. An image asset ID refers to prepared pixels; it is not an arbitrary URL or Pi path.

Example vCard job with a prepared logo asset

```
{
  "job": {
    "template": "vcard",
    "piece": "bamboo-ntag215-light",
    "backing": "backing-first",
    "placement": "left",
    "guide": false,
    "items": [
```

```
      {"id":"contact-qr","side":"front","content":{"kind":"qr","text":"BEGIN:VCARD
\nVERSION:3.0\nFN:Jane Example\nURL:https://example.com/cv\nEND:VCARD"}},
      {"id":"logo-back","side":"back","content":{"kind":"image","asset_id":"tiger"}}
    ]
  }
}
```

The response contains `review_id`, `template`, policy-projected `fields`, geometry-only `job`, `layout`, `display`, ordered `stages` and `accepted_stages`. Each stage has `name`, `side`, `goal`, resolved `values`, and `burns` with PNG/SVG pixels, pixel dimensions and physical position/size. Different sides are separate rasters, never composited together. `guide=true` adds an optional guide stage before the front; existing markings can be reused with `guide=false`.

`GET /api/jobs/review` retrieves the current review. The UI discovers it from `GET /api/state` and loads it without a browser-file injection. `POST /api/jobs/form` edits and regenerates a review using its current ID. Stale IDs, undeclared fields and forbidden edits are rejected. `DELETE /api/jobs/review` explicitly discards an idle review; an active attempt blocks replacement/discard.

5.1.1.5. Types

- `wifi-login`: network QR plus independently optional readable labels. Password is QR-only.
- `vcard`: producer-supplied vCard text or CV URL, optional readable contact label, optional logo back. The producer owns vCard serialization/escaping and contact data; the laser does not query neocv's graph.
- `github-recovery`: recovery payload QR on the front and login URL QR on the back. Small required-by-default captions identify "GitHub MFA recovery codes" and "GitHub login to use these codes". Recovery codes are transport data, not printable page text. A second small, optional caption can print the login URL. An optional front checklist adds 16 blank boxes (2 columns x 8 rows) beside the recovery QR for manually marking used codes. It defaults off and does not contain or encode code values. A producer opts in by submitting a `use-checklist` item with `kind: checkboxes`, `count: 16`, `columns: 2`, `box_mm: 3.5`, `gap_mm: 0.8`, and `line_mm: 0.35` at the declared `check-x` / `check-y` position.

The repository's `cli/` card client only submits these same review requests. Its `inputs/wifi.json`, `inputs/github.json`, `inputs/vcard.json`, and `inputs/synthetic-codes.txt` are fictional demonstration inputs, not live credentials. `wainlux-client submit --text-file recovery-qr=FILE` loads codes from a UTF-8 file. The CLI has no PDF, QR, image or font generation; the Pi renders all card content. The operator can also paste codes or load a codes-only UTF-8 `.txt` file into the masked recovery field. The browser reads the file locally; clicking Preview submits the payload through the existing review API. Do not use a file that combines codes with unrelated instructions or a password. Fields declare optional `file_input` capability through `/api/templates`; only editable multiline fields may enable it. The browser uses that metadata for recovery-code and vCard file inputs, without template-specific form logic. File loading does not start a review or a burn; Review output submits it explicitly. The client submits both `recovery-qr` (front) and `login-qr` (back) items. Small front/back purpose captions are on by default and carry no secret data; `include-url-text` is off by default and can add the URL as an additional caption. Real recovery formats and line lengths must be previewed; overflow is rejected, not silently shrunk. Private payloads remain in RAM, not template files or operational logs.

5.1.1.6. Execute One Stage

1. Review both sides before starting. Confirm placement.
2. `POST /api/plan` with `review_id`, `stage` (zero-based) and `placement_confirmed` (`true` required for a back stage).
3. Only `POST /api/run` starts the planned stage. Send `attempt_id` equal to the returned plan ID; a mismatch is refused without replacing the plan. The browser sends it on every start. Older clients may omit it; it is not authentication. A run never advances automatically to another user stage.
4. Poll `GET /api/state`. `POST /api/stop` cancels queued driver work and requests stop; the operator confirms physical rest.
5. Record `POST /api/verdict` with `status` (`ok`, `deviation`, or `skipped` for an unperformed plan) and an observation.
6. A later stage requires accepted prior stages. Inspect the front, confirm motion stopped, flip the card and seat it against the same guide before planning the back.

Plan the reviewed front stage

```
{"review_id":"returned-review-id","stage":1,"placement_confirmed":true}
```

An edited review clears stage acceptance. Re-running an earlier stage invalidates later acceptance. Simulator/Real mode changes clear acceptance too. The K6's temporary completion policy assumes completion after 90 seconds without a response; it does not prove a physical OK.

5.1.1.7. Trust and Lifetime

This is a trusted-LAN service, not a public authenticated API. Producer/operator role separation and session leases remain open. Non-revealable fields are omitted from operator source projections, but visible raster/QR pixels can disclose their contents. Reviews and acceptance are RAM-only; restart loses them. Do not write payloads into URLs, logs or template defaults.

5.1.1.8. Clear Pi and Account State

The operator UI's **Options** > **Clear session** control calls `DELETE /api/state`. It clears the in-memory review, planned attempt, progress, and pending outcome, drops idle device handles, and refuses while a run is active. The redacted `runs.jsonl` audit log is retained; a clear event is appended. Clearing is not a laser command and does not prove or change physical machine state.

Authentication is not implemented; this remains a trusted-LAN service, so session clearing is not logout. A future sign-in/access boundary must be implemented independently and is not enforced today.

5.1.1.9. Tests

Power and audio sweep previews composite the same positioned job rasters used for execution; line length and pixel rounding are not drawn by a separate schematic. Per-job powers and bed placement remain in the goal. Preview pixels do not simulate burn darkness or prove physical

placement. Basic test dimensions are checked against reach before allocation, and QR-test goals omit payload text so audit logs do not retain it.

`cargo test --workspace` includes real-HTTP simulator tests for both new types, asset retrieval, front/back separation, stale review handling and manual flip gating. `app/tests/ui.cjs` exercises the generic template editor and side controls using synthetic data. Safari/Firefox rendering still needs separate checks.

5.1.2. K6 MCP Preview Client

This is a networked Streamable HTTP MCP server for the current Rust API. It runs separately from the laser service on a trusted operator host and never owns the serial port. The current Deck-local instance calls the Pi API over Wi-Fi and binds MCP to Deck loopback for VS Code.

5.1.2.1. Scope

Tools exposed:

- `get_machine`: current machine profile.
- `list_materials`: configured pieces, backings and settings.
- `list_templates`: declared card templates.
- `preview_job`: calls `POST /api/jobs/preview` and returns layout/stage geometry only.
- `get_updates`: polls state and new redacted run-log entries using an integer `cursor`; pass `next_cursor` on the next call.
- `get_audio_analysis`: returns analyzer availability and the latest timing/report metadata event for an attempt. It never returns audio bytes or report HTML.

All MCP tools are read-only. There is no push-notification path in this integration; `get_updates` is cursor polling. The operator UI remains the place to inspect exact pixels, start work and record the physical verdict. No MCP tool can create a review, plan, run, stop, or record a verdict.

5.1.2.2. Run

Build and run the MCP container on a trusted host. When running on the Deck, point it at the Pi API and bind the published port to loopback; a Pi-hosted deployment can use the Pi-local API address instead:

Build and start the network MCP service

```
docker compose -f docker/docker-mcp/compose.yaml up -d --build
```

For a Deck-hosted service, set `LASER_API_BASE_URL=http://PI-HOST:8081` and `MCP_BIND_ADDRESS=127.0.0.1` in the Compose environment. The workspace's `.vscode/mcp.json` connects to `http://127.0.0.1:8082/mcp`.

The host bind is port 8082. Neither the MCP service nor the Rust API authenticates clients; keep them on a trusted network and restrict non-loopback binds with firewall/TLS controls.

After a successful optional audio upload, the Rust API appends an `audio_analysis` event containing only the attempt ID, timing values and same-origin report URL. `get_audio_analysis` exposes that metadata and analyzer availability, not the recording or HTML report.

5.1.2.3. API

The canonical API guide is `docker/docker-wainlux-rs/README.adoc`, served at `GET /api`. The OpenAPI 3.1 spec is served at `GET /openapi.yaml`; a Swagger UI can load that URL.

5.1.3. Bytefield-SVG

The container renders checked-in EDN packet specifications as SVG.

From the repository root, run `./scripts/generate_diagrams.sh`. The script builds the container when needed and regenerates all packet diagrams.

Packet sources and rendered files live in `documentation/images/`.

See the included Images appendix for K6 protocol packet specs.

6. Scripts

Repository maintenance and documentation utilities.

6.1. Commands

Run these commands from the repository root:

Command	Purpose
`./scripts/build_laser.sh`	Test the CLI examples and Rust service workspace, then build the ARMv6 image. Does not deploy.
`./scripts/deploy_laser.sh --target USER@PI-HOST --confirm`	Transfer the image and recreate an idle Pi service. Never starts a laser job.
`./scripts/analyze_laser_audio.sh RECORDING.m4a`	Analyze DTMF and candidate audio activity in a container. No laser control.
`./scripts/test_docs.sh`	Check AsciiDoc rendering, links, captions and PlantUML attribution.
`./scripts/generate_diagrams.sh`	Regenerate protocol SVGs from the EDN sources.

Build details: [build-and-test-the-rust-service]. Audio analysis: Burn Timing.

6.2. Steam Deck Flatpak Host Commands

Flatpak VS Code cannot access host tools directly. Use `host-spawn` for host commands:

```
$ host-spawn <COMMAND>
```

Use `flatpak-spawn --host` for piped data such as image transfer. Repository scripts select it first and fall back to `host-spawn` through `scripts/lib/container.sh`.

The build command does not transfer or deploy the image. Deployment requires an explicit target and confirmation. Audio analysis reports candidate sound activity; it does not prove laser motion or physical rest.

7. Card CLI Client

`wainlux-client` only submits content to the laser service for operator review. It does not generate PDFs, QRs or images, embed fonts, plan, run, flip, or judge a burn, change machine mode, or write NFC tags. The Pi owns templates, fonts, QR encoding, placement and exact raster generation.

7.1. Build and Test

Run from the repository root with Rust/Cargo installed, or use `./scripts/build_laser.sh` for containerized tests and the service image build.

Build and test the CLI

```
cargo test --locked --manifest-path cli/Cargo.toml
cargo build --release --locked --manifest-path cli/Cargo.toml
```

Tests use synthetic data and a local API without serial access. They verify all three examples, text-file input, front/back separation, and review-only submission.

7.2. Submit One Example

Choose one command below and replace `PI-HOST` with your explicit service host. Submission replaces an idle review; do not replace a review you still need. All examples use fictional data and `guide=false` to reuse existing backing marks. For a new backing, set `guide=true` and perform its separate markings stage with the card removed.

Submit a synthetic Wi-Fi card

```
cli/target/release/wainlux-client submit --server http://PI-HOST:8081
cli/inputs/wifi.json
```

Submit synthetic GitHub codes from a file, with a login QR on the back

```
cli/target/release/wainlux-client submit --server http://PI-HOST:8081
cli/inputs/github.json \
  --text-file recovery-qr=cli/inputs/synthetic-codes.txt
```

Submit a fictional vCard from a file, with the prepared tiger image on the back

```
cli/target/release/wainlux-client submit --server http://PI-HOST:8081
cli/inputs/vcard.json \
  --text-file contact-qr=cli/inputs/contact.vcf
```

Open the service UI, review the exact pixels and placement, and start each stage yourself. For two sides: finish and inspect the front, record the verdict, confirm physical rest, flip into the same guide, confirm placement and start the back separately. Submission never starts a burn.

Input JSON is the service's `POST /api/jobs/review` request, not another client-side template format. Repeat `--text-file ELEMENT=FILE` to replace a declared QR/text item's text with a UTF-8 file. No truncation or formatting is applied. The success receipt reports server raster-generation and preview-encoding time plus client round-trip time; these boundaries overlap and should not be added as disjoint stages. Use a codes-only file for the recovery QR. Keep private inputs in ignored `cli/private/`, not tracked examples; the root `.gitignore` owns these rules. The CLI writes no generated outputs. Keep login passwords separate from recovery codes.

Network submission uses the explicit URL and the service's trusted-LAN boundary; HTTP is not encrypted. Redirects are not followed. Errors withhold server response bodies and never print file payloads. If a request times out, check the UI before resubmitting: the server may already have accepted it.

NFC writing/read-back, recovery and custody: NFC Cards and Recovery Backups. API: Laser Service API. The same Copyright and License covers the CLI and main project.

Appendix F: Glossary

A4

A standard paper size of 210 by 297 millimetres.

ACK

Acknowledgement byte (`0x09`) observed from the K6 after a successful command.

adoc

The usual filename extension for an AsciiDoc document; it is not a separate language.

API

Application Programming Interface; the operations a library or service exposes to other code.

ARMv6

32-bit ARM architecture used by the Pi Zero W.

ASCII

American Standard Code for Information Interchange; a 128-value character encoding covering English letters, digits, punctuation, and control characters.

AsciiDoc

A plain-text markup language for writing structured documents with headings, links, lists, tables, and code examples.

Asciidoctor

A tool that converts AsciiDoc documents into formats such as HTML; PDF support uses an additional converter.

Asset

A supporting file used by the program, such as a prepared logo or Pi-renderer font.

Assumed completion

Software outcome after 90 seconds without a received device byte. It is not device confirmation or an operator verdict.

Attribute

Rust syntax beginning with #[...] that supplies instructions to the compiler or tools, such as derive or test settings.

Authentication factor

One category of evidence: something you know, something you have, or something you are.

Authentication

Checking that someone controls the credentials needed to access an account.

Backup

An additional copy or recovery method kept so that loss of the primary one does not prevent recovery.

Band

A contiguous group of raster rows transferred and burned as one unit. Progress percentages and timing records are band-local.

Base64

An encoding that represents bytes as printable text; it increases size and does not encrypt the content.

Binary file

A file whose bytes are not intended to be read directly as plain text, such as a PNG, font, or executable.

Borrowing

Giving Rust code access to a value through a reference without transferring ownership.

Build

The process of compiling source code and combining it with dependencies to produce an executable.

Byte

Eight bits; the unit used here to measure text storage and NFC capacity.

CAC

Common Access Card; a DoD smart-card credential with protections that a plain NTAG215 backup does not provide.

Calibration envelope

Coordinate range used by an explicit limit test. It may exceed normal job reach and does not establish the mechanical hard stop.

Cargo

Rust's package manager and build tool; it reads the manifest, resolves dependencies, builds the program, and runs tests.

Checksum

A value calculated from data to help detect changes; it is not encryption and by itself does not prove who created the data.

clap

The Rust library the CLI uses to parse arguments and generate help text.

CLI

Command-Line Interface; a program used through commands and text in a terminal.

Clipboard

Temporary storage used for copied text or images; recovery codes placed there can become another exposed copy.

Code

Instructions that define a program's behavior; in the phrase recovery code, code instead means a secret value.

Compiler

A tool that translates source code into executable machine instructions and checks language rules.

CP2102

USB-to-serial bridge family observed in the K6 connection; Linux exposes the serial device to the service.

Crate

A Rust compilation unit, such as a library or binary; a Cargo package can contain more than one crate.

Ctrl-C

A terminal key combination that normally interrupts a running program; cancelling a submission does not prove that the server did not receive it.

Ctrl-D

A Unix terminal key combination that signals the end of input on an empty line; it is not a character stored in the payload.

Data

Values that a program processes, such as instructions and recovery codes, rather than the program itself.

Dependency

Another package or library that the program relies on.

Depth

K6 laser on-time setting per pixel. The supported setting range is 1-255; higher values increase exposure.

Docker

Container runtime used to package the service. Compose maps the serial device into the container.

DoD

The US Department of Defense, which issues the CAC credentials used as a security comparison in the NFC guidance.

DRY

Don’t Repeat Yourself; keep a shared rule in one place instead of maintaining several copies.

Encoding

Representing data in a particular form so it can be stored or transferred; encoding is not secrecy.

Encryption

Protecting data with a cryptographic key so that it cannot normally be read without the corresponding decryption secret.

Enum

A Rust type with a fixed set of named alternatives, such as the supported NFC tag types.

EOF

End Of File; a reader’s signal that no more input is available, including completion of terminal input.

Error propagation

Passing an error back to a caller; Rust’s question-mark operator performs this early return for a failing Result.

Executable

A built program that the operating system can run; on Linux it need not have a filename extension.

Exit status

A number returned when a process ends; zero normally means success and nonzero signals failure.

File extension

A suffix such as .rs or .pdf that conventionally identifies a file's format; it is not proof of validity or safety.

FIPS

Federal Information Processing Standards; referencing these standards does not certify this project or its tag.

Font

Data describing how characters are drawn; the Pi renderer owns card fonts, not the CLI.

Git

A version-control tool that records changes to project files.

Gitignore

Rules in a .gitignore file that keep matching untracked files out of normal Git staging; they do not encrypt files or remove tracked copies.

Hash

A one-way value derived from data; storing only a hash of recovery codes would not preserve the original codes for recovery.

HTML

HyperText Markup Language; a format that structures web pages and rendered documentation.

I/O

Input and output, such as reading terminal text or writing a file.

Include directive

An AsciiDoc instruction that inserts another document's content during rendering.

K6

Wainlux K6 laser engraver controlled over USB serial. The configured normal working profile is 80 x 75 mm; this is not a measured mechanical stop.

KISS

Keep the design simple; for example, line-by-line feedback instead of a live terminal editor.

Library

Reusable code called by a program, such as the QR or PDF library.

Lockfile

Cargo.lock, the generated record of resolved dependency versions and checksums; it does not lock files or the NFC card.

Macro

A Rust construct that generates code during compilation; include_bytes! embeds a file's bytes into the built program.

Manifest

Cargo.toml, the TOML file describing the Rust package and its build dependencies.

Metadata

Information describing something else, such as a package version or an NDEF record length; recovery codes themselves are payload data.

MFA

Multi-Factor Authentication; authentication using at least two different factor categories.

Module

A named part of a Rust program that groups related code.

Monospaced

A font style with a fixed character advance, useful for aligning codes and estimating line width.

NDEF

NFC Data Exchange Format; the record format used to put payloads on a tag.

NFC

Near-field communication used by compatible contactless cards such as the measured NTAG215 card.

NIST

The US National Institute of Standards and Technology; it publishes the digital identity guidance referenced in the NFC documentation.

NTAG215

The measured card's NFC tag, with 504 bytes of user memory and optional password access protection, not payload encryption.

Octal

Base-eight notation used for Unix permissions; 0600 permits owner read/write and 0700 also permits owner directory access.

Operator verdict

Human `ok`, `deviation`, or `skipped` record for a completed or unperformed plan. It is distinct from the software outcome.

Option

A Rust type representing a value that is present with Some or absent with None.

Ownership

Rust's rules for which binding owns a value and when its memory is released.

Package

A Cargo project defined by a manifest, containing source code and one or more build targets.

Passphrase

A secret made from words or other text; it can serve as a password or an encryption secret.

Password

A secret used to authenticate or unlock something; keep the login password out of the recovery-code backup.

Payload

The actual content before format headers are added, such as Wi-Fi data, vCard text or recovery codes.

PDF

Portable Document Format; a fixed-page format available for the project documentation, not generated by the card CLI.

Permission

An operating-system access rule for a file or directory; permissions are not encryption.

Phishing

Tricking someone into disclosing credentials or using a counterfeit login service.

Pi Zero W

Raspberry Pi Zero W, an ARMv6, 512 MB Wi-Fi host for the service.

PIN

Personal Identification Number; a numeric access secret whose protection depends on the device, not merely on calling it a PIN.

Plaintext

Data that can be read without decryption, even when represented as a QR or stored in an image file.

Plan

A record of the reviewed stage and expected goal. Planning does not start machine motion.

PlantUML

A language and rendering tool for describing diagrams in text.

PNG

Portable Network Graphics; a lossless pixel-image format used for QR exports and workflow illustrations.

Point

A typographic unit equal to one seventy-second of an inch; PDF font sizes use points rather than millimetres.

Print queue

Pending print-job data, which can retain a copy of the recovery-code page.

QR quiet zone

Blank margin around a QR symbol that helps scanners locate and decode it.

QR

Quick Response code; a two-dimensional barcode containing data that a scanner can decode, not an encryption method.

Raster

One-bit image represented as rows of pixels; K6 transfer data packs eight pixels per byte.

Read protection

A tag setting requiring an access secret before reading protected memory; protecting only writes does not hide codes.

Recovery code

A service-issued secret, usually usable once, that helps regain access when the normal authentication method is unavailable.

Reference

A Rust value that points to borrowed data, written with an ampersand in types such as &str.

Rendering

Turning a description into an image or document, such as PlantUML into PNG or AsciiDoc into HTML.

Result

A Rust type representing either success with Ok or failure with Err.

Review

Composed job with layout and exact per-stage rasters. Creating or editing a review does not start the laser.

RF shielding

Material that reduces radio-frequency communication through the holder; it does not encrypt the tag or protect an exposed QR.

Rust

A compiled programming language whose ownership and type rules help prevent many memory-safety errors; it does not automatically secure the data handled by a program.

Secure erasure

Making stored data unrecoverable using an appropriate sanitization method; ordinary file deletion is not a guarantee of this.

Source code

Human-readable program instructions, stored here in .rs files before compilation.

Stage

One separately reviewed and explicitly started unit of a card workflow, such as a guide, front, or back.

Struct

A Rust type that groups named fields, such as the CLI's argument values.

SVG

Scalable Vector Graphics; an XML-based image format whose shapes remain sharp when scaled.

Terminal

A text interface to a shell and running programs.

Test

Code that exercises behavior and checks expected results; the CLI's tests use synthetic data.

TLV

Type, Length, Value; the envelope around the NDEF message in a Type 2 NFC tag, whose bytes also consume storage.

TOML

Tom's Obvious Minimal Language; a structured configuration format used by Cargo and service catalogs.

Trait

A Rust contract describing operations that a type provides, such as buffered reading or writing.

TTF

TrueType Font; the format of the embedded font asset.

Unicode

A standard assigning codes to characters from many writing systems; one visible character can involve several Unicode scalar values.

UTF-8

A byte encoding of Unicode; ASCII characters use one byte, while other characters can use several.

Vector image

An image represented by shapes rather than a fixed grid of pixels, such as an SVG QR.

Working reach

Configured normal job rectangle. For the current K6 profile it is 80 x 75 mm; the mechanical end stop has not been measured.

XML

Extensible Markup Language; a structured text format used to describe SVG elements.

Appendix G: Copyright and License

One root license covers project-owned code, documentation, examples and illustrations: personal, non-commercial use and learning only, with no AI/ML training, fine-tuning, evaluation or dataset collection without separate permission. This is a custom source-available license, not an OSI-approved open-source license. Obtain legal review before relying on its enforceability.

Third-party dependencies, bundled fonts used by the Pi renderer, and other third-party assets retain their existing licenses and notices. This change cannot revoke earlier MIT or Creative Commons grants for copies already released under those terms, nor relicense contributions without the necessary rights.

Full project license

```
Wainlux Pi Personal Non-Commercial and No-AI-Training License
Version 1.0, 12 November 2026

Copyright (c) 2026 Sean Donnellan

1. Scope

This license covers the copyright holder's code, documentation, examples,
diagrams and other original material in this repository (the "Material").
Third-party components and assets retain their own licenses and notices;
this license does not replace them or claim rights over them.

2. Permission and conditions

You may use, study, copy, modify and redistribute the Material solely for
personal, non-commercial use and non-commercial learning, subject to all
conditions below. Keep this license and the copyright notice with every
copy or substantial portion, identify modifications, and distribute the
Material and modifications under these same terms. You may not grant rights
to the Material that this license withholds.

3. Commercial use is not permitted

You may not sell, resell, commercially license, or use the Material in a
business or paid service, or use it to produce goods, cards or backups for
sale or resale, without the copyright holder's separate written permission.

4. AI and machine-learning use is not permitted

Without the copyright holder's separate written permission, you may not use
the Material or modified versions to train, pre-train, fine-tune, distill,
or evaluate artificial-intelligence or machine-learning models, or include
them in datasets for those purposes. You may not scrape, crawl, collect,
```

```
index or scan the Material for such AI or machine-learning use, or supply
it to others for those purposes. The copyright holder may separately
authorize AI-assisted development; that authorization is not general
permission to train models on this repository.

5. Prior grants and statutory rights

This license does not withdraw permissions already validly granted for
earlier copies under earlier licenses, or restrict rights that applicable
law does not permit the copyright holder to restrict. No permission is
granted to use third-party trademarks or to imply endorsement.

6. Termination

Permission under this license ends automatically if you breach its terms.
Other permission must be obtained separately from the copyright holder.

7. No warranty or liability

THE MATERIAL IS PROVIDED "AS IS" AND "AS AVAILABLE", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR IMPLIED, INCLUDING MERCHANTABILITY, FITNESS FOR A PARTICULAR
PURPOSE, NON-INFRINGEMENT, SECURITY, ACCURACY OR SUCCESSFUL RECOVERY. TO THE
FULLEST EXTENT PERMITTED BY LAW, THE COPYRIGHT HOLDER IS NOT LIABLE FOR ANY
CLAIM, DAMAGE OR OTHER LIABILITY ARISING FROM THE MATERIAL OR ITS USE,
INCLUDING LOST DATA, ACCOUNT COMPROMISE, EQUIPMENT DAMAGE OR INJURY. NOTHING
HERE EXCLUDES LIABILITY OR RIGHTS THAT CANNOT LAWFULLY BE EXCLUDED.
```

G.1. Crawler Requests

The root `robots.txt` requests that all crawlers stay out; published HTML also carries `noindex, nofollow, noarchive` metadata from `docinfo.html`. There is no universally enforced no-AI-scan file. These requests are advisory, not authentication, encryption or technical prevention of scraping. The license states the permission boundary; it cannot force a bot to comply or remove material already collected.

`robots.txt` must be served at the hosting origin's `/robots.txt`. A file at a GitHub repository URL or a project Pages subpath does not control `github.com` or the whole `github.io` origin. The docs workflow copies the file into the site output, but origin-root configuration requires control of that host. HTML metadata applies to the rendered project page for crawlers that honor it. Use private hosting or access control when actual exclusion is required.

G.2. Clean Room Statement

This project was developed through clean-room reverse-engineering methods. No vendor source code, proprietary binaries, or confidential materials were used or redistributed.

See Clean-Room Statement for methodology and exclusions.

G.3. Copyright Notice

Copyright (c) 2026 Sean Donnellan